

MC ARTWORKS STUDIO

TECHNICAL DOCUMENTATION

WordPress and WooCommerce project with a multimedia system and dynamic interface

Developed by · Gaston Rizzo

Table of Contents

Section	Page	Section	Page
1. Overview	3	21. Cart page and customization	68
2. How the project works	3	22. Session and data persistence	71
3. Technologies, plugins, and dependencies	4	23. Checkout: final purchase form and validations	73
4. WordPress: what it is and how it powers MC Artworks Studio	8	24. Manual payment method using PayPal	77
5. Database and information model	10	25. Order creation	79
6. Project structure	15	26. Logs: recording orders and emails	81
7. What WordPress, Blocksy, and WooCommerce do	17	27. Buyer email	84
8. Initial sequence and animations	18	28. Administrator email	89
9. Multimedia introduction and HLS	23	29. Email delivery and error tracking	92
10. Faction selection	29	30. Order Received and complete example	94
11. Cookies and NOD / GDI appearance	32	31. Gallery	99
12. Main menu	35	32. Services section (Our Services)	109
13. Home	39	33. JavaScript and dynamic page changes	112
14. How products are managed with WooCommerce	44	34. Browser adaptation	115
15. Custom fields with ACF	47	35. Multimedia resources and external services	118
16. Shop and product cards	48	36. Cache and performance	120
17. Progressive video loading and Fancybox	52	37. Data handling and security	122
18. Shop filters with HUSKY and AJAX	56	38. Project installation, migration, and verification	125
19. Bundles	59	39. Possible development directions	130
20. Side cart and item limit	64	40. Technical scope and conclusion	133

PART I - PROJECT AND ARCHITECTURE

1. Overview

MC Artworks Studio is a WordPress site designed to sell digital artworks and offer image editing, artwork creation, and realistic photomontage services. Built on WordPress and WooCommerce, it features an audiovisual interface inspired by video game aesthetics: visitors watch an introduction, choose a faction, and then browse the site using the visual theme associated with that choice.

The child theme changes much of the standard WordPress and WooCommerce presentation. The Shop displays videos as product previews and includes filters, bundles, a side cart, customization options, a streamlined checkout, and a manual payment method. When an order is confirmed, the project writes a record to a log file and prepares separate HTML emails for the buyer and the administrator.

Scope of this documentation

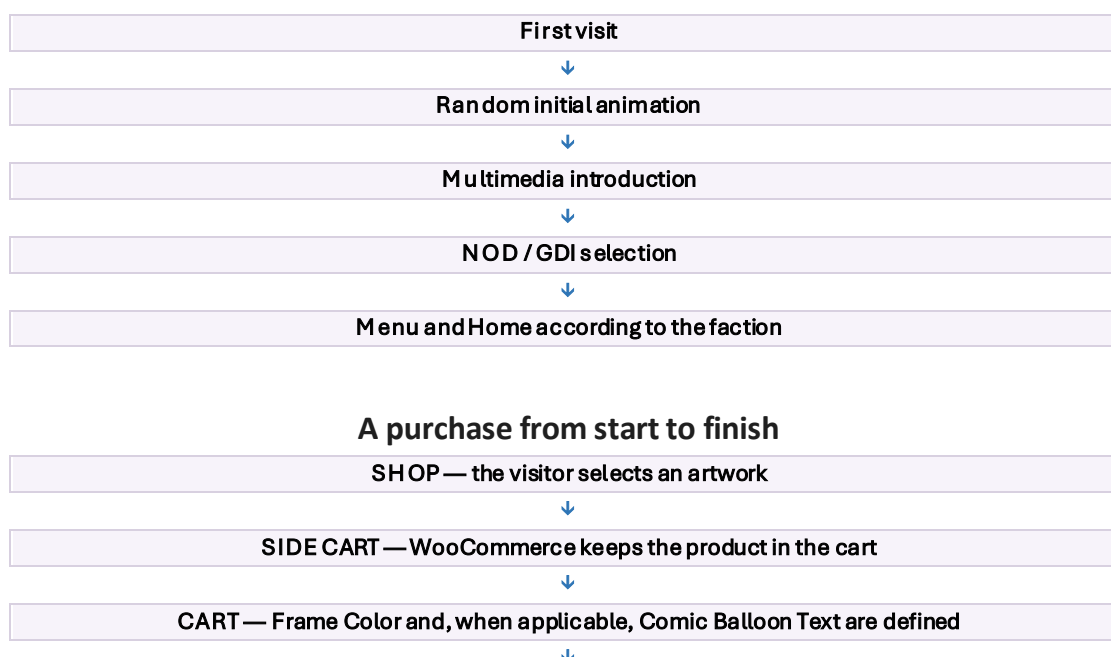
This documentation describes the architecture, operation, and main technical decisions of MC Artworks Studio. It explains how WordPress, Blocksy, WooCommerce, and Advanced Custom Fields are integrated; how products and bundles are organized; how multimedia resources, the cart, and checkout are handled; and how system data is stored and processed. It also covers the project structure, database, dependencies, purchase flow, order management, email delivery, and operation logging.

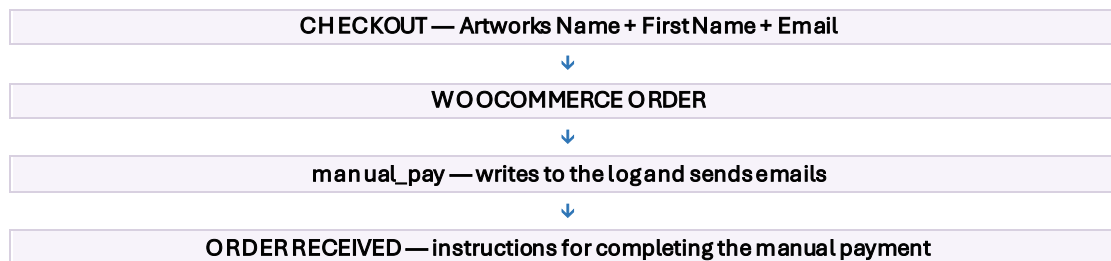
What the project sells and showcases

- Digital artworks represented as WooCommerce products.
- Bundles (packages) that group several related artworks.
- Image editing, artwork creation, and realistic photomontage services.
- A multimedia experience with videos, music, entry animations, and two visual themes: NOD and GDI.

2. How the project works

The simplest way to understand the project is to follow two paths: a first visit and a purchase. WordPress receives the requested URL and loads the child theme. When the visitor enters the Shop, Cart, Checkout, or an order page, WooCommerce manages the products and the steps in the purchase process.





Example reused throughout the document

To explain the flow, a fictional example will be used: Alex buys “Cyberpunk Portrait” for USD 18, selects Frame Color = Blue, Comic Balloon Text = “Welcome back”, and enters Artworks Name = “Raven”. The data is illustrative; the structure reflects the information handled by the code.

3. Technologies, plugins, and dependencies

MC Artworks Studio combines WordPress with a parent theme, its own child theme, WordPress plugins, JavaScript libraries, and external services. These components do not all serve the same purpose: some form the base platform, others add capabilities to WordPress, and others are loaded from the Internet when the interface needs them. Separating them by type helps clarify which part of the system each component addresses and what depends directly on the blocksy-child code.

3.1 Platform, core technologies, and libraries

These technologies support the project and provide the foundation for the custom code. Their role is summarized here; later sections explain their integration in greater detail.

Technology	Role within MC Artworks Studio
WordPress	It is the CMS on which the site runs. It manages pages, loads themes and plugins, and provides hooks and functions such as <code>wp_mail()</code> and <code>wp_upload_dir()</code> .
PHP	Runs on the server. The child theme uses it to query products, decide what content to load, and generate HTML, CSS, or JavaScript depending on the page and faction.
Blocksy	It is the parent theme. It provides a WordPress base that blocksy-child modifies without directly editing the original Blocksy files.
JavaScript	Runs in the browser. It controls videos, audio, modals, animations, AJAX, DOM observers, and much of the interface interaction.
jQuery	It is a JavaScript library used primarily to integrate with WooCommerce and with plugins that dynamically update elements.
MariaDB / MySQL	This is the database system. The installation uses MariaDB 11.8.6 to store WordPress and WooCommerce products, configuration, sessions, and orders.

3.2 WordPress plugins used

A WordPress plugin is an extension installed within the platform to add functionality that does not belong to the theme. MC Artworks Studio uses several plugins as infrastructure and then adapts their appearance or behavior from blocksy-child. The child theme does not replace these tools: it builds on them and adds its own logic on top.

3.2.1 WooCommerce

WooCommerce is the e-commerce plugin that supports the entire sales flow. It manages products, prices, the cart, sessions, checkout, orders, and the payment-method framework. Artworks and bundles remain real WooCommerce products even though their visual presentation is fully customized by the child theme.

For example, when the visitor clicks ADD TO CART, blocksy-child can validate the item limit and change the button appearance, but the product is still added to the real WooCommerce cart and later becomes part of a WC_Order.

```

Artwork
↓
WooCommerce product
↓
ADD TO CART
↓
Cart
↓
Checkout
↓
WC_Order

```

3.2.2 Advanced Custom Fields (ACF)

Advanced Custom Fields (ACF) is a plugin that allows custom information to be added to items managed through WordPress. In this project, it complements WooCommerce products with data that the standard product model does not include.

The fields confirmed in the installation are `video_destacado`, `video_workshop`, `productos_del_bundle`, and `tiene_globo`. The child theme reads these values to determine which video to display, which artworks belong to a bundle, or whether a product supports Comic Balloon Text. Section 15 later explains how these values are read with `get_field()` and how they affect the interface.

```

WooCommerce product: Cyberpunk Portrait
↓
ACF
video_destacado = cyberpunk.mp4
tiene_globo = Yes
↓
Child theme
displays the video and enables Comic Balloon Text

```

3.2.3 HUSKY / WOOF

HUSKY / WOOF is the plugin used as the product-filtering engine in the Shop. It provides the infrastructure needed to apply catalog criteria such as categories, text search, and price range, without requiring the child theme to build the entire filtering system from scratch.

MC Artworks Studio uses that foundation and adapts it. The theme changes text, hides categories that should not appear in Shop, adds a magnifying-glass icon to the search field, requires a minimum of three characters, adjusts the price range, and reapplies certain behaviors after AJAX updates. Section 18 explains this operation in detail.

```

User changes a filter
↓
HUSKY prepares the criteria
↓
AJAX request
↓
WooCommerce retrieves matching products
↓
HUSKY replaces the grid
↓
the child theme reapplies its adjustments

```

3.2.4 Xootix Side Cart

Xootix Side Cart is a WooCommerce plugin that displays cart contents in a side panel. It lets visitors review or remove products without immediately opening the Cart page.

Xootix does not maintain a separate cart. WooCommerce retains the products and processes the operations; Xootix represents that same state through a side interface that can update dynamically. On top of that interface, blocksy-child adds videos, loaders, the global item limit, and synchronization between ADD TO CART and VIEW CART.

```

ADD TO CART
↓
WooCommerce adds the product
↓
added_to_cart event
↓
Xootix updates the side cart

```

3.2.5 WP Mail SMTP

WP Mail SMTP is the plugin used to configure outgoing WordPress email through an SMTP server or service. It does not create the project emails: the buyer and administrator HTML content is built in `blocksy-child` and passed to WordPress through `wp_mail()`.

After that call, WP Mail SMTP routes the request to the configured SMTP service. As a result, an order may exist correctly even if a later email-delivery problem occurs; Sections 27 through 29 explain the message content and error tracking.

```
Child theme → wp_mail() → WordPress → WP Mail SMTP → SMTP server / service → Recipient
```

3.2.6 All-In-One Security (AIOSEC)

All-In-One Security (AIOSEC) is a plugin intended to strengthen the security of a WordPress installation. It is part of the MC Artworks Studio environment reviewed for this documentation, but it does not directly participate in the logic that builds the Shop, cart, checkout, or orders.

The specific configuration of its modules is managed outside the child theme and is not defined within `blocksy-child`. For that reason, this documentation identifies it as an auxiliary security tool without attributing specific controls that have not been verified in the WordPress configuration.

3.3 External libraries and services

In addition to the plugins installed in WordPress, the project uses JavaScript libraries and external services. These dependencies should not be confused with plugins: the theme can load them directly from a URL or use them from the browser.

3.3.1 Fancybox4

Fancybox 4 is a JavaScript library for displaying content in an overlay window, or modal, without leaving the current page. In MC Artworks Studio it is used to enlarge artwork videos from the Shop and allow visitors to move to the previous or next video without returning to the grid.

Fancybox is not installed as a WordPress plugin in this project. `blocksy-child` loads its CSS and JavaScript from `jsDelivr` when Shop, a product category, or a product tag is visited, and then binds the cards with `Fancybox.bind()`. Section 17 explains this integration and the custom modal.

```

User clicks the video
↓
data-fancybox element
↓
Fancybox opens the modal
↓
HTML5 player
↓
Previous / Next / Close

```

3.3.2 HLS.js

HLS.js is a JavaScript library that enables HLS stream playback in browsers that do not provide the required native support. HLS streams are described by `.m3u8` files that point to video segments.

MC Artworks Studio uses HLS in the multimedia introduction and on Home. When the browser can play HLS directly, that native support is used; otherwise, HLS.js connects the manifest to the `<video>` element. Section 9 explains loading, fallback behavior, and feed synchronization.

3.3.3 jsDelivr

jsDelivr is a CDN, meaning a content delivery network used to deliver external files to the browser. In this project it is used to obtain HLS.js and Fancybox. This avoids having to store those libraries inside blocksy-child, although it means they must be downloadable from the Internet when the page needs them.

3.3.4 Google Fonts

Google Fonts is the external service used to load some interface typefaces, including Orbitron and Share Tech Mono. These fonts are part of the visual presentation; they do not store products or participate in the purchase flow.

3.4 Required and auxiliary components

The dependencies do not all have the same level of importance. Some are directly involved in commercial operation, while others provide visual, email-transport, or security support.

Component	Role within the project	Dependency type
WordPress	Base platform on which the entire project runs.	Required
Blocksy	Parent theme required for blocksy-child to work as developed.	Required
WooCommerce	Foundation of the catalog, cart, checkout, and orders.	Required
ACF	Provides custom product data used directly by the theme.	Required in the current implementation
HUSKY / WOOF	Filter engine used by Shop.	Required for the current filters
Xootix Side Cart	Side-cart interface; WooCommerce retains the actual cart.	Required for the current side cart
WP Mail SMTP	Routes emails to the configured SMTP service.	Required for the documented SMTP setup
AIOS	Auxiliary security tool; it is not part of the main commercial flow.	Auxiliary
Fancybox 4	External library used by the enlarged Shop video view.	Required for the video modal
HLS.js	External library used for HLS playback when the browser requires it.	Conditional
jsDelivr	CDN from which HLS.js and Fancybox are loaded.	Required while it is used as the CDN
Google Fonts	External font service.	Visual / auxiliary
MariaDB	Storage for WordPress, WooCommerce, and site configuration.	Required in the current environment

Important distinction

Fancybox and HLS.js are not WordPress plugins in MC Artworks Studio. They are JavaScript libraries that the theme loads as external dependencies. Likewise, Blocksy is a parent theme, MariaDB is the database, and jsDelivr is a CDN. This separation avoids treating all system components as if they were the same type of component.

3.5 Server and browser

PHP and WordPress run on the server before the page reaches the visitor. HTML, CSS, and JavaScript are executed or interpreted in the browser. This distinction matters: for example, PHP decides which CSS should be loaded, while JavaScript opens the menu, plays videos, or detects when an element enters the visible area.

4. WordPress: what it is and how it powers MC Artworks Studio

Before examining Intro, Shop, or Checkout, it is useful to understand the environment that executes all of the code. WordPress is not a secondary library: it is the platform that receives the request, loads themes and plugins, fires hooks, and ultimately produces the response received by the browser.

4.1 What happens when Alex opens a URL

Step	What happens	Participants
1	Alex requests /shop/ from the browser.	Browser + web server
2	The server passes the request to WordPress.	WordPress Core
3	WordPress loads configuration, active plugins, and the theme.	Core + plugins + Blocksy + blocksy-child
4	WooCommerce recognizes the commercial context and prepares its query.	WooCommerce
5	Hooks registered by blocksy-child modify the query, HTML, CSS, or JavaScript.	Custom code
6	PHP finishes and the server sends HTML/CSS/JS to the browser.	Server
7	The browser creates the DOM and executes JavaScript, including subsequent AJAX changes.	Browser + JS plugins

Key idea: WordPress acts before Alex sees the screen. After receiving the response, the browser continues the work with JavaScript. This is why the same MC Artworks feature can begin in PHP and end in a DOM listener.

4.2 WordPress Core, the theme, and plugins are not the same thing

Layer	What it provides in MC Artworks	What should not automatically be attributed to this layer
WordPress Core	Loads pages, themes, and plugins; provides hooks, conditional tags, wp_mail(), wp_upload_dir(), and admin-ajax.php.	It does not contain the NOD/GDI experience or the project-specific commercial logic.
Blocksy	Parent theme and base WordPress structure.	Does not contain the child theme's customizations.
blocksy-child	Project-specific code: HUD, factions, videos, custom Shop, Cart, gateway, logs, Gallery, and Services.	It does not replace WordPress or create a parallel WooCommerce.
WooCommerce	WC_Product, catalog, cart, session, Checkout, WC_Order, and gateways.	Does not independently decide the videos, posters, or NOD/GDI identity.
ACF / HUSKY / Xootix	Metadata, filters, and side cart.	They are not the source of truth for the product or cart.

4.3 The request lifecycle and hooks

A hook is a specific point in the WordPress or WooCommerce lifecycle at which custom code can run. The theme or plugin registers a function for that hook and, when WordPress reaches that point, automatically calls the function.

In this example, `add_action()` registers the `insertar_presentacion()` function for the `template_redirect` hook:

```
add_action('template_redirect', 'insertar_presentacion');
```

When WordPress loads `functions.php`, it registers that relationship, but it does not execute `insertar_presentacion()` yet.

Later, when WordPress reaches the `template_redirect` hook, it automatically calls the `insertar_presentacion()` function.

Hook / point in the lifecycle	Verified use in MC Artworks
<code>wp_enqueue_scripts</code>	Loads CSS/JS such as Fancybox and the faction styles.
<code>wp_head</code>	Activates components such as <code>hud_intro()</code> and other blocks that output content/code.
<code>template_redirect</code>	Allows the initial presentation to be decided before the final template loads.
<code>pre_get_posts</code>	Modifies the main Shop query to exclude Bundles.
<code>wp_footer</code>	Place where several blocks output JavaScript that will run in the browser.
Hooks WooCommerce	Insert or modify data during Shop, Cart, Checkout, and order creation.

4.4 Actions and filters

An action specifies which function should run when WordPress or WooCommerce reaches a particular point in its lifecycle. For example:

`add_action('wp_footer', 'mi_script');` In this case, `mi_script()` is registered to run when WordPress reaches the `wp_footer` hook.

A filter, by contrast, receives a value that WordPress or WooCommerce is processing, allows that value to be modified, and must return the result so the system can continue working with the modified version. For example:

```
add_filter(
    'woocommerce_checkout_fields',
    'simplificar_checkout_nombre_y_correo'
);
```

4.5 Priority: multiple callbacks can share the same hook

The third argument of `add_action()/add_filter()` can set a priority. A lower number runs first. If two callbacks have the same priority, their registration order also matters. This detail is relevant to the project because there are two faction-CSS callbacks with priority 999 and the same handle; the caching section analyzes that case without assuming that the second automatically replaces the first.

4.6 Conditional tags: determining which screen is being processed

Functions such as `is_shop()`, `is_cart()`, `is_checkout()`, and `is_page()` ask questions about the current context. In a `functions.php` file that concentrates many features, these checks prevent a block intended for Shop from altering Cart or Gallery.

```
if (!is_shop()) {
    return;
}
```

```
// From this point on, the block continues only when WordPress/WooCommerce
// is processing Shop.
```

4.7 Server-side PHP vs. browser-side JavaScript

MC Artworks combines PHP with HTML, CSS, and JavaScript fragments. PHP runs on the server. When functions.php echoes a `<script>`, PHP does not execute `document.addEventListener()`; it only writes that text into the response. The browser receives it later and only then executes the JavaScript.

Server	HTTP boundary	Browser
PHP / WordPress decides and generates content.	HTML + CSS + JavaScript travel in the response.	The DOM is created and events, observers, audio, video, and AJAX run.

4.8 admin-ajax.php: WordPress infrastructure, custom callback

When the child theme sends an AJAX request to `admin-ajax.php`, the URL belongs to WordPress. The action and callback that respond to it may belong to MC Artworks. This can be seen in the Xootix synchronization: a click generated by the plugin ultimately calls WordPress infrastructure and then custom code.

4.9 WordPress and plugin “noise”

When inspecting a page, not all visible DOM, CSS, or JavaScript comes from blocksy-child. WordPress, Blocksy, WooCommerce, HUSKY, and Xootix also generate elements or replace them dynamically. This is a common source of confusion during debugging.

Visible element	Expected primary source
Base site structure	WordPress + Blocksy
Product and price	WooCommerce
Audiovisual card preview	blocksy-child + ACF
Filtering controls	HUSKY/WOOF + custom adaptations
Side cart	Xootix + WooCommerce + custom synchronization
Cart / Checkout	WooCommerce + blocksy-child hooks
Intro / Faction / HUD	Primarily blocksy-child

4.10 How to reason through a screen when debugging

When an unexpected element appears, it is useful to ask, in order: was it generated by WordPress/Blocksy? by WooCommerce? inserted by a plugin? output by the child theme? did it appear later through AJAX? Only after identifying its owner should you look for the code. This rule reduces the typical “noise” of a WordPress ecosystem with several active plugins.

Code-reading rule used in this document: whenever the file, function, callback, or execution layer changes, the path and the component containing the fragment are identified, together with what activates it and when it takes part. “Source code” identifies the fragment’s actual origin; it does not imply that WordPress executes that fragment the moment functions.php is loaded.

5. Database and information model

WordPress does not store artworks inside PHP files. The child theme code defines how they are displayed and processed, while products created from the administration panel, their custom fields, cart sessions, and orders are stored in the installation’s MariaDB database. This separation allows the catalog to be changed without editing functions.php every time a product changes.

5.1 What the database stores

The database contains the content managed by WordPress and WooCommerce as well as part of the configuration needed to interpret it. In MC Artworks Studio it stores products, categories, attachments, ACF fields, site options, temporary cart sessions, and orders.

```
Child theme PHP code
  ↓ defines behavior
WordPress + WooCommerce + ACF
  ↓ manage content
MariaDB
  ↓ stores products, metadata,
    sessions, orders, and configuration
```

5.2 Relevant installation tables

Table / group	Function in MC Artworks Studio
wp_posts	Products, pages, attachments, and internal WordPress/ACF definitions.
wp_postmeta	Product metadata, ACF references, and attachment metadata.
wp_terms · wp_term_taxonomy · wp_term_relationships	Categories and the relationship between each product and its categories.
wp_options	General WordPress, WooCommerce, theme, and plugin configuration.
wp_woocommerce_sessions	Temporary session with which WooCommerce reconstructs the cart.
wp_wc_orders · wp_wc_order_addresses · wp_wc_orders_meta	HPOS orders, buyer data, and general order metadata.
wp_woocommerce_order_items · wp_woocommerce_order_itemmeta	Order lines and per-product metadata, such as Frame Color and Comic Balloon Text.

The installation uses the wp_ prefix. Another WordPress installation may use a different prefix. The theme does not create custom tables for MC Artworks Studio and works mainly through the WordPress, WooCommerce, and ACF APIs.

5.3 Why there are no artworks or bundles tables

Registering and managing a product with WooCommerce does not create a table named artworks. There is no custom bundles table either. Both are represented as standard WooCommerce products and are distinguished by their categories and metadata.

```
There is NO custom table: artworks(id, name, price, video, ...) Instead: wp_posts └─ post_type = product └─
WooCommerce/ACF metadata + taxonomies
```

5.4 How an artwork is represented

The database makes it possible to trace a real product. Example: record 988 corresponds to a published artwork. Although its name contains “VARIATION #1”, WooCommerce classifies it as a simple product: that wording is part of the commercial name and does not represent a technical WooCommerce variation.

```

wp_posts
ID          = 988
post_title  = Geralt #1 | The Witcher | VARIATION #1
post_type   = product
post_status = publish

WooCommerce product type = simple

```

5.5 ACF and wp_postmeta

Advanced Custom Fields adds custom data without creating new columns in wp_posts. Values are stored as related metadata through post_id. The actual database contains four product ACF fields: video_destacado, video_workshop, productos_del_bundle, and tiene_globo.

ACF field	What it represents	Example use
video_destacado	Main video associated with the product.	The Shop and Bundles card uses it as a preview.
video_workshop	Optional Workshop video.	Optional field available for storing a Workshop resource associated with the product.
productos_del_bundle	Relates a bundle to other products.	The bundle card iterates through the associated artworks.
tiene_globo	Indicates whether the artwork supports balloon text.	When enabled, Cart displays Comic Balloon Text.

Bundle identification

Bundles are identified through the Bundles category and use productos_del_bundle to relate the artworks that make up each package.

ACF also uses an internal key beginning with “_” to link a value to its field definition. For example, video_destacado contains the file ID, while _video_destacado points to the corresponding ACF field definition.

```

video_destacado = 989
_video_destacado = field_68422c183110a

tiene_globo      = 1
_tiene_globo     = field_6888eb6010b7a

```

5.6 Videos and attachments

MP4 files are not stored inside the product row. video_destacado stores a reference to a WordPress attachment. That attachment keeps the physical file path within wp-content/uploads. The ACF field is configured with return_format = url, so get_field() returns a usable URL to the theme.

```

PRODUCT — wp_posts.ID = 988
↓
wp_postmeta: video_destacado = 989
↓
ATTACHMENT — wp_posts.ID = 989
post_type = attachment · mime = video/mp4
↓
_wp_attached_file / _wp_attachment_metadata
↓
MP4 file inside wp-content/uploads

```

```
$video_url = get_field(
    'video_destacado',
    $product->get_id()
);
```

5.7 Categories and taxonomies

WooCommerce organizes the catalog through WordPress taxonomies. A product can belong to several categories, so the following counts represent relationships and should not be added together to obtain the catalog total.

Category	Related products
Characters	307
Bundles	134
Fantasy	131
Cyberpunk	76
Universes	73
Anime	40

5.8 How a bundle is represented

A bundle is not a WooCommerce “grouped” product. The 134 bundles are simple products that belong to the Bundles category. The ACF field `productos_del_bundle` stores the relationship to the artworks displayed inside its card.

```
PRODUCT SIMPLE
category = Bundles
↓
productos_del_bundle
├── Product #1086
├── Product #1089
└── Product #1092
```

Each bundle currently has three related products, and all 402 references point to existing products. This describes the current catalog composition; the ACF configuration does not impose a technical maximum of three.

5.9 Session and cart

Before an order is created, WooCommerce keeps the cart in its session system. The theme adds `comic_balloon_text` and `color_frame` to each cart item. These data are temporary: they survive page changes, but only become permanent when they are copied to the order.

```
Cart item
├── product_id
├── comic_balloon_text
└── color_frame
    ↓
wp_woocommerce_sessions
    ↓
Checkout
```

5.10 Orders and HPOS

The installation has High-Performance Order Storage (HPOS) enabled: `woocommerce_custom_orders_table_enabled = yes`. Orders are stored in WooCommerce-specific tables. The theme uses `WC_Order` objects and functions such as `wc_get_order()`, so it does not need to query those tables directly.

```
$order = wc_get_order($order_id);
$artworks_name = $order->get_meta('Artworks Name');
```

Checkout



WC_Order / WooCommerce CRUD API

```
├── wp_wc_orders
├── wp_wc_order_addresses
└── wp_wc_orders_meta
```



wp_woocommerce_order_items



wp_woocommerce_order_itemmeta

5.11 Order-level data and line-item data

Artworks Name belongs to the entire order. Frame Color and Comic Balloon Text belong to a specific line item, because two products in the same order can have different customizations.

Data	Level	Persistence
Artworks Name	Entire order	wp_wc_orders_meta
First Name / Email	Billing data	wp_wc_order_addresses / wp_wc_orders
Frame Color	Product line	wp_woocommerce_order_itemmeta
Comic Balloon Text	Product line	wp_woocommerce_order_itemmeta

5.12 Current database composition

Data	Value
Database engine	MariaDB 11.8.6
Tables	74
Published products	627
Product type	627 simple products
Artworks outside Bundles	493
Bundles	134
tiene_globo = Yes / No	55 / 572
Attachments	544 (540 MP4, 3 PNG, 1 WebP)
HPOS	Enabled
Orders stored in HPOS	6
Payment method	manual_pay
Manual-order status	wc-pending
Currency	USD

Data	Value
Content data These figures describe the current contents of the installation and do not represent system limits. The catalog and number of orders can grow without changing this data model.	

5.13 How the theme accesses this data

Unlike a project that queries SQL manually, MC Artworks Studio uses the WooCommerce and ACF APIs. This keeps the code decoupled from physical storage and is especially important with HPOS.

```
$product->get_id();
$product->get_price();
get_field('video_destacado', $product->get_id());
wc_get_products([...]);
wc_get_order($order_id);
```

6. Project structure

MC Artworks Studio is built on a WordPress installation. Custom code is concentrated in the blocksy-child child theme, while WordPress, Blocksy, WooCommerce, the plugins, and wp-content/uploads complete the project's runtime environment.

6.1 General organization

```
public_html/
├── wp-content/
│   ├── themes/
│   │   ├── blocksy/           ← parent theme
│   │   └── blocksy-child/    ← custom code
│   ├── plugins/
│   │   ├── WooCommerce
│   │   ├── Advanced Custom Fields
│   │   ├── HUSKY / WOOF
│   │   ├── Xootix Side Cart
│   │   ├── WP Mail SMTP
│   │   ├── All-In-One Security (AIOS)
│   │   └── other plugins
│   └── uploads/
│       ├── categories/
│       ├── images/
│       ├── videos/
│       ├── songs/
│       ├── sounds/
│       ├── logs/
│       ├── wc-logs/
│       ├── woocommerce_uploads/
│       └── other WordPress/plugin folders
```

The images, videos, songs, and sounds folders are physically located inside wp-content/uploads. The categories folder organizes some videos by category and character; for example, uploads/categories/characters/chainsaw-man/ contains MP4 files associated with that group.

6.2 blocksy-child child theme

File	Purpose
style.css	Declares the child theme and its relationship with Blocksy.
functions.php	Contains most of the logic: Home, Shop, Bundles, cart, checkout, manual payment, emails, logs, Gallery, and the services section.

File	Purpose
header.php	Builds the main HUD-style menu, its videos, sounds, navigation, and audio persistence.
animations.php	Contains eight presentation animations: four GDI and four NOD.
hud-intro.php	Builds the multimedia introduction with two videos, music, and controls.
faction-selection.php	Implements NOD/GDI selection and stores the choice in cookies.
gdi-faction.css / gdi-faction-min.css	Styles for the GDI variant.
nod-faction.css / nod-faction-min.css	Styles for the NOD variant.
scrin-faction.css	Stylesheet present in the theme, although Scrin is not part of the active selection flow.

6.3 Code, data, and resources

Area	Content
wp-content/themes/blocksy-child/	PHP and CSS developed specifically for MC Artworks Studio.
MariaDB	Products, ACF, categories, sessions, configuration, and HPOS orders.
wp-content/uploads/categories/	Multimedia resources organized by categories/characters.
wp-content/uploads/images/	Images used by the interface.
wp-content/uploads/videos/	General site videos.
wp-content/uploads/songs/	Music used by the audiovisual experience.
wp-content/uploads/sounds/	Sound effects.
wp-content/uploads/logs/	Custom logs generated by the manual gateway.

6.4 Concentration of logic in functions.php

MC Artworks Studio does not separate each screen into a page-*.php file. Much of the behavior is activated from functions.php through hooks and context checks such as `is_shop()`, `is_cart()`, `is_checkout()`, and `is_page()`. As a result, pages such as Bundles, Gallery, and Our Services are built through this centralized logic rather than through separate page-bundles.php, page-gallery.php, or page-our-services.php files.

```

WordPress requests a page
↓
functions.php checks the context
↓
is_shop() · is_cart() · is_checkout() · is_page(...)
↓
the hooks, HTML, CSS, and JavaScript
for that screen are executed

```

6.5 Custom code, dependencies, and resources

Project code and configuration	Related environment and resources
Child theme PHP and CSS	WordPress Core and Blocksy
Menu, intro, and faction selector	WooCommerce, ACF, and other plugins

Project code and configuration	Related environment and resources
Custom Shop, cart, and checkout logic	MariaDB database and managed content
GDI/NOD/Scrin styles	wp-content/uploads with videos, images, music, and sounds
Centralized organization functions.php exceeds 8,800 lines and serves as the main integration point for Shop, Bundles, cart, checkout, manual payment, emails, logs, Gallery, and services. This centralization is part of the project's current architecture.	

7. What WordPress, Blocksy, and WooCommerce do

WordPress receives the requested URL and loads the theme. Blocksy provides the foundation on which the site operates, while blocksy-child contains the MC Artworks Studio customizations. WooCommerce handles products, prices, the cart, checkout, and orders.

What parent theme and child theme mean

A parent theme provides a reusable foundation. A child theme makes it possible to modify that foundation without directly editing the parent theme files. In this project, Blocksy is the parent theme and blocksy-child contains the custom code. For example, the audiovisual menu lives in the child theme's header.php; Blocksy's header.php does not need to be modified to build it.

What a hook is

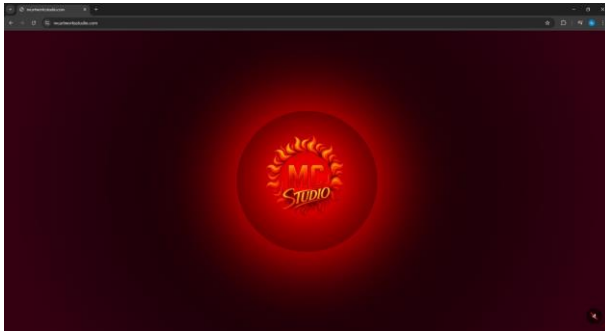
WordPress and WooCommerce provide specific points where a theme or plugin can execute code. These points are called hooks. They allow behavior to be added or modified without editing WordPress or WooCommerce core. For example, the Shop uses a WooCommerce hook to place each artwork's video before the product name.

PART II — ENTRY, FACTIONS, AND NAVIGATION

8. Initial sequence and animations

The initial sequence is a presentation shown before Intro when the visitor has not yet chosen a faction. It is not a single video or an animation loaded from an external file: `functions.php` decides whether it should be shown, and `animations.php` contains eight independent PHP functions that generate the visual experience in the browser.

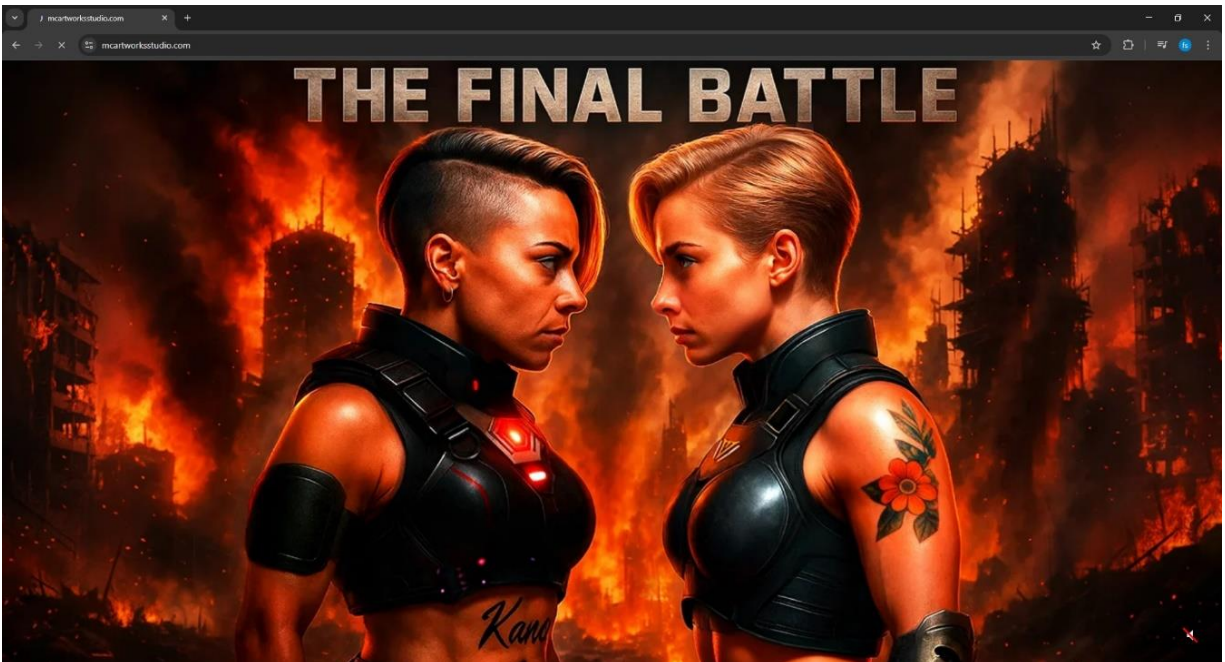
To understand it, it is useful to separate two responsibilities. WordPress and PHP make the decision on the server; the selected animation function outputs HTML, CSS, and JavaScript. That response contains browser-side timers and redirects, but `insertar_presentacion()` also attempts a server-side `wp_redirect()`. For this reason, the effective exit behavior is analyzed in Section 8.8, and it should not be assumed that every timer always gets a chance to execute.



Red variation



Purple variation



Presentation image shown before the multimedia introduction.

8.1 Where the sequence is activated

functions.php registers `insertar_presentacion()` on the `template_redirect` hook. That hook runs before WordPress loads the final template, so the code can still decide whether to let the normal request continue or intervene in the entry flow. The following fragment is abbreviated: it preserves the real activation conditions but omits the full body where the variant is selected, the cover is output, and the transition to Intro is attempted.

```
add_action('template_redirect', 'insertar_presentacion');

function insertar_presentacion() {
    if (is_page(['intro', 'faction-selection'])) return;

    if (empty($_COOKIE['faction'])) {
        if (is_page() || is_single() || is_archive() ||
            (function_exists('is_woocommerce') && is_woocommerce()) ||
            is_shop() || is_product() || is_product_category() || is_product_tag()) {
            // Abbreviated fragment: selection continues here,
            // followed by the visual output and redirection.
        }
    }
}
```

The absence of the faction cookie is required, but it is not the only condition. The function does not act inside `/intro/` or `/faction-selection/` and continues only in the frontend contexts covered by the code: pages, posts, archives, and several WooCommerce screens.

Project location: `wp-content/themes/blocksy-child/functions.php` · `template_redirect` hook around line 220 · `insertar_presentacion()` approximately between lines 230 and 324.

8.2 How functions.php makes the eight animations available

Before `insertar_presentacion()` can choose a variant, the child theme loads `animations.php`. `require_once` includes the file once during that request and defines its functions. Loading the file does not mean executing all of the animations; it only makes their names available so that one of them can be invoked later.

```
if (file_exists(get_stylesheet_directory() . '/animations.php')) {
    require_once get_stylesheet_directory() . '/animations.php';
}
```

```
WordPress loads blocksy-child
↓
functions.php executes require_once animations.php
↓
eight animation functions are defined
↓
template_redirect reaches insertar_presentacion()
↓
only then is one of those functions selected
```

8.3 The eight available functions

`animations.php` contains four GDI variants and four NOD variants. They all share the concept of a fullscreen presentation, but each function builds its own set of layers, effects, sounds, and timing.

Faction	PHP function	Common responsibility
GDI	<code>insertar_animacion_apocalipsis_GDI_UNO()</code>	Generates a fullscreen GDI presentation and schedules its exit.
GDI	<code>insertar_animacion_apocalipsis_GDI_DOS()</code>	Generates a second GDI visual variant.
GDI	<code>insertar_animacion_apocalipsis_GDI_TRES()</code>	Generates a third GDI visual variant.
GDI	<code>insertar_animacion_apocalipsis_GDI_CUATRO()</code>	Generates a fourth GDI visual variant.

NOD	insertar_animacion_apocalipsis_NOD_UNO()	Generates a fullscreen NOD presentation and schedules its exit.
NOD	insertar_animacion_apocalipsis_NOD_DOS()	Generates a second NOD visual variant.
NOD	insertar_animacion_apocalipsis_NOD_TRES()	Generates a third NOD visual variant.
NOD	insertar_animacion_apocalipsis_NOD_CUATRO()	Generates a fourth NOD visual variant.

Project location: wp-content/themes/blocksy-child/animations.php · the eight functions occupy almost the entire file; the first begins near the top and the others are distributed across more than 2,300 lines.

8.4 How an animation is selected

Selection occurs in two steps. First, PHP chooses a provisional faction with `rand(0,1)`. Then, within the selected group, `array_rand()` chooses one of the four available indexes. The final result is not yet an executed animation; it is the name of a function stored in the `$funcion` variable.

```
$faccion = rand(0,1) ? 'GDI' : 'NOD';

$animaciones = [
    'GDI' => [
        'insertar_animacion_apocalipsis_GDI_UNO',
        'insertar_animacion_apocalipsis_GDI_DOS',
        'insertar_animacion_apocalipsis_GDI_TRES',
        'insertar_animacion_apocalipsis_GDI_CUATRO',
    ],
    'NOD' => [
        'insertar_animacion_apocalipsis_NOD_UNO',
        'insertar_animacion_apocalipsis_NOD_DOS',
        'insertar_animacion_apocalipsis_NOD_TRES',
        'insertar_animacion_apocalipsis_NOD_CUATRO',
    ]
];

$funcion = $animaciones[$faccion][array_rand($animaciones[$faccion])];

if (function_exists($funcion)) {
    $funcion();
}
```

The `$funcion();` statement is a dynamic call: PHP takes the text stored in the variable and executes the function whose name exactly matches that text. `function_exists()` acts as a preliminary check so that the code does not attempt to invoke a function that does not exist.

```
rand(0,1)
↓
GDI or NOD
↓
array of 4 function names
↓
array_rand() chooses an index
↓
$funcion contains the selected name
↓
function_exists($funcion)
↓
$funcion() executes a single variant
```

8.5 How an animation is actually generated

Each variant in `animations.php` is a PHP function that outputs its own block of HTML, CSS, and JavaScript. The presentation therefore does not reach the browser as a finished “animation file.” PHP builds the structure and some values; the browser then applies styles, creates additional elements, and plays the sequence.

The first GDI variant clearly shows this mixture. The function locks scrolling with hud-lock, creates a fullscreen container, generates sparks from PHP, defines smoke, waves, fragments, flashes, and shake effects in CSS, includes audio elements, and finally registers JavaScript that coordinates the sequence.

```
<?php for ($i = 0; $i < 60; $i++): ?>
  <div class="spark"
    style="top:<?= rand(0,100) ?>%;
      left:<?= rand(0,100) ?>%;
      animation-duration:<?= rand(1,4) ?>s;">
  </div>
<?php endfor; ?>
```

In this fragment, PHP generates sixty .spark elements before sending the response. Each receives a different position and duration through rand(). The browser receives those sixty tags already constructed, and CSS then animates each one.

```
const wave = document.createElement('div');
wave.className = 'wave';
wave.style.animationDuration = `${1.5 + Math.random()*0.5}s`;
compuerta.appendChild(wave);

const frag = document.createElement('div');
frag.className = 'fragment';
frag.style.setProperty('--x', `${(Math.random()-0.5)*2000}px`);
frag.style.setProperty('--y', `${(Math.random()-0.5)*1200}px`);
```

Here, randomness already occurs on the browser side. JavaScript creates waves and fragments after the page has been received. In other words, the same selected function can produce small visual variations between runs because some values are randomized in PHP while others are calculated with Math.random() in JavaScript.

8.6 PHP and JavaScript do not perform the same part

Moment	Mechanism	Real example
Server, before sending the page	PHP	Selects GDI/NOD, chooses the function, generates 60 sparks, and writes HTML/CSS/JS.
Browser, after receiving the response	JavaScript	Waits for DOMContentLoaded, creates waves/fragments, plays audio, and executes setTimeout().
Visual presentation	CSS	The @keyframes control pulse, waveExpand, shake, flash, and other transitions.

Key idea

PHP does not “play” the animation on the server. PHP builds a response containing the required elements and scripts; the visible animation occurs in the browser.

8.7 Timing, sound, and interface locking

The functions use setTimeout() to chain specific moments. In GDI_UNO, for example, the logo begins to appear after about 400 ms; several waves are triggered from approximately 1.2 s; at 4 s an explosion with shake, sound, and fragments is activated; and later the final fade begins. Other variants have their own sequences.

The first variant also creates a sound button. The initial state soundEnabled = false prevents the effects from playing until the visitor enables audio. When sound is muted again, the code pauses and resets the <audio> elements present in the DOM. While the presentation occupies the screen, hud-lock blocks scrolling and temporarily hides side-cart elements so they do not interfere with the fullscreen experience.

8.8 Final cover and exit to Intro

After executing the selected function, insertar_presentacion() outputs a second general layer called #transition-cover. Its image is built from uploads/images/image7.jpg. The JavaScript for that layer schedules its appearance at 4 seconds

and navigation to `/intro/` at 6 seconds. However, this is not the only exit mechanism: immediately after outputting the block, PHP also attempts a `wp_redirect()` and executes `exit`. The following fragment brings those two sections of the same code together; the cover's CSS and HTML are omitted.

```
setTimeout(function() {
    const cover = document.getElementById('transition-cover');
    cover.classList.add('show');
}, 4000);

setTimeout(function() {
    window.location.href = "<?php echo home_url('/intro'); ?>";
}, 6000);

// After outputting the overlay and the script:
wp_redirect(home_url('/intro'));
exit;
```

The functions in `animations.php` also contain timed redirects to `/intro/`. The current code therefore overlaps three strategies: variant timers, `#transition-cover` timers, and a server-side `wp_redirect()`. If PHP can still send the Location header—for example, because buffering is in place and headers have not yet been sent—the browser may receive the HTTP redirect before it gets a chance to execute the full presentation. If prior output means the headers can no longer be changed, that server redirect may have no effect and the JavaScript paths become relevant. There is therefore no single guaranteed timing sequence across all environments.

Current implementation detail

The comment above `insertar_presentacion()` still mentions a `visited_intro` cookie, but the condition actually executed by the code checks `$_COOKIE['faction']`. The behavior described here is based on the effective condition, not the outdated comment.

8.9 Complete technical flow

```
WordPress receives a frontend request
↓
functions.php has already loaded animations.php
↓
template_redirect executes insertar_presentacion()
↓
/intro/ and /faction-selection/ are excluded
↓
does the faction cookie exist?
↓ no
the page context is validated
↓
rand() chooses GDI/NOD
↓
array_rand() chooses 1 of 4 functions
↓
$funcion() outputs HTML + CSS + JavaScript
↓
#transition-cover + timers are output
↓
PHP attempts wp_redirect('/intro') + exit
↓
can the Location header be sent?
├ yes → the browser may go to Intro before all timers complete
└ no → the JavaScript redirects in the response remain
↓
/intro/
```

8.10 Guided example: Alex's first visit

Now that the mechanism has been defined, a specific visit can be followed. Alex opens `/shop/` for the first time and the browser does not yet have a faction cookie.

1. WordPress loads `blocksy-child`. `functions.php` includes `animations.php`, but it does not display any variant yet.
2. When `template_redirect` is reached, WordPress calls `insertar_presentacion()`. The function checks that Alex is not in Intro or Faction Selection and confirms that Shop is an allowed context.
3. `$_COOKIE['faction']` is empty. PHP executes `rand(0,1)`. Assume the result selects GDI.

4. Within the GDI array, `array_rand()` returns one of the four indexes. Assume `insertar_animacion_apocalipsis_GDI_UNO()` is selected.
5. `function_exists()` confirms that the function is loaded and `$function()` executes it. PHP generates the fullscreen container, sparks, audio elements, and scripts for the variant.
6. The selected function outputs the variant's HTML, CSS, and JavaScript; `insertar_presentacion()` also outputs `#transition-cover` and its timers. Before PHP finishes executing, it attempts `wp_redirect('/intro/')` and calls `exit`.
7. If the HTTP Location header can be sent, Alex's browser may jump to `/intro/` before fully running the body timers. If that server-side redirect cannot be applied because earlier output has already committed the headers, the browser may execute the presentation and the JavaScript redirects. In both paths, the intended destination is `/intro/`, but the current code does not guarantee that the visible animation will always last the same amount of time.

8.11 What should not be confused with this choice

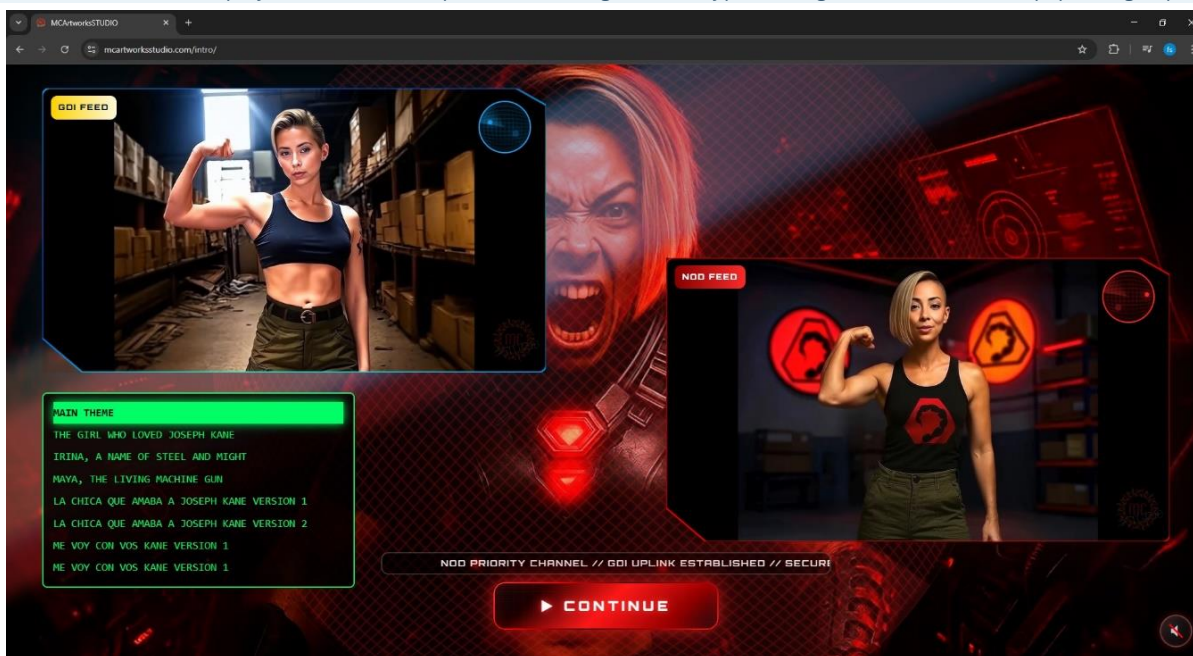
Important

Whether the presentation shown was GDI or NOD does not establish the visitor's identity. This choice only determines which entry animation is displayed. The persistent faction is created later, when the visitor confirms YES inside `faction-selection.php` and the faction cookie is written.

9. Multimedia introduction and HLS

The intro page loads `hud-intro.php`, an audiovisual module of more than 1,500 lines that builds the transition between the initial animation and faction selection. It combines PHP, HTML, CSS, and JavaScript with two HLS feeds, MP4 videos, music, sound effects, doors, radars, dynamically generated grids, and its own navigation flow.

Project location: `wp-content/themes/blocksy-child/hud-intro.php` · WordPress includes it from `functions.php`, and the file generates the Intro interface, both feeds, the music player, and the JavaScript initialization logic. The entrypoint is registered from `functions.php` through `wp_head`.



Multimedia introduction: two videos, information panels, music, and a continue button.

9.1 Integration with WordPress

The introduction is loaded conditionally. `functions.php` checks whether the current page is `intro`; only then does it include `hud-intro.php` and execute `nod_gdi_hud_intro()`. In this way, the HUD code, styles, and interface are not generated on the rest of the site.

```
function hud_intro() {
    $hud_intro = get_page_by_path('intro');
    if (!$hud_intro || !is_page($hud_intro->ID)) return;
    require_once get_stylesheet_directory() . '/hud-intro.php';
    if (function_exists('nod_gdi_hud_intro')) nod_gdi_hud_intro();
}
```

```
WordPress loads a page
↓
hud_intro() checks the slug
↓
is it /intro/?
↓ yes
require_once hud-intro.php
↓
nod_gdi_hud_intro()
```

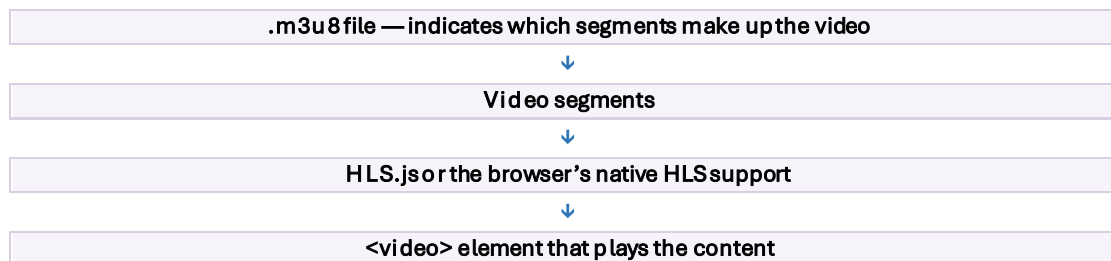
9.2 Screen composition

hud-intro.php builds a fixed HUD that fills the entire viewport and blocks scrolling while it is active. The screen consists of several independent layers: seven animated backgrounds, a lens and curved grids, two video feeds, GDI/NOD doors, radars, a music panel, a 0–100% initialization sequence, a ticker, sounds, and the CONTINUE button.

```
INTRO HUD
├── image1.jpg ... image7.jpg backgrounds
├── lens and curved grids
├── GDI FEED → HLS + MP4 fallback
├── NOD FEED → HLS + MP4 fallback
├── doors and radars
├── music selector + hud-audio
├── 0–100% initialization
├── ticker
└── CONTINUE + global sound control
```

9.3 What HLS is

HLS stands for HTTP Live Streaming. Instead of delivering a video as one large file, it uses an .m3u8 playlist that points to smaller segments. The player requests those segments as needed. In this project, HLS is used both in the introduction and on the Home page.



9.4 Video loading by browser

Source code: wp-content/themes/blocksy-child/hud-intro.php → nod_gdi_hud_intro() → JavaScript block executed on DOMContentLoaded around line 1032. That block detects Firefox/Chromium, defines the GDI/NOD paths, and prepares the elements used by loadVideo().

hud-intro.php distinguishes Firefox from Chromium-based browsers such as Chrome and Opera, and adjusts the paths and HLS.js configuration. It also attempts to keep the two videos synchronized: if their time difference exceeds a small threshold, it moves both to the earlier timestamp so that one does not advance noticeably ahead of the other.

Example

If GDIvideo is at 12.40 seconds and NODvideo at 12.30, the difference is small. If they drift by more than 0.05 s, the code can move both to the common point so that one screen does not advance noticeably ahead of the other.

```
const isFirefox = typeof InstallTrigger !== 'undefined';
const isChromium = !!window.chrome;

const videos = isFirefox
  ? { GDIvideo: '/videos/gdi-intro-firefox/GDI-INTRO.m3u8',
      NODvideo: '/videos/nod-intro-firefox/NOD-INTRO.m3u8' }
  : { GDIvideo: '/wp-content/uploads/videos/GDI-INTRO/GDI-INTRO.m3u8',
      NODvideo: '/wp-content/uploads/videos/NOD-INTRO/NOD-INTRO.m3u8' };
```

9.5 Loading with HLS.js and simultaneous preparation

Source code: hud-intro.php → same DOMContentLoaded callback in nod_gdi_hud_intro() (approx. line 1032) → loadVideo(), Hls.loadSource()/attachMedia(), and Promise.all(). It runs in the browser once the Intro DOM exists; PHP only output the script as part of the response.

The loadVideo() function connects each .m3u8 file to its <video> element. When HLS.js is available, it uses loadSource() and attachMedia(); if the browser supports HLS natively, it assigns the file directly to the src attribute. The HLS.js configuration also differs between Firefox and Chromium.

```
const hls = new Hls({
  enableWorker: false,
  lowLatencyMode: false,
  backBufferLength: 1,
  maxBufferLength: 10,
  maxMaxBufferLength: 20
});
hls.loadSource(src);
hls.attachMedia(vid);
```

Both feeds are prepared with Promise.all(). They are then both reset to second 0, one render cycle is awaited through requestAnimationFrame(), and playback is attempted. This lets GDI and NOD start from a coordinated state.

```
await Promise.all(
  Object.entries(videos).map(([id, src]) => loadVideo(id, src))
);
Object.values(elements).forEach(v => { v.currentTime = 0; v.pause(); });
await new Promise(r => requestAnimationFrame(r));
Object.values(elements).forEach(v => v.play().catch(() => {}));
```

9.6 Synchronizing GDI and NOD

Source code: hud-intro.php → same DOMContentLoaded callback (approx. line 1032) → synchronization setInterval. It receives the already prepared GDIvideo and NODvideo elements and corrects currentTime when the absolute difference exceeds 0.05 s.

Even when both videos start together, buffering and decoding can cause a small timing difference. The code periodically checks both currentTime values and, if the separation exceeds 0.05 seconds, moves both to the lower observed time. In Chromium the check runs approximately every 1.5 seconds; in other cases, every 2 seconds.

```
const diff = v1.currentTime - v2.currentTime;
if (Math.abs(diff) > 0.05) {
  const target = Math.min(v1.currentTime, v2.currentTime);
  v1.currentTime = target;
  v2.currentTime = target;
}
```

Technical example

If GDI is at 12.40 s and NOD at 12.31 s, the difference is 0.09 s. The code uses 12.31 s as the common point so that neither feed continues noticeably ahead.

9.7 Fallback video and MutationObserver

Source code: hud-intro.php → DOMContentLoaded callback approximately between lines 1472–1507 → canplay listener for each feed + MutationObserver on the corresponding door. The observer reacts to DOM class changes; it does not query the server by itself.

Each HLS feed also has an MP4 fallback. The fallback remains available while the main stream cannot yet play. When the canplay event confirms that HLS is ready, the MP4 is hidden. A MutationObserver also watches the door class: if it opens before the stream has enough data, the alternate video remains visible instead of leaving an empty area.

```
video.addEventListener('canplay', () => {
  fallback.style.display = 'none';
});

const observer = new MutationObserver(() => {
  if (door.classList.contains('open') && video.readyState < 3)
    fallback.style.display = 'block';
});
```

9.8 Initialization, doors, and music

Source code: hud-intro.php → main JavaScript in nod_gdi_hud_intro() → IIFE init(ms = 3000), approximately from line 1177. The timer updates the percentage/bar and, when it reaches 100%, enables the doors, audio, and CONTINUE portion of the flow.

The 0–100% bar is a timed visual initialization lasting approximately three seconds. Every 40 ms the percentage is recalculated. At 100%, the loading beep stops, the NOD/GDI doors open, the door effect plays, a brief shake is applied to the panels, CONTINUE is enabled, and the music panel is activated.

```
const pct = Math.min(100, Math.round((elapsed / ms) * 100));
percent.textContent = pct + '%';
bar.style.width = pct + '%';

if (pct >= 100) {
  nodDoor.classList.add('open');
  gdiDoor.classList.add('open');
  btn.classList.add('ready');
}
```

The music player uses a single hud-audio element and several options with data-src. The trackPositions object remembers each track's currentTime while the visitor remains in Intro, so when switching tracks and later returning to a previous one, playback can continue from the stored position. Changes are accompanied by a volume fade.

9.8.1 How the music selector is enabled and used

Source code: hud-intro.php → HTML for the #hud-music-panel panel and <audio id="hud-audio"> around lines 827–837; its JavaScript listeners begin in the DOMContentLoaded callback near line 855. The visible tracks are controls that change the source of a single <audio> element.

The music panel is not active from the first instant. When Intro starts, JavaScript disables clicks with pointerEvents = none and reduces its opacity. When initialization reaches 100%, the doors open and, a few moments later, the panel accepts interaction again.

The eight visible options are not eight <audio> elements. Each option is a <div class="track"> with a data-src path. All of them share a single player, <audio id="hud-audio">. Changing songs means changing the source used by that same element.

```
<div class="track" data-src="/songs/Main_Theme">MAIN THEME</div>
<div class="track" data-src="/songs/The_Girl_Who_Loved_Joseph_Kane">THE GIRL WHO LOVED JOSEPH KANE</div>
...
<audio id="hud-audio" preload="none" loop playsinline></audio>
```

MAIN THEME is prepared as the initial track and marked active. That block assigns its data-src to hud-audio, configures loop, leaves audio.muted = true, and calls load(). It also attempts play() followed immediately by pause() and

currentTime = 0 to force initial preparation of the resource. This is not equivalent to leaving the track playing continuously.

Current implementation detail

The global sound control declares let isMuted = true inside one DOMContentLoaded callback and exposes window.setGlobalMute() / window.setMainReady(). The 0–100% initialization, however, lives in a different IIFE. When it reaches 100%, that second block later executes audio.muted = isMuted, but isMuted is not defined in that scope. In the current source code, that line can produce ReferenceError: isMuted is not defined. Before reaching it, initialization already checks mainAudio.muted: if unmuted, it attempts to play it; if it remains muted, it pauses it and resets currentTime = 0. Therefore, automatic MAIN THEME playback at 100% should not be documented as a guaranteed sequence.

Track-list implementation detail

The last visible option contains data-src="/songs/Me_Voy_con_Vos_Kane_VERSION_2", but its text again reads ME VOY CON VOS KANE VERSION 1. This is a literal inconsistency in the current source code: the VERSION_1 and VERSION_2 paths are different even though the last two visible labels both display VERSION 1.

The track click listeners and the trackPositions object are registered in another DOMContentLoaded callback. Therefore, the manual selection and position-restoration mechanism described below is independent of that out-of-scope reference; the inconsistency specifically affects the automatic MAIN THEME initialization stage.

9.8.2 How the position of each song is remembered

Source code: hud-intro.php → DOMContentLoaded callback approximately between lines 855–929 → trackPositions, timeupdate event, .track click listeners, and loadedmetadata event. That state lives in browser memory during the current stay in Intro.

trackPositions is a JavaScript object that associates each song path with its currentTime. The timeupdate event continuously updates the position of the active track. When the visitor selects another track, the previous track's position is saved again before the source is changed.

```
const trackPositions = {};

audio.addEventListener('timeupdate', () => {
  if (audio.dataset.currentTrack) {
    trackPositions[audio.dataset.currentTrack] = audio.currentTime;
  }
});

// when changing tracks
trackPositions[audio.dataset.currentTrack] = audio.currentTime;
audio.dataset.currentTrack = newSrc;
audio.src = newSrc;
```

After loadedmetadata, the code looks for trackPositions[newSrc]. If a saved position exists, it assigns that value to currentTime and continues from there. fadeIn() gradually raises the volume from 0 to the target level, avoiding an abrupt transition between songs.

Do not confuse these concepts

trackPositions exists only in JavaScript memory during that stay in Intro. It is not the faction cookie, not sessionStorage, and not the WooCommerce session. When the page is left or reloaded, the object is created again.

9.8.3 Guided example: Alex changes the music

Alex reaches Intro. While the bar is still between 0 and 100%, the music panel appears disabled. When initialization completes, the doors open, CONTINUE becomes available, and the panel accepts clicks.

1. MAIN THEME is already prepared in hud-audio, but it initially remains muted. At 100%, if mainAudio is still muted, the code pauses it and resets it to 0; if Alex had unmuted it earlier, the code attempts to play it. Later, the out-of-scope reference to isMuted explained in Section 9.8.1 is reached. Alex can use the global sound control: setGlobalMute(false), with mainReady already active, attempts to play the current hud-audio track.
2. With sound enabled and MAIN THEME actually playing, Alex listens for about 35 seconds and then clicks THE GIRL WHO LOVED JOSEPH KANE. Before the switch, trackPositions stores approximately 35 as the MAIN THEME position.
3. JavaScript visually marks the new option, pauses hud-audio, changes audio.src, and waits for loadedmetadata. When the track is ready, it restores its previous position if it has already been used and applies the fade.

4. Later, Alex returns to MAIN THEME. Because trackPositions retains the previous timestamp, hud-audio can continue approximately where it left off instead of necessarily starting again at 0.

9.9 CONTINUE and navigation origin

Source code: hud-intro.php → the main script reads and removes sessionStorage.fromMenu around lines 1158–1161; freezeVideo() is declared near line 1295 and used before leaving Intro. The destination decision occurs in browser-side JavaScript.

The introduction can be opened during the first visit or from VIEW INTRO in the main menu. The menu stores sessionStorage.fromMenu = true. Intro reads and removes that value; when CONTINUE is clicked, it chooses the destination according to that origin.

```
const fromMenu = sessionStorage.getItem('fromMenu') === 'true';
sessionStorage.removeItem('fromMenu');

if (fromMenu) {
  window.location.href = 'https://mcartworksstudio.com';
} else {
  window.location.href = 'https://mcartworksstudio.com/faction-selection';
}
```

In Chromium, before leaving, freezeVideo() captures the visible frame from both feeds using Canvas and leaves it overlaid while navigation takes place. This prevents a cut to black during the transition.

```
/intro/
↓
loads GDI + NOD
↓
Promise.all() + playback
↓
periodic synchronization
↓
0-100% → doors → music → CONTINUE
↓
fromMenu?
├ yes → Home
└ no → Faction Selection
```

9.10 Guided example: Alex goes through Intro from start to finish

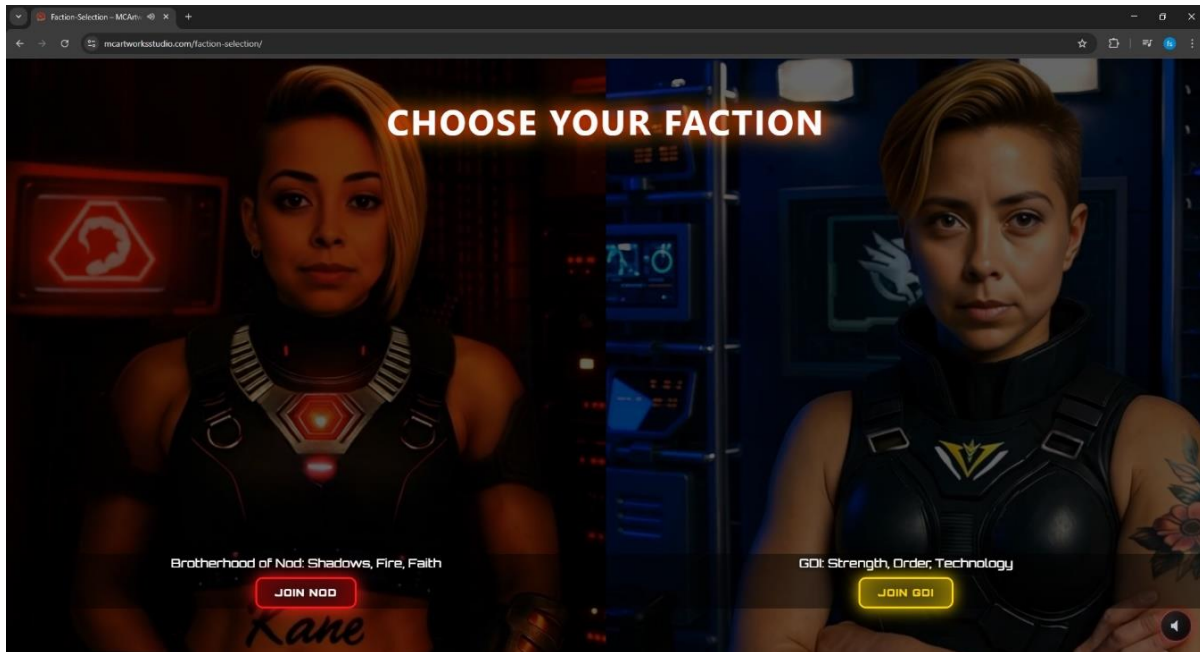
Alex reaches /intro/ from the initial presentation. WordPress confirms the slug, includes hud-intro.php, and executes nod_gdi_hud_intro(). The browser receives both GDI/NOD feeds, the fallback MP4 files, the HUD, the controls, and the JavaScript code.

```
Alex enters /intro/
↓
GDI + NOD are prepared with Promise.all()
↓
both feeds start in sync
↓
synchronization corrects differences greater than 0.05 s
↓
the visual bar reaches 100%
↓
the doors open and music + CONTINUE are enabled
↓
Alex can select/change a track
↓
Alex clicks CONTINUE
↓
was sessionStorage.fromMenu true on entry?
├ yes → Home
└ no → Faction Selection
```

The final decision does not depend on the selected music or on a direct “first visit” check. The music selector only affects the Intro experience; CONTINUE decides the destination solely from the fromMenu value read from sessionStorage on entry, as explained in Section 9.9.

10. Faction selection

After the introduction, the visitor reaches `faction-selection.php`. This file, more than 1,000 lines long, builds a fullscreen screen with its own loading sequence, two audiovisual panels, idle and confirmation videos, music, sound effects, Canvas particles, and the logic that makes the NOD/GDI choice persistent.



Selection screen: NOD on the left and GDI on the right.

10.1 Loading the selection screen

`faction-selection.php` connects to WordPress through `wp_body_open` and acts only on the `faction-selection` page. The order is defined: first the general loading overlay is generated and then the selection screen. While the overlay is active, `loading-active` is applied to block scrolling and keep the main content hidden.

```
add_action('wp_body_open', function() {
    if (is_page('faction-selection')) {
        mostrar_loading_general_overlay();
        pantalla_faccion_epica();
    }
});
```

JavaScript waits for the loading bar's `animationend` event to fade out the overlay and display the NOD/GDI screen. There is also a 3.5-second fallback that triggers the same ending if the event does not occur.

10.2 Two panels with two video states

Each faction is represented by a `.faccion` container and a `data-faccion` attribute whose value is `nod` or `gdi`. Inside, there is a looping `video-default` for the idle state and a `video-confirm` that is used after `JOIN` is clicked.

```
<div class="faccion nod" data-faccion="nod">
    <video class="video-default" autoplay muted loop playsinline></video>
    <video class="video-confirm" playsinline style="display:none;"></video>
</div>
```

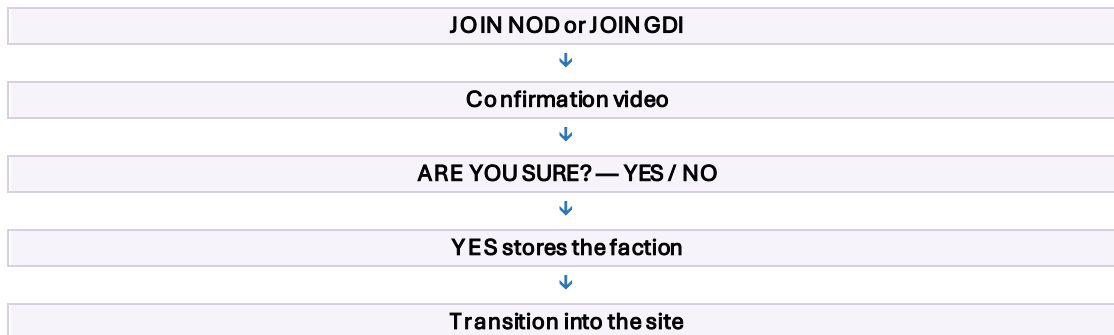
```

INITIAL STATE
video-default visible
overlay-default visible
↓ JOIN
video-default hidden
video-confirm visible
overlay-confirm visible
↓
ARE YOU SURE? → YES / NO

```

10.3 Preview, JOIN, and confirmation

Each faction has a preview video and a confirmation video. Hovering over JOIN activates the corresponding preview; clicking it plays the confirmation video and displays the YES / NO question. The faction is stored only when the visitor confirms YES. The code also creates a temporary cookie to indicate that the visitor has just left this screen.



10.4 Why confirmation exists

The confirmation step prevents a simple pointer movement from changing the entire appearance of the site. The choice is stored only when the visitor confirms the faction.

10.5 State management and the NO button

When a confirmation begins, the code checks whether the other faction already has a visible video-confirm. If it does, that state is reset before the new choice is activated. This ensures that NOD and GDI do not remain in the confirmation state at the same time.

```

function resetThisFaction() {
  videoConfirm.pause();
  videoConfirm.currentTime = 0;
  videoConfirm.style.display = 'none';
  overlayConfirm.style.display = 'none';
  videoDefault.style.display = 'block';
  overlayDefault.style.display = 'flex';
}

```

The NO button executes this function and returns to the initial state without storing any cookie. When the confirmation video ends, it remains paused on its final frame while the visitor decides.

10.6 YES confirmation and persistence of the choice

YES obtains the faction from the data-faccion attribute of the selected container. Only then is the faction cookie created for approximately one year, along with the temporary from_faction_selection cookie for ten seconds. After the transition, the site returns to Home.

```
const faction = btn.closest('.faction');

document.cookie = 'faction=' + faction.dataset.faction
  + '; path=/; max-age=' + (60 * 60 * 24 * 365);
document.cookie = 'from_faction_selection=1; path=/; max-age=10';

setTimeout(() => { window.location.href = '/'; }, 1000);
```

Difference between the two cookies

faction retains the selected identity and controls the site's subsequent appearance. from_faction_selection is only a transient signal used to recognize that navigation has just left the selector.

10.7 Audiovisual transition and Canvas

After YES, a red fullscreen layer is created and zoom, brightness, and blur are applied to the confirmation content. Chrome and Opera receive special handling: the code captures the last video frame in a Canvas, converts it to a temporary JPEG image, and animates that image before redirecting. In other browsers, the effect is applied directly to the video.

```
const canvas = document.createElement('canvas');
canvas.width = videoConfirm.videoWidth || 1920;
canvas.height = videoConfirm.videoHeight || 1080;
const ctx = canvas.getContext('2d');
ctx.drawImage(videoConfirm, 0, 0, canvas.width, canvas.height);
img.src = canvas.toDataURL('image/jpeg', 0.9);
```

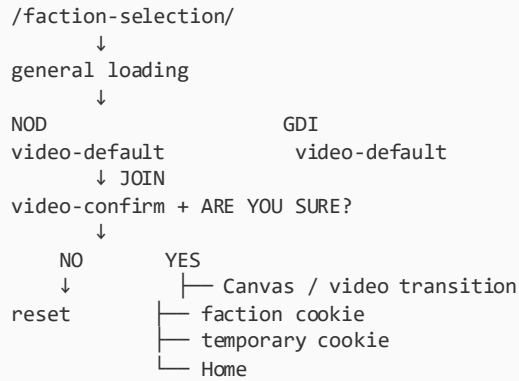
10.8 Global sound and particles

The screen includes background music and different effects for hover, JOIN, YES, and NO. A single isMuted state controls both those audio elements and the videos. When sound is enabled for the first time, the user interaction allows the music to start without violating normal autoplay restrictions.

The background uses a separate Canvas element that generates 80 particles with random position, radius, and velocity. animateParticles() updates their positions and uses requestAnimationFrame() to keep the movement synchronized with browser rendering.

```
for (let i = 0; i < 80; i++) {
  particles.push({
    x: Math.random() * canvas.width,
    y: Math.random() * canvas.height,
    r: Math.random() * 2 + 1,
    dy: Math.random() * -1 - 0.2
  });
}
requestAnimationFrame(animateParticles);
```

10.9 Complete selection flow



11. Cookies and NOD / GDI appearance

The faction cookie is the value that allows a choice made on one browser page to continue affecting later pages. Without that persistence, Home, the menu, Gallery, and the rest of the interface would not know whether to use GDI or NOD resources on each new request.

To understand this part, two moments must be separated: JavaScript writes the cookie in the browser when the visitor confirms a faction; then, on the next HTTP request, PHP receives that value in `$_COOKIE` and generates the corresponding response.

11.1 Where the faction cookie is created

The definitive cookie is created in `faction-selection.php` inside the click event for the YES buttons. The value is not taken from the visible button text, but from the `data-faction` attribute of the selected `.faction` container.

```

const faction = btn.closest('.faction');

document.cookie =
  'faction=' + faction.dataset.faction
  + '; path=/; max-age=' + (60 * 60 * 24 * 365);

document.cookie =
  'from_faction_selection=1; path=/; max-age=10';

setTimeout(() => { window.location.href = '/'; }, 1000);

```

Project location: `wp-content/themes/blocksy-child/faction-selection.php` · YES button block around lines 894–977; the cookies are written approximately on lines 969–971.

11.2 What each part of the cookie means

Part	Example	Meaning
Name	faction	Identifies the data that PHP will later read.
Value	nod / gdi	The faction confirmed by the visitor.
path	/	Causes the cookie to be sent across the different paths of the same site.
max-age	60 × 60 × 24 × 365	Keeps faction for approximately one year.
Temporary cookie	from_faction_selection=1	Indicates for a few seconds that navigation has just left the selector.

The long lifetime belongs to `faction`. `from_faction_selection` serves a different purpose and lasts only ten seconds: it does not represent identity, but rather the immediate origin of a navigation event.

11.3 JavaScript writes it; PHP receives it on the next request

When Alex clicks YES, the current page has already been generated by PHP. Writing `document.cookie` does not cause the PHP for that same response to run again. Instead, the browser stores the new value and automatically attaches it to the next request to the same site.

```
JavaScript executes document.cookie
↓
the browser stores faction=nod or faction=gdi
↓
window.location.href opens another URL
↓
a new HTTP request begins
↓
the browser sends the cookie to the server
↓
PHP receives $_COOKIE['faction']
↓
the theme chooses resources and styles before responding
```

Key idea

The cookie acts as a bridge between two different requests. JavaScript writes it on the client; PHP consumes it later on the server.

11.4 How PHP reads the faction

```
$faction = $_COOKIE['faction'] ?? 'gdi';
```

The `??` operator uses the cookie value if it exists; if it does not, the code shown uses `gdi` as the default. This read appears in different modules because the faction affects more than a single stylesheet.

11.5 How the GDI or NOD CSS is enqueued

`functions.php` registers callbacks on `wp_enqueue_scripts`. The production block reads `faction`, removes previous handles, and enqueues `gdi-faction-min.css` or `nod-faction-min.css`. The version is calculated with `filemtime()` so that it changes when the file changes.

```
$faction = $_COOKIE['faction'] ?? 'gdi';

wp_dequeue_style('gdi-faction');
wp_dequeue_style('nod-faction');

if ($faction === 'gdi') {
    $css_file = get_stylesheet_directory() . '/gdi-faction-min.css';
    $version = file_exists($css_file) ? filemtime($css_file) : '1.0';
    wp_enqueue_style('gdi-faction', get_stylesheet_directory_uri() . '/gdi-faction-min.css', [], $version, 'all');
} elseif ($faction === 'nod') {
    $css_file = get_stylesheet_directory() . '/nod-faction-min.css';
    $version = file_exists($css_file) ? filemtime($css_file) : '1.0';
    wp_enqueue_style('nod-faction', get_stylesheet_directory_uri() . '/nod-faction-min.css', [], $version, 'all');
}
```

The current code also contains a second active callback with the same priority, 999, and the same handles, but it uses `time()` to force a different URL on every load. Section 36 analyzes that overlap from a caching and performance perspective; the important point here is that both blocks read the same `faction` cookie.

Project location: `functions.php` · `faction-CSS` callbacks approximately between lines 77 and 185 · both registered on `wp_enqueue_scripts` with priority 999.

11.6 Which parts of the site depend on faction

Area	How faction is used
------	---------------------

Global CSS	Chooses gdi-faction-min.css or nod-faction-min.css.
Home	Selects the faction title, HLS source, and visual resources.
header.php / menu	Chooses the HUD video and music set.
Gallery	Selects the GDI/NOD array and its visual identity.
Our Services and other screens	Receive the general appearance provided by the active stylesheet.

Therefore, faction does not simply mean “a stored color.” It is a context value that several components read to decide which resources and appearance to use.

11.7 Why from_faction_selection exists

from_faction_selection lasts only ten seconds and is used to recognize the navigation immediately after the selector. Home checks it in mostrar_overlay_faccion_inicio(); if present, it reads faction, shows the corresponding GDI or NOD access overlay, and then deletes the cookie so that a simple reload does not repeat that transition.

```
if (isset($_COOKIE['from_faction_selection'])) {
    $faction = $_COOKIE['faction'] ?? 'gdi';

    if ($faction === 'gdi') {
        mostrar_loading_GDI_overlay();
    } elseif ($faction === 'nod') {
        mostrar_loading_NOD_overlay();
    }

    setcookie('from_faction_selection', '', time() - 3600, '/');
}
```

```
YES in Faction Selection
↓
faction = nod/gdi for ~1 year
+
from_faction_selection = 1 for 10 s
↓
Home receives both cookies
↓
faction determines identity
↓
from_faction_selection triggers the access overlay
↓
PHP deletes the temporary cookie
```

11.8 What happens if faction does not exist

The absence of faction becomes significant again at site entry: insertar_presentacion() can activate the random sequence explained in Section 8 when the request belongs to one of the allowed contexts. Elsewhere in the code, GDI default values are used to prevent certain modules from being left without resources before a persistent choice has been made.

This does not mean that GDI is stored automatically. A default value used by PHP to build a response is not the same as writing document.cookie. Persistent identity appears only when Faction Selection confirms YES.

11.9 Do not confuse cookies, sessionStorage, and the WooCommerce session

Mechanism	What it stores	Use in MC Artworks
cookie faction	NOD/GDI identity for an extended period.	Appearance and resources across different pages.

cookiefrom_faction_selection	Temporary signal lasting a few seconds.	Shows the overlay when returning to Home from the selector.
sessionStorage.fromMenu	State of that tab/navigation session.	Intro knows whether CONTINUE should return to Home.
WooCommerce session	Temporary commercial state.	Cart and data associated with its items.

11.10 Guided example: Alex chooses NOD and navigates the site

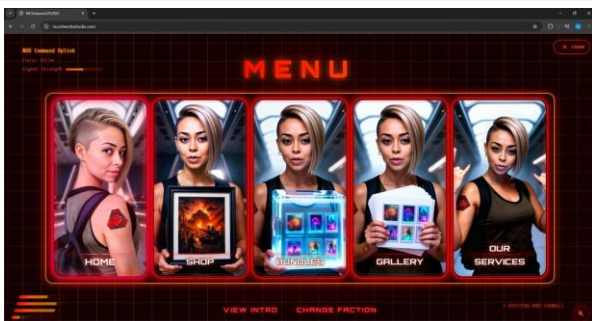
With the mechanism explained, the full experience can now be followed. Alex reaches Faction Selection, clicks JOIN NOD, and finally confirms YES.

1. JavaScript obtains the NOD .faccion container and reads data-faccion="nod".
2. The browser stores faction=nod with path=/ and a lifetime of approximately one year. It also stores from_faction_selection=1 for ten seconds.
3. One second later, JavaScript navigates to /. That navigation creates a new request; the browser sends the cookies to the server.
4. PHP receives \$_COOKIE['faction'] = "nod". The style callbacks enqueue nod-faction-min.css, and Home selects its NOD resources.
5. Because from_faction_selection still exists, Home displays the NOD access overlay and then deletes that temporary cookie.
6. Alex opens Shop and later Gallery. On those new requests, the browser continues sending faction=nod, so the site retains the NOD identity without asking him to choose again.
7. If Alex returns another day and the cookie has not expired or been deleted, the browser continues sending it. If it disappears, the entry logic treats the visit again as one without a persistent faction.

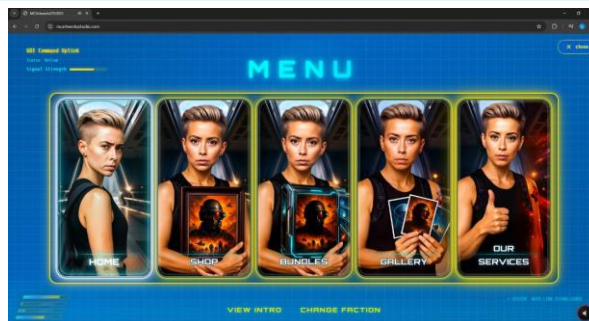
12. Main menu

header.php builds one of the central components of MC Artworks Studio: a fullscreen HUD menu of more than 560 lines. It does not behave like a traditional navigation bar; it combines PHP, HTML, and JavaScript to generate video cards, faction-specific resources, persistent music, sound effects, accessible states, Canvas transitions, and coordination with Gallery and Intro.

Project location: wp-content/themes/blocksy-child/header.php · this file builds the menu HUD, selects resources according to the faction cookie, and contains most of the JavaScript listeners for opening, closing, audio, and navigation. functions.php participates in global site components, but the audiovisual menu is concentrated in header.php.



NOD menu: red and dark.



GDI menu: blue and yellow.

12.1 Integration with Blocksy

Source code: wp-content/themes/blocksy-child/header.php → header-template PHP, approximately lines 17–19. `ob_start()` captures the output of `blocksy_output_header()`; `ob_get_clean()` stores it in `$global_header` so it can later be reinserted together with the custom HUD.

`header.php` remains part of Blocksy's header system. Before building the HUD, it captures the output generated by `blocksy_output_header()` in a buffer. It then inserts the custom menu and outputs the parent theme header again within the normal WordPress flow.

```
ob_start();
blocksy_output_header();
$global_header = ob_get_clean();

...

do_action( 'blocksy:header:before' );
echo $global_header;
do_action( 'blocksy:header:after' );
```

```
header.php
↓
Blocksy generates its header
↓
ob_start() / ob_get_clean() store it
↓
the custom HUD is built
↓
the Blocksy header is reinserted
```

12.2 Resources according to the faction

Source code: `header.php` → PHP around lines 21–42. The faction cookie selects one of the video arrays before the server generates the menu cards.

PHP reads the faction cookie and selects a different set of five videos. The menu's HTML structure is the same for NOD and GDI; what changes are the audiovisual resources and the faction stylesheet.

```
$faction = $_COOKIE['faction'] ?? 'gdi';

$videos = $videos_by_faction[$faction];
```

12.3 Building the cards

Source code: `header.php` → PHP/HTML approximately between lines 68–109. `foreach($videos as $index => $video_url)` combines videos, links, and labels; this block runs on the server and delivers the five already-built cards to the browser.

Destinations, labels, and videos are connected by index. `foreach()` generates the five cards, marks the current one with `is-current`, and creates a link covering the entire surface of each card. Values are output through `esc_url()`, `esc_html()`, and `esc_attr()`.

```
foreach ($videos as $index => $video_url) {
    $link_url = $links[$index];
    $is_current = rtrim($link_url, '/') === rtrim($current_url, '/');
    // card + video + link + label
}
```

In addition to the card rail, the overlay includes data-stream layers, scanlines, sweep-beam, radar-pulse, HUD bars, status information, and a fictional terminal. The title NOD Command Uplink or GDI Command Uplink is generated by PHP according to the faction.

12.4 Opening, closing, and staggered entrance

Source code: `header.php` → HUD JavaScript; `openOverlay()` begins around line 225 and is called by the `#zajMenu` button listeners. At this point execution is already in the browser: classes, ARIA attributes, scrolling, and animation delays are modified.

The #zajMenu button controls the #zajOverlay overlay. openOverlay() adds is-open, updates aria-expanded/aria-hidden, locks scrolling, and retriggers each card's animation. The index * 0.07 s delay creates a staggered entrance. The menu can also be closed with Escape or by clicking the overlay background.

```
function openOverlay() {
  btn.classList.add('is-open');
  overlay.classList.add('is-open');
  btn.setAttribute('aria-expanded', 'true');
  overlay.setAttribute('aria-hidden', 'false');

  cards.forEach((card, index) => {
    card.style.animationDelay = `${index * 0.07}s`;
    card.classList.add('animate-in');
  });
}
```

12.5 Card sound effects

JavaScript creates three Audio objects: hoverSound, clickSound, and currentClickSound. Hover sound is played only on cards that do not represent the current page; clicking uses a different effect if the card has is-current. Before each playback, currentTime is reset to 0 to allow repeated interactions.

12.6 Audio that resumes where it left off

The menu stores in a cookie the exact second at which the music stopped. The cookie name depends on the faction, for example gdi_menu_audio_time or nod_menu_audio_time. When the menu is opened again, JavaScript restores that value and resumes from that point.

Example

If the visitor closes the menu when the NOD track is at 42.6 seconds, the code stores that time. When the NOD menu is reopened, it attempts to start playback from approximately 42.6 s instead of always starting from zero.

12.7 Music, fade, and mute control

Source code: header.php → HUD JavaScript → initMenuAudio() around line 301. The function creates/configures the menu's single Audio instance, restores the second stored in the faction cookie, and maintains that position while the track plays.

initMenuAudio() builds the /songs/nod-menu-theme or /songs/gdi-menu-theme path, configures loop and preload, and restores the second saved in the cookie. While the track is active, an interval updates that position every two seconds.

```
const cookieName = `${faction}_menu_audio_time`;
const savedTime = parseFloat(getCookie(cookieName));
if (!isNaN(savedTime)) menuAudio.currentTime = savedTime;

setInterval(() => {
  if (menuAudio && !menuAudio.paused)
    setCookie(cookieName, menuAudio.currentTime);
}, 2000);
```

Opening the menu fades the volume up to 0.4 over about 800 ms; closing it reduces the volume to 0 and then pauses the track. If the menu was opened automatically, it starts muted and displays a control that allows the user to enable music and effects afterward.

12.8 Navigation with transition

Source code: header.php → .zaj-card-link listeners around lines 194–218 → showLoadingAndRedirect() from approximately line 432. The listener cancels immediate navigation and delegates the transition to that function.

When a card is selected, the page does not change abruptly. The menu displays a brief loading screen and then opens the destination. VIEW INTRO returns to the introduction, while CHANGE FACTION returns to the faction selector.

The transition is implemented in `showLoadingAndRedirect()`. Normal link navigation is canceled with `preventDefault()`; before changing `window.location.href`, the final frame of all five previews is frozen, ESTABLISHING UPLINK is shown, and a progress bar is animated.

```
link.addEventListener('click', e => {
  e.preventDefault();
  showLoadingAndRedirect(link.getAttribute('href'));
});
```

12.9 Freezing the last frame

To avoid a visual cut, `showLoadingAndRedirect()` creates a Canvas for each `.zaj-card-video`, draws the visible frame, and converts it into a JPEG image that is temporarily overlaid on the video. It then activates the loader and navigates after about 800 ms.

```
const canvas = document.createElement('canvas');
canvas.width = v.videoWidth;
canvas.height = v.videoHeight;
const ctx = canvas.getContext('2d');
ctx.drawImage(v, 0, 0, canvas.width, canvas.height);
snapshot.style.backgroundImage = `url(${canvas.toDataURL('image/jpeg')})`;
```

12.10 VIEW INTRO, CHANGE FACTION, and Gallery

Source code: `header.php` → `sessionStorage.fromMenu` is written near line 424; `stopAllOtherAudio()/restorePreviousAudio()` begin approximately around lines 514 and 533. These are separate pieces of the same menu script: one preserves the origin of Intro, while the others coordinate audio/video with Gallery.

VIEW INTRO stores `sessionStorage.fromMenu = true` before opening `/intro`, allowing CONTINUE to return to Home afterward. CHANGE FACTION uses the same loader to open `/faction-selection`. Gallery also has a specific integration: opening the menu pauses audio outside the overlay and mutes external videos; closing it restores the previous state.

```
sessionStorage.setItem('fromMenu', 'true');
showLoadingAndRedirect('https://mcartworksstudio.com/intro');
```

```
user opens MENU
↓
openOverlay()
├─ locks scrolling
├─ animates five cards
├─ restores music from the cookie
├─ enables sounds
↓
user selects a destination
↓
showLoadingAndRedirect()
├─ captures frames with Canvas
├─ shows ESTABLISHING UPLINK
├─ animates progress
└─ changes page
```

12.11 Guided example: Alex opens the menu and enters Gallery

The previous subsections described each component separately. This example combines them to show what happens when a person uses the navigation HUD.

Alex is on Home with `faction=nod`. PHP has already generated `header.php` with the five NOD videos and resources. When Alex clicks MENU, JavaScript executes `openOverlay()`: it adds `is-open`, updates `aria-expanded/aria-hidden`, locks scrolling, and applies a staggered entrance to the cards.

The menu initializes the NOD track and checks the `nod_menu_audio_time` cookie. If Alex previously closed the HUD at second 42.6, the player attempts to resume from approximately that point. Hover and click sounds continue reacting to the cards according to the current state.

Alex moves the pointer over Gallery and then clicks. The listener cancels immediate navigation with `preventDefault()` and calls `showLoadingAndRedirect()`. Before changing pages, the function captures the visible frame of each preview with Canvas, leaves it as a snapshot, shows ESTABLISHING UPLINK, and runs the progress transition. Only then does it change `window.location.href` to Gallery.

```
Alex clicks MENU
↓
openOverlay() opens the HUD
↓
NOD music and sounds are restored
↓
Alex selects Gallery
↓
preventDefault() prevents the immediate jump
↓
showLoadingAndRedirect()
↓
Canvas freezes the previews
↓
ESTABLISHING UPLINK + progress
↓
navigation to Gallery
```

12.12 Guided example: Alex returns to Intro from the menu

VIEW INTRO uses the same transition system, but first stores `sessionStorage.fromMenu = true`. When Intro opens, it reads and removes that value. Therefore, if Alex clicks CONTINUE after entering from the menu, he returns to Home instead of going through Faction Selection again.

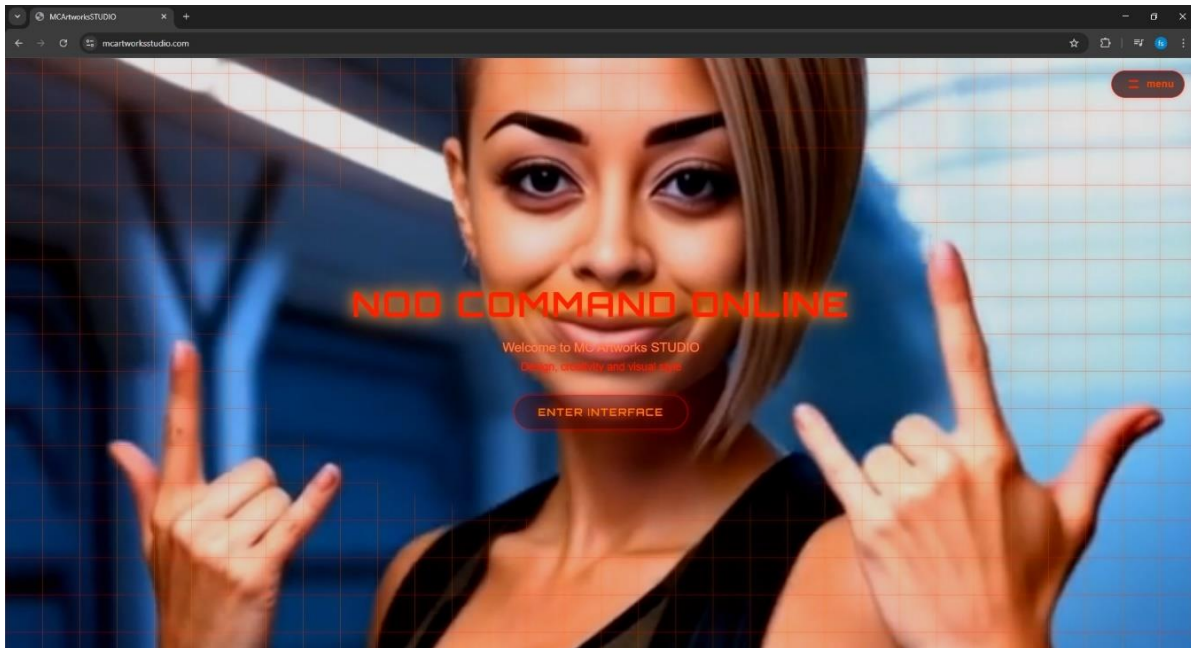
```
Alex opens MENU
↓
VIEW INTRO
↓
sessionStorage.fromMenu = true
↓
showLoadingAndRedirect(/intro/)
↓
Intro reads and removes fromMenu
↓
Alex clicks CONTINUE
↓
Home
```

13. Home

The home page changes its video according to the stored faction. GDI uses a different HLS video from NOD. That video fills the main background, and the ENTER INTERFACE button opens an enlarged view of the content.

Project location: `wp-content/themes/blocksy-child/functions.php` · `add_action('wp_head', 'home')` around line 381 activates `home()`; `mostrar_overlay_faccion_inicio()` is also registered on `wp_head` around line 667. Both components read the faction before outputting Home HTML/CSS/JavaScript.

In web design, the main block occupying the most visible area of a page is commonly called the Hero. In MC Artworks Studio, that block is not a static image: it combines video, text, and overlaid controls.



NOD Home with content over the main video and access through ENTER INTERFACE.

13.1 When the home page is activated

Source code: wp-content/themes/blocksy-child/functions.php → add_action('wp_head', 'home') around line 381 → home() from line 391. WordPress calls home() while processing wp_head; the function returns immediately when the request is not for the front page.

The home() function is connected to the wp_head hook, but before generating the interface it checks that WordPress is displaying the front page. If the request belongs to another page, the function ends immediately. This prevents the HLS video, modal, and scripts from loading in areas where they are not needed.

```
add_action('wp_head', 'home');

function home() {
    if (!is_front_page() && !is_home()) return;
    // ... Home interface
}
```

Clear example. If the visitor enters /gallery/, the condition fails and Home does not generate its player. If the visitor enters the front page, the function continues and builds the audiovisual interface.

13.2 Selecting content according to the faction

Source code: functions.php → home() around lines 395–400. PHP reads \$_COOKIE['faction'] on the server and, before outputting the response, calculates \$title_text and \$video_src.

Home reads the faction cookie and uses that value to decide the title and main video. The choice does not change a WooCommerce product or a database entry; it changes the visual experience generated for that visit.

```
$faction = $_COOKIE['faction'] ?? 'gdi';
$title_text = ($faction === 'gdi') ? 'GDI' : 'NOD';
$video_src = ($faction === 'gdi')
    ? '/videos/gdi-home/gdi-home.m3u8'
    : '/videos/nod-home/nod-home.m3u8';
faction = gdi
↓
GDI title + GDI HLS video

faction = nod
↓
NOD title + NOD HLS video
```

13.3 How the Hero is assembled

Source code: functions.php → home() → HTML output approximately between lines 403–432. The Hero and modal originate in the server response; the events that make them interactive are connected later in the JavaScript within the same function.

The main block combines three layers: an oval container holding the video, initialization text, and the final content with the title, subtitle, and ENTER INTERFACE. The video starts muted to make use of the autoplay behavior allowed by browsers.

```
<section class="hero-video-section">
  <div class="oval-video-wrapper">
    <div class="home-loading-text">INITIALIZING ...</div>
    <video autoplay muted loop playsinline id="header-video"></video>
  </div>
  <div class="hero-content" id="hero-content">
    <h1>GDI / NOD COMMAND ONLINE</h1>
    <a class="hero-hud-btn">ENTER INTERFACE</a>
  </div>
</section>
```

13.4 Preloading the first segment

Source code: functions.php → home() → <script> executed on DOMContentLoaded from approximately line 437; manual preloading of the manifest/first segment is around lines 444–454. PHP outputs the script and the browser performs the fetch() calls.

The code attempts to have data available before playback begins in order to reduce the wait for the first frame. In HLS, each segment is a small part of the video. Having the first one ready allows playback to begin without waiting for the entire content to download.

In the actual implementation, the HLS manifest is first downloaded as text. A regular expression locates the first .ts file, its path is rebuilt from the same directory as the .m3u8, and a second fetch is performed to start that download early.

```
fetch(hlsSource)
  .then(response => response.text())
  .then(manifest => {
    const firstSegment = manifest.match(/^(?!#).*\.ts/m);
    if (firstSegment && firstSegment[0]) {
      const basePath = hlsSource.substring(0, hlsSource.lastIndexOf('/') + 1);
      fetch(basePath + firstSegment[0]);
    }
  })
  .catch(() => {});
```

13.5 Initialization with HLS.js and native fallback

Source code: functions.php → home() → JavaScript function initHLS(videoElement), approximately lines 457–471. It receives each <video>, chooses between HLS.js and native HLS support, and connects the same HLS source to both the Hero and the modal.

After obtaining the .m3u8 path, JavaScript attempts to play it with HLS.js. If the browser already supports HLS natively, it assigns the playlist directly to the video element. These are two different paths to the same result: playing HLS content without downloading an entire MP4 file at once.

```
if (Hls.isSupported()) {
  const hls = new Hls({ maxBufferLength: 10, startLevel: 0 });
  hls.loadSource(hlsSource);
  hls.attachMedia(videoElement);
} else if (videoElement.canPlayType('application/vnd.apple.mpegurl')) {
  videoElement.src = hlsSource;
}
```

13.6 When the loading screen disappears

Source code: functions.php → home() → JavaScript approximately between lines 473–520. startIntroSequence() is triggered by canplay/loadeddata/play; those events belong to the <video> element already loaded in the browser.

The home page does not expand the video as soon as the HTML element is created. It waits until the browser has enough data to draw a frame. The `canplay` and `loadeddata` events activate `startIntroSequence()`, hide the initialization text, expand the oval, and show the Hero content.

```
HLS begins loading
↓
canplay / loadeddata
↓
startIntroSequence()
↓
loading text disappears
↓
the oval expands
↓
ENTER INTERFACE appears
```

13.7 What ENTER INTERFACE does

Source code: `functions.php` → `home()` → click listener for `.hero-hud-btn` around lines 522–554. The click does not call WordPress or change the URL: it modifies the DOM, locks MENU, and plays `fullVideo` inside the current modal.

The button does not navigate to another page. It opens a fullscreen modal with the same HLS content, temporarily blocks interaction with the rest of the page, and disables the MENU button while the modal is active.

```
ENTER INTERFACE
↓
gdiModal receives class="active"
↓
body receives no-interaction
↓
MENU is disabled
↓
fullVideo returns to 0 and starts playing
```

13.8 Closing the modal and NOD behavior

Source code: `functions.php` → `home()` → NOD timers from approximately line 573 and `#closeModal` close listener around line 605. Everything occurs in the same `DOMContentLoaded` callback that opened the modal; on close, classes, playback, timers, and the temporary image are cleaned up.

When X is clicked, the modal closes, the video is paused, and MENU becomes available again. NOD also uses a temporary image (`wuhrer-kucan.jpg`) that appears during playback. The code maintains dedicated timers to show, hide, and remove it without leaving duplicate elements. Closing is not limited to hiding the modal: it also restores general interaction, re-enables MENU, and clears the NOD timers and temporary image so that no effects remain pending after the fullscreen view is closed.

```
closeBtn.addEventListener('click', () => {
  modal.classList.remove('active');
  document.body.classList.remove('no-interaction');
  fullVideo.pause();

  if (menuBtn) {
    menuBtn.disabled = false;
    menuBtn.classList.remove('zaj-menu--disabled');
  }

  clearTimeout(nodShowTimeout);
  clearTimeout(nodHideTimeout);
  clearTimeout(nodRemoveTimeout);
  document.querySelectorAll('.nod-fade-image')
    .forEach(img => img.remove());
});
```

In GDI, the fullscreen video restarts when it ends. NOD also includes a restart control so that the temporary image can be scheduled again when the video begins another cycle.

```
nodShowTimeout = setTimeout(() => {  
  nodImage.classList.add('visible');  
  
  nodHideTimeout = setTimeout(() => {  
    nodImage.classList.remove('visible');  
    nodRemoveTimeout = setTimeout(() => nodImage.remove(), 1500);  
  }, 6000);  
}, 16500);
```

13.9 Entry from faction selection

Source code: functions.php → add_action('wp_head', 'mostrar_overlay_faccion_inicio') around line 667 → mostrar_overlay_faccion_inicio() from approximately line 679. This component runs on a new Home load, reads the cookies, and deletes from_faction_selection after deciding which overlay to show.

When Home opens immediately after GDI or NOD is confirmed, the temporary from_faction_selection cookie makes it possible to show a faction-specific overlay before revealing the home page. The cookie is then deleted so that a simple reload does not show the overlay again.

```
YES in Faction Selection  
↓  
faction cookie + temporary cookie  
↓  
Home  
↓  
GDI or NOD overlay  
↓  
from_faction_selection is deleted
```

PART III — SHOP, PRODUCTS, AND BUNDLES

14. How products are managed with WooCommerce

WooCommerce is the source of the commercial product: it maintains the product identifier, name, price, status, and relationship with the cart and orders. The child theme changes the presentation and adds custom behavior, but it does not create a second parallel catalog.

To understand the flow, it is useful to follow how the representation of the same product changes. In Shop, the code works with a `WC_Product` object; after ADD TO CART, a temporary cart item appears; and when Checkout is confirmed, WooCommerce creates a persistent line item within the `WC_Order`.

14.1 What a `WC_Product` is

`WC_Product` is the class WooCommerce uses to represent a product in PHP. Instead of manually querying MariaDB tables for a name, price, or ID, the theme receives an object and calls its methods. This also decouples the code from internal storage details.

```
global $product;

$product_id = $product->get_id();
$title      = $product->get_name();
$price      = $product->get_price();
```

The object is not the HTML card. The card is a visual representation generated later. `WC_Product` is the domain object that identifies the product WooCommerce is processing.

14.2 Where global `$product` comes from in Shop

WooCommerce runs a loop to display catalog products. During each iteration, it makes the current product available in the global `$product` variable and fires hooks around its card. The child theme uses one of those hooks, `woocommerce_before_shop_loop_item_title`, to execute `mostrar_videos_shop()` once for each visible product. The following block is an abbreviated source-code fragment: it preserves the actual hook, main condition, and ACF read, but omits the rest of the HTML and JavaScript that build the preview.

```
add_action(
    'woocommerce_before_shop_loop_item_title',
    'mostrar_videos_shop',
    10
);

function mostrar_videos_shop() {
    if (!is_shop()) return;

    global $product;
    $video_url = get_field('video_destacado', $product->get_id());
    if (!$video_url) return;

    // Abbreviated fragment: preview construction continues.
}
```

The consequence is important: `mostrar_videos_shop()` does not search for “which product matches” by text or URL. WooCommerce has already established the context. The function takes the current object’s ID and uses that same ID to query ACF.

Project location: `functions.php` · `woocommerce_before_shop_loop_item_title` hook around line 2878 · `mostrar_videos_shop()` begins approximately at line 2897.

14.3 What WooCommerce provides and what ACF adds

Data	Primary source	Use
Product ID	WooCommerce / WC_Product	Connects catalog, ACF, cart, and order.
Name	WooCommerce / WC_Product	Commercial text and order references.
Price	WooCommerce / WC_Product	Commercial calculation and total.
Status/publication	WordPress + WooCommerce	Determines whether the product participates in the catalog.
video_destacado	ACF associated with product_id	Audiovisual preview for Shop/Bundles.
tiene_globo	ACF associated with product_id	Determines whether Cart offers Comic Balloon Text.

ACF complements the product; it does not replace it. If Cyberpunk Portrait has product_id = X, get_field('video_destacado', X) queries the metadata for that same product.

14.4 product_id is the thread connecting each stage

The identifier makes it possible to recognize the same product even when the structure representing it changes. In Shop it is obtained from WC_Product; in the cart, the cart item retains product_id; and when the order is created, WooCommerce generates a line item associated with that product.

```

WC_Product
product_id = X
↓
Shop generates the card
↓
ADD TO CART sends/reuses X
↓
cart item contains product_id = X
↓
Checkout
↓
WC_Order
↓
WC_Order_Item_Product associated with the same product

```

14.5 What “sold individually” means

The theme registers the woocommerce_is_sold_individually filter, and the current implementation returns true for all products. WooCommerce interprets that result as individual quantity: a quantity greater than one cannot be freely managed within the same cart line.

```

add_filter(
    'woocommerce_is_sold_individually',
    'forzar_artworks_individuales',
    10,
    2
);

function forzar_artworks_individuales($individually, $product) {
    return true;
}

```

Do not confuse this

“Sold individually” does not mean that only one total copy of the artwork exists in the database. It means that, within the purchase flow, WooCommerce forces quantity = 1 for that line and removes the normal quantity selector.

Project location: functions.php · woocommerce_is_sold_individually filter approximately around lines 6062–6075.

14.6 Product, cart item, and order line are not the same thing

Stage	Structure	What it represents
Catalog	WC_Product	Product managed by WooCommerce: ID, name, price, and status.
Cart	cart item	Temporary selection that references the product and can include data such as color_frame or comic_balloon_text.
Order	WC_Order_Item_Product	Persisted line inside the WC_Order containing the purchased product and its copied metadata.

This distinction explains why Frame Color and Comic Balloon Text should not be stored by changing the original WC_Product. They are decisions for a specific purchase, so they first travel with the cart item and are then copied to the order line.

14.7 Technical path of a product

```

WooCommerce obtains the product
↓
creates/exposes WC_Product
↓
loop hooks let the child theme customize the card
↓
ACF adds visual data through product_id
↓
user clicks ADD TO CART
↓
WooCommerce creates a cart item
↓
the theme adds customizations to the cart item
↓
Checkout validates
↓
WooCommerce creates WC_Order + line items
↓
hooks copy customizations to WC_Order_Item_Product

```

The theme works mainly through WooCommerce APIs and hooks instead of building manual SQL. That decision is especially important because the installation uses HPOS for orders, as explained in Section 5.

14.8 Guided example: Alex buys Cyberpunk Portrait

Assume WooCommerce manages Cyberpunk Portrait at USD 18. The product has a specific product_id, and ACF associates it with a video_destacado and tiene_globo = Yes.

1. Alex opens Shop. During the loop, WooCommerce sets Cyberpunk Portrait as the current \$product. mostrar_videos_shop() obtains its ID and queries video_destacado with get_field().
2. The theme generates the audiovisual card, but the name, price, and identifier still belong to WC_Product. The presentation changed; the commercial product did not.
3. Alex clicks ADD TO CART. WooCommerce adds the product_id to the cart and creates a cart item. Because the product is sold individually, the line uses quantity 1.
4. In Cart, Alex selects Frame Color = Blue and enters Comic Balloon Text = “Welcome back”. Those values are associated with that cart item; they do not modify the master Cyberpunk Portrait product.
5. Alex proceeds to Checkout. WooCommerce reconstructs the cart from its session, and the customizations remain associated with the corresponding line.
6. On confirmation, WooCommerce creates the WC_Order and its WC_Order_Item_Product. The theme hooks copy Frame Color and Comic Balloon Text to the order line so that they no longer depend on the temporary session.

7. Later, logs and emails can iterate over the order and display Cyberpunk Portrait together with Blue and “Welcome back”. The same product moved through the entire flow, but each stage used the appropriate structure.

14.9 What the child theme does not create

Core idea

blocksy-child does not maintain its own catalog in parallel with WooCommerce. It customizes how the product is queried, displayed, and accompanied through the purchase process, but WooCommerce remains the commercial source of truth.

15. Custom fields with ACF

Advanced Custom Fields (ACF) makes it possible to add custom data to products. Section 5 explained how these values are stored; here the focus is on how the child theme uses them to decide which video to display, whether to enable Comic Balloon Text, and which products make up a bundle.

Source of the code shown in this section: ACF values are not read from a single custom class. The theme queries them from functions.php in the flows that need them: Shop uses `video_destacado` inside `mostrar_videos_shop()`; Bundles uses `productos_del_bundle` and `video_destacado`; Cart/Emails query `tiene_globo`. Each subsection should be read as a `get_field()` call made from the indicated technical flow, not as an independent ACF process.

ACF field	What it represents	Example use
<code>video_destacado</code>	Main video associated with the product.	The Shop and Bundles card uses it as a preview.
<code>video_workshop</code>	Optional Workshop video.	Optional field available for storing a Workshop resource associated with the product.
<code>productos_del_bundle</code>	Relates a bundle to other products.	The bundle card iterates through the associated artworks.
<code>tiene_globo</code>	Indicates whether the artwork supports balloon text.	When enabled, Cart displays Comic Balloon Text.

What metadata means

Metadata is additional information associated with another primary piece of data. Here, the WooCommerce product is the primary element, while “has balloon” or “featured video” are additional data describing how it should behave or be displayed.

Example

Product: Cyberpunk Portrait → `video_destacado` = `/uploads/.../cyberpunk.mp4` → `tiene_globo` = Yes. The same product can have audiovisual behavior and custom options while remaining a simple WooCommerce product.

15.1 Reading `video_destacado`

The Shop obtains the main video with `get_field()`. If the field has no value, `mostrar_videos_shop()` ends and does not attempt to create an empty player.

```
$video_url = get_field('video_destacado', $product->get_id());
if (!$video_url) return;
```

15.2 The `tiene_globo` field

`tiene_globo` acts as an interface decision. When enabled, the product supports Comic Balloon Text in Cart and that data can be passed to the order. When disabled, balloon text is hidden from summaries where it does not apply.

```
$tiene_globo = (bool) get_field('tiene_globo', $product_id);

if ($tiene_globo) {
```

```
// display or preserve Comic Balloon Text
}
```

15.3 The productos_del_bundle relationship

A bundle uses `productos_del_bundle` to determine which artworks it should represent inside its card. The bundle remains the product being purchased; the related products are used to visually show its contents.

```
$productos_in_bundle = get_field('productos_del_bundle', $bundle_id);

foreach ($productos_in_bundle as $prod) {
    $video_url = get_field('video_destacado', $prod_id);
}
```

15.4 The video_workshop field

`video_workshop` is available as an additional field for associating a Workshop resource with a product. In the current version, it does not participate in the main Shop, Cart, or Checkout flow, so it is documented as available data rather than as a purchase requirement.

Difference between the code and the ACF configuration

The `wpo_filtrar_meta_en_pedido()` filter expects an ACF field named `es_bundle`, and the comment in `functions.php` itself presents it as a dependency together with `tiene_globo`. However, the current review of the installation identifies only four product ACF fields, and `es_bundle` is not among them. If that field does not exist, `get_field('es_bundle', $product_id)` returns an empty value and this function no longer recognizes bundles through that route; filtering Comic Balloon Text then depends on `tiene_globo`. Elsewhere in the project, bundles are identified by category or name. This difference is part of the project's current state.

16. Shop and product cards

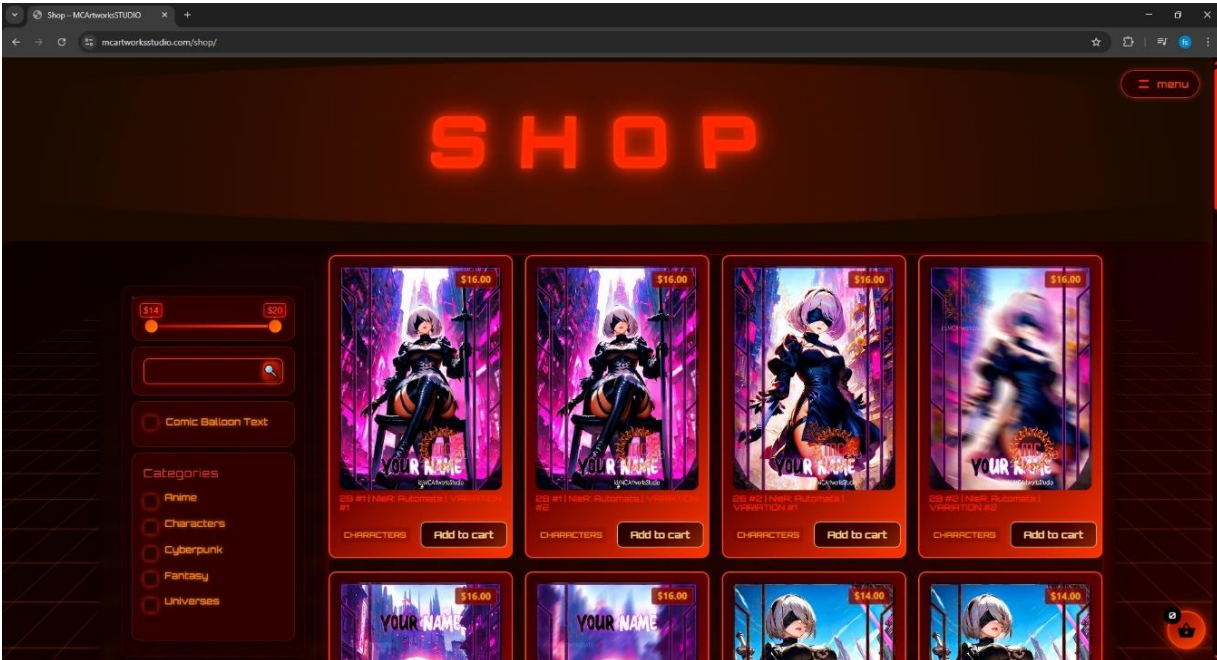
The Shop uses the standard WooCommerce catalog and loop as its foundation, but MC Artworks Studio modifies which products participate, how many are shown per page, and how each artwork is represented visually.

WooCommerce retains the query, product objects, and commercial buttons; the child theme adds the grid, videos, posters, overlaid price, animations, and audiovisual behavior.

Project location: `wp-content/themes/blocksy-child/functions.php` · `ocultar_bundles_de_shop()` is registered on `pre_get_posts` around line 2844; `mostrar_videos_shop()` is connected to `woocommerce_before_shop_loop_item_title` around line 2878 and begins near line 2897; `productos_por_pagina()` is registered on `loop_shop_per_page` around lines 3315–3320. The Shop entrance animation is activated from `wp_footer` around line 2021.

How to read the fragments in this section. WooCommerce creates the query and loop; `functions.php` hooks intervene at specific points in that flow. Therefore, an `add_action()` or `add_filter()` shown here registers a callback; it does not mean that the code executes at the moment WordPress reads that line.

```
WooCommerce queries products
↓
the Bundles category is excluded
↓
WooCommerce generates the loop
↓
<ul class="products">
↓
<li class="product"> for each artwork
↓
the child theme adds video + price + effects
```



NOD Shop: filter sidebar and artwork grid with video previews.

16.1 Which products participate in the Shop

Before running the main query, the theme uses the `pre_get_posts` hook to exclude products belonging to the Bundles category. In this way, bundles remain WooCommerce products but are reserved for their dedicated page.

```
add_action('pre_get_posts', 'ocular_bundles_de_shop');

$tax_query[] = [
    'taxonomy' => 'product_cat',
    'field'    => 'slug',
    'terms'    => ['bundles'],
    'operator' => 'NOT IN',
];
```

The NOT IN condition means that products associated with the Bundles category are excluded from the main Shop loop.

The complete function does more than add NOT IN. It first limits the change to the main Shop query and preserves any `tax_query` conditions that WordPress or WooCommerce may already have added. This prevents secondary queries or administrator screens from being modified.

```
function ocular_bundles_de_shop($query) {
    if (!is_admin() && $query->is_main_query() && is_shop()) {
        $tax_query = (array) $query->get('tax_query');
        $tax_query[] = [
            'taxonomy' => 'product_cat',
            'field'    => 'slug',
            'terms'    => ['bundles'],
            'operator' => 'NOT IN',
        ];
        $query->set('tax_query', $tax_query);
    }
}
```

How to read the example. `is_admin()` prevents changes to the admin panel; `is_main_query()` limits the change to the main loop; `is_shop()` restricts it to the Shop; `get()` preserves existing conditions; NOT IN excludes Bundles; `set()` returns the modified query to WordPress.

WordPress prepares the Shop query
↓

```
pre_get_posts
↓
frontend + main query + Shop?
↓ yes
preserve existing tax_query
↓
add NOT IN bundles
↓
WooCommerce executes the query
```

16.2 Number of products per page

WooCommerce controls catalog pagination through the `loop_shop_per_page` filter. MC Artworks Studio sets the value to 28 products per page and uses a high priority so that this number takes precedence over other changes to the same filter.

```
add_filter('loop_shop_per_page', 'productos_por_pagina', 999);

function productos_por_pagina($cols) {
    return 28;
}
```

16.3 WooCommerce builds the loop; CSS builds the grid

WooCommerce generates a `ul.products` list and one `li.product` for each record. The child theme does not replace that mechanism: it applies CSS Grid to the existing structure to turn it into a responsive grid.

```
body.woocommerce-shop ul.products {
    display: grid;
    grid-template-columns:
        repeat(auto-fill, minmax(220px, 1fr));
    gap: 20px;
}
```

`repeat(auto-fill, minmax(220px, 1fr))` allows the browser to calculate how many columns fit according to the available width. On narrower screens, the grid becomes a single column. The filter sidebar occupies the left area and can remain visible through position: sticky while the user browses the results.

16.4 Building an audiovisual card

WooCommerce executes the `woocommerce_before_shop_loop_item_title` hook before each product title. The theme connects `mostrar_videos_shop()` there, so the function runs once for every artwork visible in the loop.

```
add_action(
    'woocommerce_before_shop_loop_item_title',
    'mostrar_videos_shop',
    10
);
```

The function obtains the current product, queries the ACF field `video_destacado`, and, if it exists, generates a link that acts both as the card preview and as the source of the video that Fancybox can later enlarge.

```
global $product;

$video_url = get_field(
    'video_destacado',
    $product->get_id()
);

if (!$video_url) return;
```

16.5 Poster according to the faction

Before the MP4 is ready, a poster is displayed. The path is chosen from the faction cookie: GDI uses irina_poster.jpg and NOD uses poster.jpg, both inside wp-content/uploads/images. This lets the same card preserve the selected visual identity even while the video is initially loading.

```
$faction = $_COOKIE['faction'] ?? 'gdi';

if ($faction == 'gdi')
    $poster_path = $uploads_dir['baseurl'] . '/images/irina_poster.jpg';
else
    $poster_path = $uploads_dir['baseurl'] . '/images/poster.jpg';
```

16.6 HTML generated for video, price, and Fancybox

```
<a href="VIDEO.mp4"
  data-fancybox="videos-tienda"
  class="video-loop-link video-loop-wrapper">

  <div class="video-container">
    <video
      poster="POSTER.jpg"
      data-src="VIDEO.mp4"
      preload="metadata"
      muted autoplay playsinline
      class="video-loop">
    </video>

    <span class="price precio-video">
      PRECIO
    </span>
  </div>
</a>
```

The MP4 URL is used in two ways: href allows the artwork to be opened in Fancybox, while data-src stores the preview source without forcing the browser to activate it immediately. The price is obtained from WooCommerce with get_price() and output with wc_price().

16.7 Title and commercial buttons

The product title is output with the standard woocommerce-loop-product__title class, but without turning it into a link. Add to Cart buttons remain WooCommerce buttons, preserving AJAX, validation, added_to_cart events, and Xootix integration.

```
echo '<h2 class="woocommerce-loop-product__title">'
    . esc_html($product->get_title())
    . '</h2>';
```

The commercial buttons are not a new cart system. They remain WooCommerce controls. The theme changes their visual state according to whether the product is already in WC()->cart and listens for AJAX events to update the card without reloading the page.

Product not in the cart
→ ADD TO CART

```
click
↓
WooCommerce AJAX
↓
product added
↓
added_to_cart event
↓
ADD TO CART is hidden
↓
VIEW CART appears
```

Clear example. Alex adds Cyberpunk Portrait. WooCommerce keeps the product in its real cart. Xootix opens the side panel, and the same Shop card stops offering ADD TO CART and displays VIEW CART instead. If Alex removes the product through Xootix, the card re-enables ADD TO CART.

16.8 Category displayed next to the product

The theme can retrieve the categories associated with the product and display them next to the commercial controls. The information comes from WooCommerce taxonomies; it is not duplicated in a custom theme field.

```
$terms = get_the_terms($product_id, 'product_cat');
// The categories returned by WooCommerce are used by the interface.
```

16.9 Visual class when a video exists

The `post_class` filter can add the `tiene-video` class to the card when `video_destacado` exists. This class lets CSS distinguish an audiovisual card from one that would depend on WooCommerce's standard product image.

```
product
↓
does video_destacado have a value?
├ yes → class="tiene-video" → the video preview is prioritized
└ no → the available visual behavior is preserved
```

16.10 Random entrance animations

The Shop includes five animation functions. At the end of the page, `animacion_random_shop()` checks that products actually exist and selects a variant at random. The animations control the sidebar and card appearance, including progressive delays that create a cascading effect.

```
$animaciones = [
    'animacion_shop_uno',
    'animacion_shop_dos',
    'animacion_shop_tres',
    'animacion_shop_cuatro',
    'animacion_shop_cinco'
];

$seleccion = $animaciones[array_rand($animaciones)];
```

17. Progressive video loading and Fancybox

A catalog page can contain dozens of MP4 files. To avoid downloading all of them at once, the Shop uses progressive loading: the HTML keeps the URL in `data-src` and JavaScript decides when to turn it into an actual media source. The same video can later be opened in a Fancybox modal without leaving the grid.

Project location: `wp-content/themes/blocksy-child/functions.php` · `cargar_fancybox_tienda()` is registered on `wp_enqueue_scripts` near the beginning of the file; `video_lazy_loop_shop()` is registered on `wp_footer` around line 2940; `fix_videos_chrome_shop_universal()` around 3039; `custom_fancybox_video_script()` around 3972; `cerrar_fancybox_con_jquery()` around 4142; and `cf_fix_cart_over_fancybox()` around 4173.

17.1 Why data-src is used

```
<video
  data-src="artwork.mp4"
  preload="metadata"
  muted
  playsinline>
</video>
```

As long as the address remains in `data-src`, the theme can decide when to activate it. The browser does not receive a direct `src` for every card from the outset.

```
Initial HTML
  data-src = artwork.mp4
  src      = empty
    ↓
JavaScript detects proximity
    ↓
  src = data-src
    ↓
actual loading begins
```

17.2 Videos visible when the Shop opens

On `DOMContentLoaded`, the code searches for all `video[data-src]` elements. For each one, its position is calculated with `getBoundingClientRect()`. If it is already inside the viewport, `reproducirVideo(video, true)` prepares it with an initial delay of approximately 850 ms.

```
const rect = video.getBoundingClientRect();
const inViewport =
  rect.top < window.innerHeight
  && rect.bottom > 0;

if (inViewport) {
  reproducirVideo(video, true);
}
```

That small delay keeps the poster visible while the browser finishes preparing the first resources and avoids an abrupt entrance of several videos at the same time.

17.3 IntersectionObserver for videos not yet visible

Videos located farther down are passed to `IntersectionObserver`. The `rootMargin: "200px"` option expands the detection area, allowing the resource to begin loading before it physically enters the screen.

```
const observer = new IntersectionObserver(
  (entries, obs) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        reproducirVideo(entry.target, false);
        obs.unobserve(entry.target);
      }
    });
  },
  { rootMargin: "200px" }
);
```

Once activated, the video is no longer observed because it no longer needs to go through the lazy-loading process again.

17.4 Activating the MP4

```
if (!video.src) {
  video.src = video.getAttribute('data-src');
}

video.preload = 'auto';
video.currentTime = 0.01;
video.play();
```

The `data-src` → `src` step is the specific moment when the browser receives the real source. Playback begins near the first frame, and the active class produces an opacity fade of approximately 0.35 seconds.

17.5 Manual loop

The code also controls the end of each preview. When approximately 0.05 seconds remain, it pauses, jumps back to 0.01 s, and requests the next playback through `requestAnimationFrame()`. The goal is to make the transition between loops more stable.

```
video.addEventListener('timeupdate', () => {
  if (video.currentTime >= video.duration - 0.05) {
    video.pause();
    video.currentTime = 0.01;
    requestAnimationFrame(() => video.play());
  }
});
```

17.6 Exit handling in Chrome and Opera

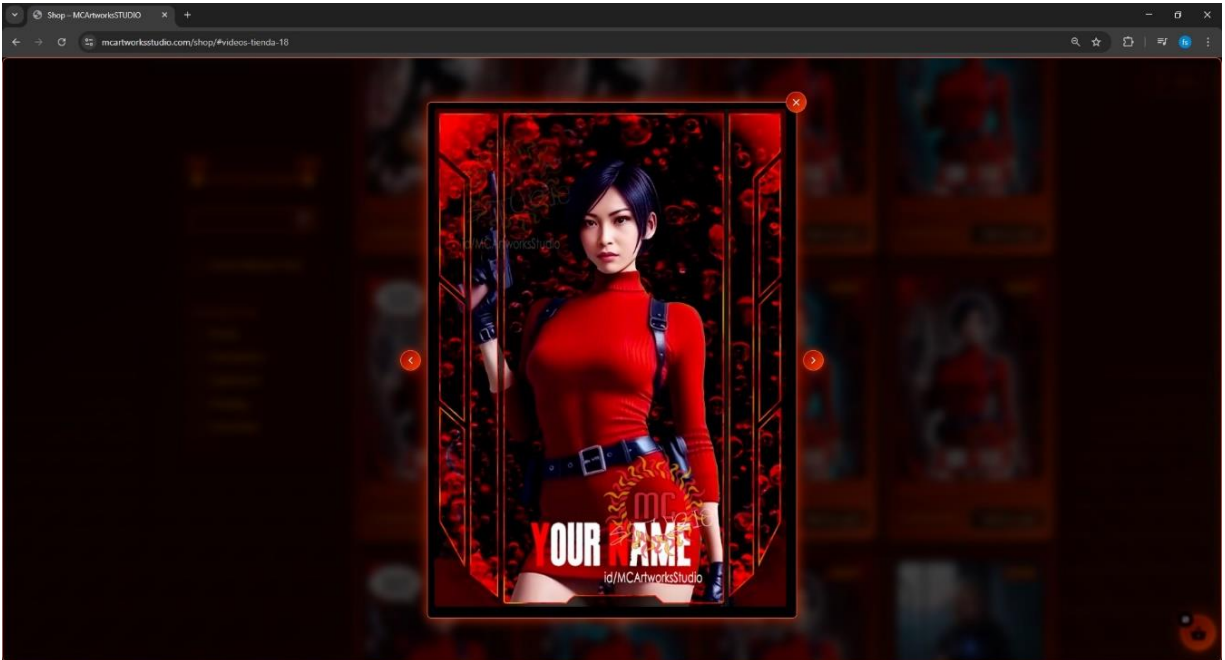
The `fix_videos_chrome_shop_universal()` function acts in the Shop to avoid black frames when leaving the page in Chrome or Opera. Before navigating, it attempts to capture the visible frame in a canvas; if that fails, it uses the poster as a fallback. It also separately accounts for videos that Xootix may contain inside the side cart.

```
visible video
↓
canvas.drawImage(video, ...)
↓
the frame remains overlaid
↓
navigation begins
↓
no black frame appears
```

17.7 Conditional loading of Fancybox

Fancybox is registered through `wp_enqueue_scripts` only when WordPress is displaying the Shop, a product category, or a product tag. The library is obtained from jsDelivr.

```
if (
  is_shop()
  || is_product_category()
  || is_product_tag()
) {
  wp_enqueue_style('fancybox-css', ...);
  wp_enqueue_script('fancybox-js', ...);
}
```



Enlarged view of an artwork through the Fancybox video modal.

17.8 How a card opens

Each card contains `data-fancybox="videos-tienda"`. The script binds those elements with `Fancybox.bind()` and, when it detects an MP4, replaces the slide content with its own HTML5 player.

```
Fancybox.bind(
  '[ data-fancybox="videos-tienda"]',
  {
    dragToClose: false,
    width: '90vw',
    height: '80vh'
  }
);
```

17.9 Custom modal and navigation

The modal adds its own close button and two side buttons. The actions call `fancybox.prev()` and `fancybox.next()`; `ArrowLeft` and `ArrowRight` are also accepted from the keyboard. This allows the Shop videos to be browsed without closing the enlarged window between one artwork and the next.

```
Artwork anterior ← VIDEO AMPLIADO → Artwork siguiente

ArrowLeft → prev()
ArrowRight → next()
```

17.10 Background locking and Xootix integration

When Fancybox is revealed, the `fancybox-lock` class is added to the body and a transparent layer blocks interaction with the background. On close, both are removed. Another function, `cf_fix_cart_over_fancybox()`, allows the Xootix side cart to appear above the modal without reinitializing Fancybox.

```

Fancybox open
↓
user opens the cart
↓
Xootix receives visual priority
↓
the cart remains interactive
↓
Fancybox remains open behind it

```

17.11 Guided example: Alex scrolls through Shop and opens an artwork

Alex enters Shop. PHP has already generated the cards with data-src, but many MP4 files do not yet have an active src. Videos that are visible are prepared when the page loads, while those farther down remain under IntersectionObserver.

When Alex scrolls, a card enters the area expanded by rootMargin. JavaScript copies data-src to src, calls load()/play(), and stops observing that element. If Alex clicks the preview, the data-fancybox link provides the same MP4 URL to Fancybox, which creates the enlarged player without taking Alex out of Shop.

```

Alex opens Shop
↓
PHP delivers cards with data-src
↓
Alex scrolls
↓
IntersectionObserver detects proximity
↓
data-src becomes src
↓
the preview plays
↓
Alex clicks the video
↓
Fancybox.bind() opens the modal
↓
Alex navigates Previous / Next or closes it

```

18. Shop filters with HUSKY and AJAX

HUSKY/WOOF provides the filtering engine for WooCommerce. MC Artworks Studio uses that infrastructure and adds custom logic for text, visible categories, text search, interface blocking, price range, and restoration of elements after dynamic updates.

Project location: wp-content/themes/blocksy-child/functions.php · the HUSKY adaptations are distributed mainly between approximately lines 1282 and 1955 through wp_footer/wp_head callbacks; the custom “no results” state is registered later from init around line 2646. The HUSKY plugin generates its own controls and AJAX; functions.php modifies its interface and reattaches behaviors when the plugin replaces the DOM.

18.1 Visible categories

The theme changes the filter heading to Categories and removes Uncategorized and Bundles from the visible list. Bundles is not offered here because it has its own page and filtering system.

```

const categoriasOcultas = [
  'Uncategorized',
  'Bundles'
];

```

18.2 Comic Balloon Text inside HUSKY

The filter associated with the tiene_globo field may appear after HUSKY rebuilds part of the DOM. For that reason, a MutationObserver waits for the corresponding label and replaces its text with Comic Balloon Text. Once the change has been made, the observer disconnects.

```
const label = document.querySelector(
  'label[for="woof_meta_checkbox_tiene_globo"]'
);

label.textContent = 'Comic Balloon Text';
```

18.3 Search field and magnifying-glass button

The theme locates `.woof_husky_txt-input` and dynamically adds a search button. Because HUSKY can recreate that input after filters are applied, another `MutationObserver` detects the new version and adds the magnifying glass again without duplicating it.

The button is not added only once when the page loads. HUSKY can replace the input during an AJAX update, so the code observes new nodes and reconnects the magnifying glass when it detects a new instance of the search field.

```
function findInput() {
  return document.querySelector('.woof_husky_txt-input');
}

function agregarLupaSiInputListoEl(inputEl) {
  if (!inputEl) return;
  if (inputEl.parentElement?.querySelector('#btn_search')) return;

  const btn = document.createElement('button');
  btn.id = 'btn_search';
  btn.type = 'button';
  btn.textContent = '🔍';
  inputEl.parentElement.appendChild(btn);
}

const observer = new MutationObserver(mutations => {
  for (const m of mutations) {
    for (const node of m.addedNodes) {
      if (!(node instanceof HTMLElement)) continue;
      const found = node.matches?.('.woof_husky_txt-input')
        ? node
        : node.querySelector?.('.woof_husky_txt-input');
      if (found) agregarLupaSiInputListoEl(found);
    }
  }
});

observer.observe(document.body, { childList: true, subtree: true });
```

18.4 Minimum of three characters

Text search requires at least three characters both when the magnifying glass is clicked and when Enter is pressed. If the value is too short, execution stops and the custom alert opens.

```
const val = (input.value || '').trim();

if (val.length < 3) {
  mostrarAlertaPersonalizada(
    'Please enter at least 3 characters.'
  );
  return false;
}
```

Example: “cy” is blocked because it contains two characters; “cyr” can continue into the HUSKY flow.

18.5 Intercepting `woof_submit_link()`

HUSKY can execute its global `woof_submit_link()` function directly. To ensure that validation is also respected through that path, the project preserves the original function and places a custom function in front of it. If the text is valid, `original.call(this, link)` is called; otherwise, the filter is canceled.

```
const original = window.woof_submit_link;

window.woof_submit_link = function(link) {
  ... validation ...
  return original.call(this, link);
};
```

18.6 What AJAX provides

When HUSKY applies a filter, it can update the results area without reloading the entire page. WooCommerce obtains the matching products again, and the plugin replaces the corresponding part of the grid. The menu, background, and rest of the interface remain on the same page.

```
user changes a filter
↓
HUSKY prepares parameters
↓
AJAX request
↓
WooCommerce returns results
↓
HUSKY replaces the grid
```

18.7 Visual blocking while filtering

When the user interacts with checkboxes, radio buttons, price controls, or the filter button, `loading-active` is added to the body. A `screen-blocker` prevents further interaction while the change is being processed. Blocking is also applied during `beforeunload` to cover cases where the operation ends in a full navigation.

Blocking is activated only when the interaction belongs to a HUSKY control that can start an update. The same state is forced during `beforeunload` to cover cases where the filter ultimately triggers a full navigation.

```
const blocker = document.createElement('div');
blocker.className = 'screen-blocker';
document.body.appendChild(blocker);

function activarBloqueo() {
  if (!document.body.classList.contains('loading-active')) {
    document.body.classList.add('loading-active');
  }
}

const validSelectors = [
  '.woof_checkbox',
  '.woof_radio',
  '.woof_price_filter input',
  '.woof_submit_search_form'
].join(',');

document.addEventListener('click', e => {
  if (e.target.closest(validSelectors)) activarBloqueo();
});

document.addEventListener('change', e => {
  if (e.target.closest(validSelectors)) activarBloqueo();
});

window.addEventListener('beforeunload', () => {
  document.body.classList.add('loading-active');
});
```

18.8 Price range

The Shop forces the HUSKY slider to an overall USD 14–20 range. If the URL already contains a range, for example `price-14-to-18`, JavaScript restores those values and updates `ionRangeSlider`. After the `woof_ajax_done` event, the range is applied again because HUSKY may rebuild the control.

```

slider.setAttribute('data-min', '14');
slider.setAttribute('data-max', '20');

jQuery(document).on(
  'woof_ajax_done',
  () => setTimeout(forzarRangoWoof, 300)
);

```

18.9 No results

When no products remain, the standard WooCommerce message is replaced with a custom presentation. The image changes according to the faction, and the result is treated as a normal search state rather than as a site error.

NOD → NOD “no results” image

GDI → GDI “no results” image

18.10 Guided example: Alex uses HUSKY search

Alex types “cy” in the search field. The custom listener detects two characters, stops submission, and displays the project alert. He then types “cyr”. Validation allows the process to continue, and HUSKY runs its filtering flow.

If HUSKY updates the grid through AJAX, it may replace entire nodes. For that reason, the MutationObserver instances and the woof_ajax_done event reapply elements such as the magnifying glass, the Comic Balloon Text label, and the price range after the update.

```

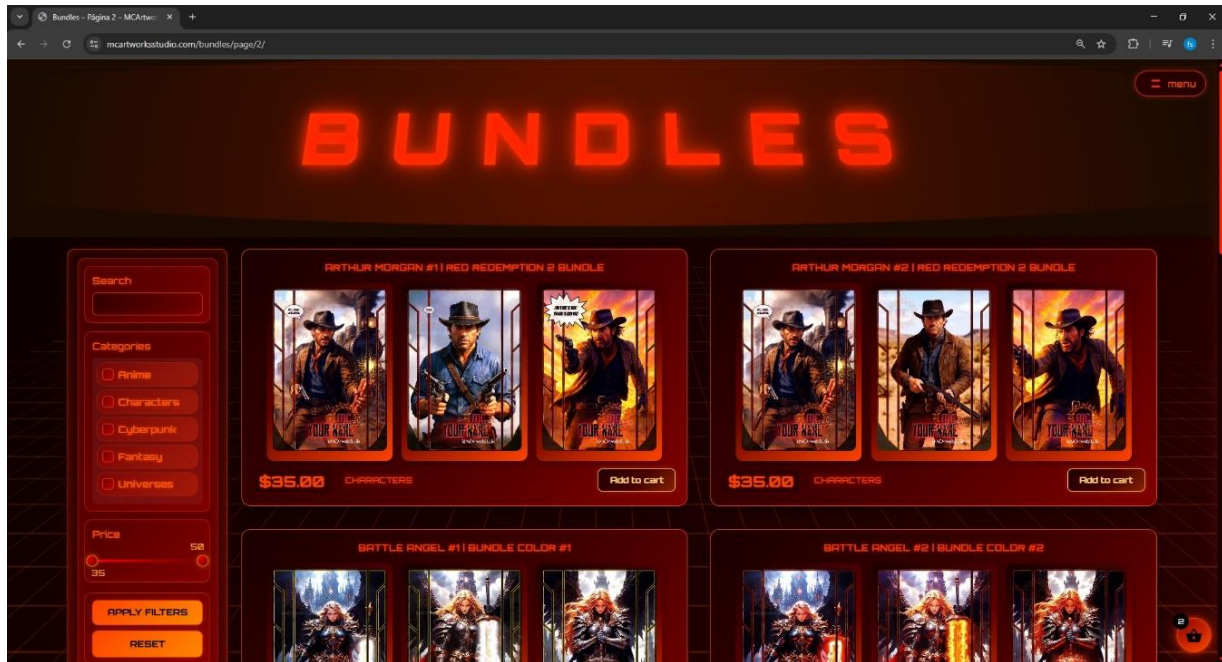
Alex types “cyr”
↓
custom validation: 3 or more characters
↓
HUSKY prepares filters
↓
AJAX request
↓
WooCommerce obtains matching products
↓
HUSKY replaces the grid
↓
theme callbacks/observers reapply adjustments

```

19. Bundles

The Bundles page uses WooCommerce products from the Bundles category, but it does not visually reuse the standard Shop loop. The `mostrar_bundles_filtrados_rapido()` function replaces the page content with a custom sidebar, queries bundles through `wc_get_products()`, applies filters, paginates the results, and manually builds each block with its related artworks.

Project location: `wp-content/themes/blocksy-child/functions.php · add_filter('the_content', 'mostrar_bundles_filtrados_rapido', 21)` appears around line 4454 and the function begins near line 4466. Lazy loading for its videos is connected from `wp_footer` around line 4828; other visual and cart synchronizations are distributed approximately between lines 5401 and 5795.



NOD Bundles: custom filters and blocks with several related artworks.

19.1 Filter parameters

The page receives search text, price, and categories through GET parameters. Values are sanitized before use. Selected categories are compared against the actual product_cat slugs to discard values that do not belong to the taxonomy.

```
$q          = sanitize_text_field(...);
$min_price = floatval(...);
$max_price = floatval(...);
$cats_raw  = (array) $_GET['cat'];
```

19.2 Product query

wc_get_products() obtains all published products that belong to Bundles. If the user selects additional categories, tax_query uses relation = AND to require the bundle to belong to those categories as well.

```
$all_bundles = wc_get_products([
    'status'    => 'publish',
    'limit'     => -1,
    'orderby'   => 'title',
    'order'     => 'ASC',
    'tax_query' => $tax_query,
]);
```

19.3 Manual text and price filtering

Once the bundles have been retrieved, PHP iterates over the objects and keeps only those within the price range and whose name matches the search term, if one was entered.

```
if (
    $price >= $min_price
    && $price <= $max_price
    && ($q === '' || stripos($name, $q) !== false)
) {
    $filtered_products[] = $bundle;
}
```

19.4 Pagination of 12 bundles

After filtering, the current page, offset, and block to be displayed are calculated. The page contains up to 12 bundles.

```
$per_page = 12;
$offset = ($paged - 1) * $per_page;

$filtered_products = array_slice(
    $filtered_products,
    $offset,
    $per_page
);
```

If there are 30 matching results, the pages are distributed as 1–12, 13–24, and 25–30. Pagination links preserve the filter parameters so that changing pages does not clear the current selection.

19.5 Custom sidebar

Unlike the Shop, Bundles does not depend on HUSKY for its filter interface. The project directly builds a form with Search, Categories, Price, Apply Filters, and Reset. The overall price range is USD 35–50, and the final values are sent through hidden inputs.

The sidebar is a custom GET form. Search is sent in q, categories in cat[], and the price limits in hidden fields. Apply Filters requests the page again with those parameters; Reset opens /bundles/ again without filters.

```
<form method="get" action="<?php echo esc_url($bundles_url); ?>">
  <label for="q">Search</label>
  <input id="q" name="q" type="text"
    value="<?php echo esc_attr($q); ?>">

  <?php foreach ($cats_list as $term): ?>
    <label>
      <input type="checkbox" name="cat[]"
        value="<?php echo esc_attr($term->slug); ?>"
        <?php checked(in_array($term->slug, $cats, true)); ?>>
      <span><?php echo esc_html($term->name); ?></span>
    </label>
  <?php endforeach; ?>

  <input type="hidden" name="min_price" id="min_price_input"
    value="<?php echo esc_attr($min_price); ?>">
  <input type="hidden" name="max_price" id="max_price_input"
    value="<?php echo esc_attr($max_price); ?>">

  <button type="submit" class="button">Apply Filters</button>
  <button type="button" class="button"
    onclick="window.location.href= 'https://mcartworksstudio.com/bundles/';">
    Reset
  </button>
</form>
```

19.6 productos_del_bundle relationship

Each bundle retrieves the ACF field productos_del_bundle. The relationship returns the associated artworks; for each item, the WooCommerce product and its video_destacado are obtained.

```
$productos_in_bundle = get_field(
    'productos_del_bundle',
    $bundle_id
);

foreach ($productos_in_bundle as $prod) {
    $video_url = get_field(
        'video_destacado',
        $prod_id
    );
}
```

The product added to the cart is the bundle. The related artworks are used to visually represent its contents inside the card.

19.7 Lazy loading of internal videos

Bundle videos use a variant of the Shop's lazy-loading system. The URL remains in `<source data-src="...">`, and the `<video>` element is created with `preload="none"` and a faction-specific poster.

```
<video class="lazy-video"
    autoplay muted loop playsinline
    preload="none"
    poster="POSTER">
    <source data-src="VIDEO.mp4"
        type="video/mp4">
</video>
```

When `IntersectionObserver` detects the video, JavaScript copies `data-src` to `source.src`, calls `video.load()`, and waits for `loadeddata` before executing `play()`. The observer uses a 200 px margin and threshold 0.25.

19.8 MutationObserver for dynamic content

In addition to observing the videos present when the page loads, Bundles creates a `MutationObserver` on the body. If a new `.lazy-video` appears in the DOM, it is also registered with `IntersectionObserver`. This preserves the same behavior even when nodes are inserted later.

The `MutationObserver` does not load MP4 files directly. Its job is to register each new `.lazy-video` with the existing `IntersectionObserver`; that second observer decides when to assign `data-src` to `src` and begin the download.

```
const mutationObserver = new MutationObserver(mutations => {
    mutations.forEach(mutation => {
        mutation.addedNodes.forEach(node => {
            if (node.nodeType !== 1) return;

            if (node.matches('.lazy-video')) {
                observer.observe(node);
            } else {
                node.querySelectorAll('.lazy-video')
                    .forEach(v => observer.observe(v));
            }
        });
    });
});

mutationObserver.observe(document.body, {
    childList: true,
    subtree: true
});
```

19.9 Animations and the Add to Cart button

If results exist, one of five Bundle animations is selected at random. Each block also generates a WooCommerce button with the `add_to_cart_button` and `ajax_add_to_cart` classes, so the product participates in the same AJAX flow, global limit, and side cart as individual artworks.

```
class="button add_to_cart_button  
ajax_add_to_cart product_type_simple"
```

19.10 Guided example: Alex filters Bundles and adds one

Alex opens /bundles/ and enters text or changes the price range. The GET form requests the page again with those parameters. WordPress executes the `the_content` filter, and `mostrar_bundles_filtrados_rapido()` reads and sanitizes the values, obtains products in the Bundles category, and applies filtering and pagination.

When Alex clicks ADD TO CART on a bundle, the button retains the WooCommerce AJAX classes. The product added to the cart is the bundle's `WC_Product`; `productos_del_bundle` is used only to display the related artworks inside the card.

```
Alex applies filters in /bundles/  
↓  
GET q / cat[] / price  
↓  
mostrar_bundles_filtrados_rapido()  
↓  
wc_get_products()  
↓  
filtering + pagination  
↓  
PHP generates cards and lazy videos  
↓  
Alex clicks ADD TO CART  
↓  
WooCommerce adds the bundle product
```

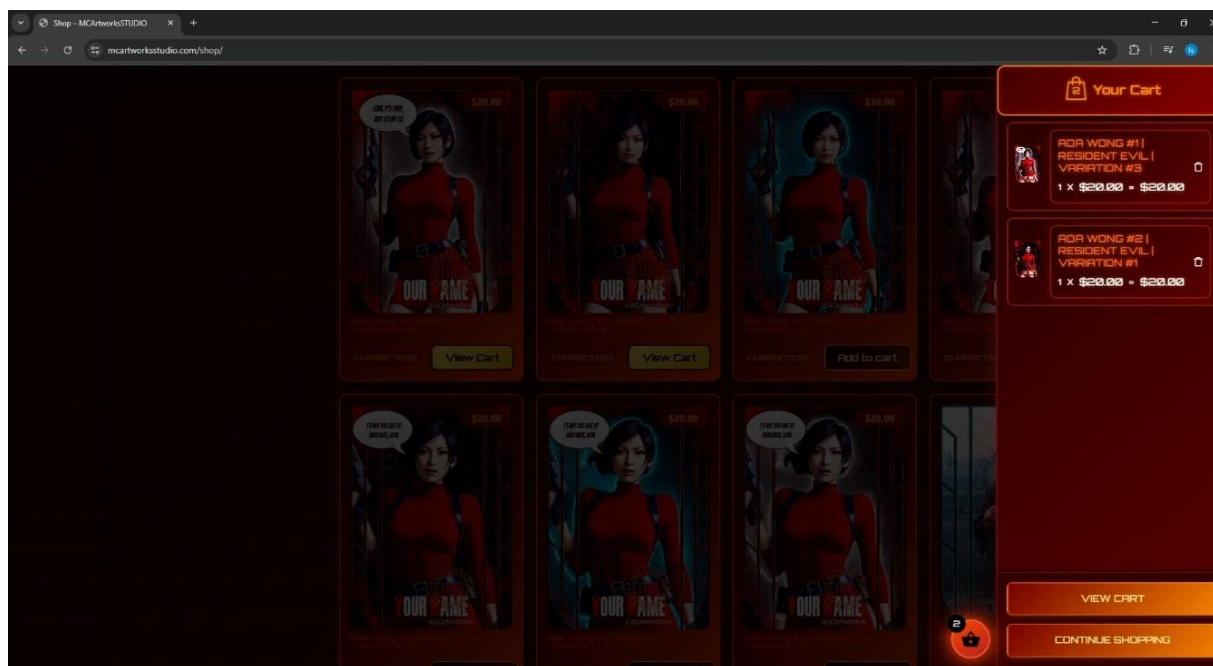
PART IV — CART, CHECKOUT, AND ORDERS

20. Side cart and item limit

Xootix Side Cart displays the cart as a dynamically updated side panel. WooCommerce retains the real cart and processes operations; Xootix represents that state without requiring a page reload; the child theme adds validation, button states, multimedia, loaders, and synchronization with Shop and Bundles.

Project location: wp-content/themes/blocksy-child/functions.php - this section crosses several blocks in the file. The PHP limit is registered on woocommerce_add_to_cart_validation around line 3391; the JavaScript check of the Xootix counter around 3510; removal synchronization uses admin-ajax.php with wp_ajax_obtener_id_desde_slug and obtener_id_desde_slug_callback() around lines 3693–3707; the thumbnail filter mc_cart_thumbnail_media() is registered around 6091; the removal loader is activated from wp_footer around 6145.

What this distribution means. Xootix does not replace the WooCommerce cart. Some pieces live in JavaScript because they react to the side interface; others live in PHP because they must validate or query the real cart. The diagrams in this section connect both sides of the same operation.



Xootix side cart open over the Shop.

20.1 What happens when an artwork is added

When the user clicks Add to Cart, the WooCommerce button contains `ajax_add_to_cart` and `data-product_id`. The theme performs its checks first; if the operation is allowed, WooCommerce adds the product through AJAX. When it completes successfully, the `added_to_cart` event is fired and Xootix updates the side panel.

```

User clicks ADD TO CART
↓
JavaScript checks the limit
↓
the button enters the loading state
↓
WooCommerce executes AJAX
↓
PHP validates the cart
↓
product added
↓
added_to_cart
↓
Xootix updates the side cart

```

20.2 Products sold one at a time

The `woocommerce_is_sold_individually` filter returns true for all products. This forces quantity 1 and removes the quantity selector: each artwork or bundle occupies an individual line in the cart.

```
add_filter(
    'woocommerce_is_sold_individually',
    'forzar_artworks_individuales',
    10,
    2
);

function forzar_artworks_individuales(...) {
    return true;
}
```

20.3 Limit of ten: first browser-side check

Before adding, `verificar_limite_productos()` reads the Xootix counter. If ten items already exist, it cancels the click and uses the project's custom alert.

```
const limite = 10;
const cantidadActual = parseInt(
    $('.xoo-wsc-items-count').text().trim() || '0'
);

if (cantidadActual >= limite) {
    e.preventDefault();
    mostrarAlertaPersonalizada(
        'Only up to 10 artworks are allowed per order.'
    );
}
```

20.4 Second check in PHP

Definitive validation occurs on the server through `woocommerce_add_to_cart_validation`. Therefore, the maximum does not depend solely on browser-side JavaScript.

```
$cantidad_actual =
    WC()->cart->get_cart_contents_count();

$nueva_cantidad_total =
    $cantidad_actual + $quantity;

if ($nueva_cantidad_total > 10) {
    return false;
}
```

Example: if there are nine products and the user attempts to add another, $9 + 1 = 10$ and the operation can continue. With ten items, any new addition is blocked.

20.5 Button loading state

When an AJAX Add to Cart begins, the button receives the disabled and loading classes, `aria-disabled=true`, and `pointer-events:none`. This prevents duplicate clicks while WooCommerce is still processing the same action. `added_to_cart` reverts those properties when the operation finishes.

```
$btn
    .addClass('disabled loading')
    .attr('aria-disabled', 'true')
    .css('pointer-events', 'none');
```

20.6 Add to Cart and View Cart

When the Shop is rendered, the `woocommerce_loop_add_to_cart_link` filter checks whether the product is already in WC()->cart. If it is, it hides Add to Cart and adds View Cart. After an AJAX add, another `added_to_cart` listener makes the same change immediately without waiting for a reload.

Product not in the cart
→ ADD TO CART

Product added
→ the original button is hidden
→ VIEW CART appears

20.7 Removal from Xootix and synchronization with the Shop

When the user removes a line through `.xoo-wsc-smr-del`, JavaScript obtains the slug from the product link and sends a request to `admin-ajax.php`. The `obtener_id_desde_slug` action finds the product with `get_page_by_path()` and returns its ID. With that value, only the corresponding Shop card is updated: View Cart disappears and Add to Cart becomes available again.

```
$.post(
    admin_url,
    {
        action: 'obtener_id_desde_slug',
        slug: slug
    }
);
```

```
Xootix: REMOVE
↓
product slug
↓
AJAX → WooCommerce ID
↓
the card is located
↓
VIEW CART is removed
ADD TO CART reappears
```

20.8 Multimedia inside the side cart

The `woocommerce_cart_item_thumbnail` filter replaces the standard thumbnail. Artworks can display `video_destacado`; bundles use a custom poster that changes with NOD/GDI. In this way, the side panel preserves the audiovisual identity of the Shop.

The same filter receives each cart line and decides what content to return. Bundles use a faction-dependent poster; artworks with `video_destacado` receive a `<video>`; if neither condition applies, WooCommerce keeps the original thumbnail.

```
add_filter(
    'woocommerce_cart_item_thumbnail',
    'mc_cart_thumbnail_media',
    10,
    3
);

function mc_cart_thumbnail_media($thumbnail, $cart_item, $cart_item_key) {
    $product = $cart_item['data'];
    $product_name = (string) $product->get_name();

    if (stripos($product_name, 'bundle') !== false) {
        $faction = $_COOKIE['faction'] ?? 'gdi';
        $uploads_dir = wp_upload_dir();
        $poster_url = $faction === 'gdi'
            ? trailingslashit($uploads_dir['baseurl']) . 'images/poster_bundle.png'
            : trailingslashit($uploads_dir['baseurl']) . 'images/poster_bundle_nod.jpg';
    }
}
```

```

    return '';
}

$video_url = get_field('video_destacado', $product->get_id());
if ($video_url) {
    return '<video class="xoo-wsc-thumb-video" autoplay muted loop playsinline src="'
        . esc_url($video_url) . '"></video>';
}

return $thumbnail;
}

```

20.9 Removal loader

The theme listens for clicks on Xootix removal controls and places a loading state over the row while the plugin processes the change. The `xoo_wsc_removed_item` and `xoo_wsc_cart_updated` events are used to clear the loader after the information has been updated.

The loader is attached to the specific row the user is removing. A fixed timeout is not the primary removal mechanism: Xootix events confirm that the item was removed or that the cart finished updating, and only then is the overlay removed.

```

document.addEventListener('click', function(e) {
    const btn = e.target.closest(
        '.xoo-wsc-smr-del, .xoo-wsc-remove, .xoo-wsc-smr .remove'
    );
    if (!btn) return;

    const productRow = btn.closest(
        '.xoo-wsc-smr, .xoo-wsc-product, .xoo-wsc-cart-item'
    );
    if (!productRow) return;

    const overlay = document.createElement('div');
    overlay.className = 'xoo-wsc-loading-overlay';
    overlay.innerHTML =
        '<div class="xoo-wsc-spinner"></div><div>Loading...</div>';
    productRow.appendChild(overlay);

    function limpiar() {
        if (overlay.parentNode) overlay.remove();
    }

    document.addEventListener('xoo_wsc_removed_item', limpiar, { once: true });
    document.addEventListener('xoo_wsc_cart_updated', limpiar, { once: true });
});

```

20.10 The side cart does not replace the Cart page

On pages where this behavior applies, a `MutationObserver` detects the Checkout button dynamically generated by Xootix and removes it. The expected flow goes through the Cart page, because Comic Balloon Text and Frame Color are configured there before proceeding to checkout.

Because Xootix can insert its buttons after the page loads, removing Checkout once is not enough. The observer continues watching the DOM and removes it again whenever the plugin generates it dynamically.

```

document.addEventListener('DOMContentLoaded', () => {
    const observer = new MutationObserver(() => {
        const checkoutBtn = document.querySelector(
            '.xoo-wsc-ft-btn-checkout'
        );
        if (checkoutBtn) checkoutBtn.remove();
    });

    observer.observe(document.body, {

```

```

    childList: true,
    subtree: true
  });
});

```

```

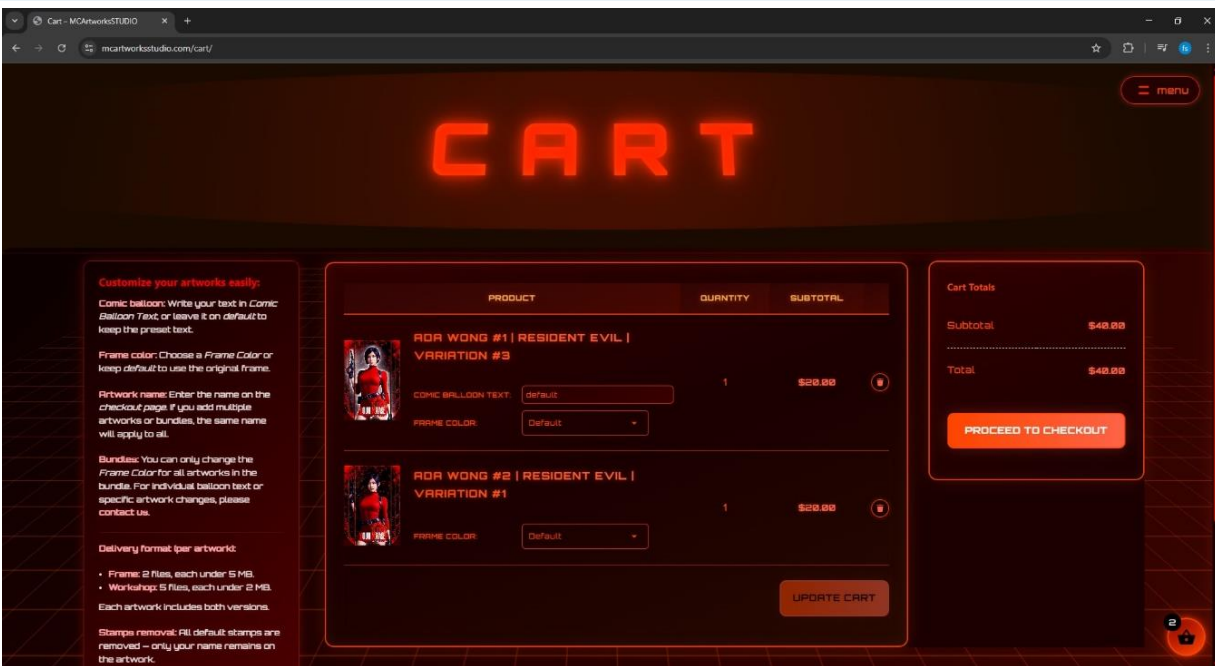
Shop / Bundles
↓
Xootix Side Cart
↓
VIEW CART
↓
Cart page
↓
customization
↓
Checkout

```

21. Cart page and customization

The Cart page is the stage where WooCommerce displays the cart lines and MC Artworks Studio adds custom information to each one. In addition to replacing the thumbnail, the theme removes some standard behaviors, adds Comic Balloon Text and Frame Color, and stores those values using the unique key for each cart item.

Project location: wp-content/themes/blocks-child/functions.php · mc_cart_thumbnail_media() begins around line 6099; mostrar_campos_personalizados_en_carrito() is connected to woocommerce_after_cart_item_name around 6248 and begins near 6261; mcart_guardar_campos_personalizados_en_carrito() is registered around 6383 and begins near 6391; restoration from session is registered around 6438 and forzar_valores_por_defecto_en_carrito() around 6471.



NOD Cart page with products, customization, and order summary.

21.1 Audiovisual thumbnail

woocommerce_cart_item_thumbnail is used to replace the conventional thumbnail. Cart reuses the same mc_cart_thumbnail_media() function documented in Section 20.8: bundles receive a NOD/GDI poster, artworks can display video_destacado, and if no custom branch applies, the original WooCommerce thumbnail is returned.

Cart and Xootix reuse the same woocommerce_cart_item_thumbnail filter. Therefore, the choice between bundle poster, ACF video, or standard thumbnail remains consistent in both views; the complete implementation is shown in Section 20.8.

```

add_filter(
    'woocommerce_cart_item_thumbnail',

```

```
'mc_cart_thumbnail_media',
10,
3
);
```

21.2 Fields below each product

WooCommerce executes `woocommerce_after_cart_item_name` after the name of each line. The theme connects `mostrar_campos_personalizados_en_carrito($cart_item, $cart_item_key)` there. `cart_item_key` is the key that distinguishes a specific line from the rest of the cart.

```
add_action(
    'woocommerce_after_cart_item_name',
    'mostrar_campos_personalizados_en_carrito',
    10,
    2
);
```

21.3 Comic Balloon Text

The field is displayed only when the ACF field `tiene_globo` is enabled for that product. The input includes `maxlength=50`, a placeholder, and a visible character counter.

```
$tiene_globo = get_field(
    'tiene_globo',
    $product_id
);

if ($tiene_globo) {
    // output input Comic Balloon Text
}
```

The input name includes `cart_item_key`, for example `comic_balloon_text[abc123]`. This associates the text with that specific line rather than with the entire cart.

21.4 Frame Color

Each line contains a `color_frame[cart_item_key]` select. Available options are default, blue, green, pink, red, violet, white, and yellow. They do not represent different products; they are customization data for the same item.

```
default · blue · green · pink · red · violet · white · yellow
```

21.5 Why cart_item_key is used

```
abc123 → Cyberpunk Portrait
    comic_balloon_text[abc123]
    color_frame[abc123]
```

```
def456 → Fantasy Portrait
    comic_balloon_text[def456]
    color_frame[def456]
```

The key allows two products in the same cart to maintain independent customizations.

21.6 What happens when the cart is updated

The `woocommerce_update_cart_action_cart_updated` hook executes `mcart_guardar_campos_personalizados_en_carrito()`. The function iterates over `WC()->cart->get_cart()`, looks for the submitted values for each `cart_item_key`, sanitizes them, and stores them inside `cart_contents`.

```
$texto = sanitize_text_field(
    $_POST['comic_balloon_text'][$cart_item_key]
);

$color = sanitize_text_field(
    $_POST['color_frame'][$cart_item_key]
);
```

WC()->cart->set_session() is then executed so WooCommerce updates its persistent representation of the cart.

21.7 Restoration from the session

When WooCommerce reconstructs the cart after a reload, the `woocommerce_get_cart_item_from_session` filter restores `comic_balloon_text` and `color_frame` from the stored values. This keeps the selection visible after refreshing or reopening the page.

```
if (isset($values['color_frame'])) {
    $cart_item['color_frame'] =
        $values['color_frame'];
}
```

21.8 Default values

`woocommerce_before_calculate_totals` executes `forzar_valores_por_defecto_en_carrito()`. If an item does not yet have one of the fields, default is assigned and the session is saved again. In this way, the system works with a defined value even if the user never changed the controls.

```
comic_balloon_text = 'default'
color_frame        = 'default'
```

21.9 Cart interface adjustments

The product name is no longer a link through `woocommerce_cart_item_name`. Coupons are disabled with `woocommerce_coupons_enabled`, and several standard WooCommerce notices related to adding, removing, or restoring products are removed, keeping the interface under tighter theme control.

These filters modify the interface without changing the WC_Product or the actual cart contents: the product-name text is preserved but its link is removed, coupons are disabled, and some standard notices are no longer output.

```
add_filter(
    'woocommerce_cart_item_name',
    'mca_cart_item_name_no_link',
    10,
    3
);

function mca_cart_item_name_no_link($product_name_html) {
    if (false === strpos($product_name_html, '<a')) {
        return $product_name_html;
    }

    $without_link = preg_replace(
        '#<a[^\>]*>(.*?)</a>#is',
        '$1',
        $product_name_html
    );

    return '<span class="product-title">' . trim($without_link) . '</span>';
}

add_filter('woocommerce_coupons_enabled', '__return_false');
add_filter('wc_add_to_cart_message_html', '__return_empty_string');
add_filter('woocommerce_cart_undo_item_notice', '__return_empty_string');
```

21.10 Complete example: from the Shop to customization

```

Cyberpunk Portrait · USD 18
↓
ADD TO CART
↓
WooCommerce AJAX
↓
Xootix displays the artwork
↓
VIEW CART
↓
Cart detects tiene_globo = true
↓
Comic Balloon Text = "Welcome back"
Frame Color = Blue
↓
UPDATE CART
↓
PHP sanitizes the values
↓
cart_item_key preserves the association
↓
WooCommerce saves the session
↓
Cart reload
↓
the values remain selected
↓
Checkout

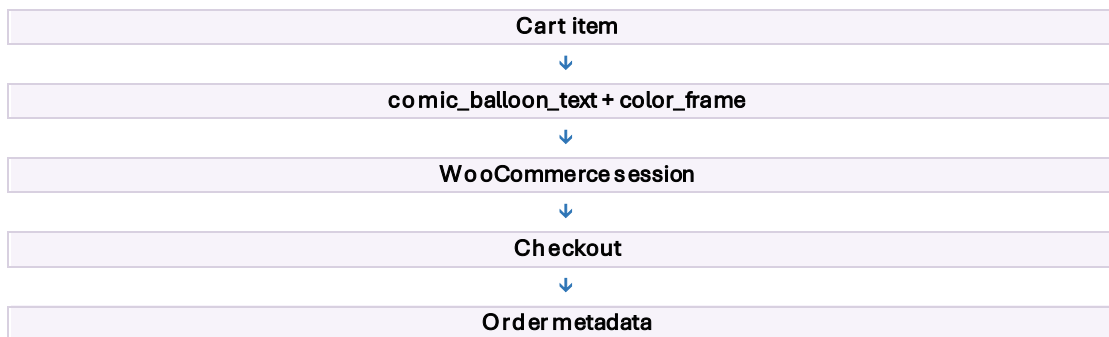
```

22. Session and data persistence

The following section explains in greater detail how WooCommerce preserves this data between requests and how it later moves from the cart session into the order.

Project location: `wp-content/themes/blocksy-child/functions.php` · this section examines in greater detail three callbacks already used by Cart: `woocommerce_update_cart_action_cart_updated` → `mcart_guardar_campos_personalizados_en_carrito()`; `woocommerce_get_cart_item_from_session` → `mcart_restaurar_campos_personalizados_desde_sesion()`; and `woocommerce_before_calculate_totals` → `forzar_valores_por_defecto_en_carrito()`. They are located approximately between lines 6381 and 6495.

A session is temporary information that the server retains while the visitor continues the purchase. WooCommerce uses its own session and the `wp_woocommerce_sessions` table to preserve the cart across pages. The theme adds `comic_balloon_text` and `color_frame` to the item data and restores them when WooCommerce reconstructs the cart.



Sanitization

Before storing text received from the browser, the code uses WordPress cleaning functions. “Sanitizing” means converting the input to a safe and expected form for that field, removing content that does not belong in ordinary text.

Length control

The Comic Balloon Text interface limits visible input to 50 characters. On the server, sanitization also retains a maximum of 160 characters. In normal use of the form, the limit encountered by the buyer is the interface limit.

22.1 Saving when Update Cart is clicked

When the user updates Cart, WooCommerce fires `woocommerce_update_cart_action_cart_updated`. The theme function iterates over each line using its `cart_item_key`, cleans the submitted values, and stores them inside `cart_contents`.

```
add_action(
    'woocommerce_update_cart_action_cart_updated',
    'mcart_guardar_campos_personalizados_en_carrito'
);

$texto = sanitize_text_field($_POST['comic_balloon_text'][$cart_item_key]);
$texto = mb_substr($texto, 0, 160);
WC()->cart->cart_contents[$cart_item_key]['comic_balloon_text'] = $texto;
```

Frame Color follows the same path. At the end, `WC()->cart->set_session()` is called so that WooCommerce writes the updated cart state to the session.

22.2 Restoration when WooCommerce reconstructs the cart

When changing pages or rebuilding the cart from the session, `woocommerce_get_cart_item_from_session` copies `comic_balloon_text` and `color_frame` back into the active item. This means the selected values do not depend on the Cart page remaining open.

```
add_filter(
    'woocommerce_get_cart_item_from_session',
    'mcart_restaurar_campos_personalizados_desde_sesion',
    10, 3
);

if (isset($values['color_frame'])) {
    $cart_item['color_frame'] = $values['color_frame'];
}
```

22.3 Default values before calculating totals

There is a third safety layer. Before calculating totals, the theme checks each line and sets default when Comic Balloon Text or Frame Color does not yet have a value. This allows the order to maintain a consistent structure even if the user proceeds without clicking Update Cart.

```
add_action('woocommerce_before_calculate_totals', 'forzar_valores_por_defecto_en_carrito', 5);

if (!isset($cart_item['color_frame']) || $cart_item['color_frame'] === '') {
    $cart->cart_contents[$cart_item_key]['color_frame'] = 'default';
}
```

22.4 Complete persistence example

```
Alex chooses Frame Color = Blue
↓
POST color_frame[cart_item_key] = blue
↓
sanitize_text_field()
↓
cart_contents receives blue
↓
WC()->cart->set_session()
↓
Alex reloads or proceeds to Checkout
```

```

↓
woocommerce_get_cart_item_from_session
↓
Frame Color is Blue again

```

23. Checkout: final purchase form and validations

Checkout is the final stage of the purchase, where the visitor completes the required information and confirms the transaction. WooCommerce can normally request an address, city, country, phone number, and other fields. MC Artworks Studio replaces that section with a much shorter form because the flow is designed for digital content.

Project location: wp-content/themes/blocks-child/functions.php · `simplificar_checkout_nombre_y_correo()` is registered on `woocommerce_checkout_fields` around line 7202; `limpiar_campos_checkout()` around 7233; the Artworks Name field is connected to `woocommerce_before_checkout_billing_form` around 7953 and its function begins near 7961; the three server validations are registered on `woocommerce_checkout_process` around lines 8179, 8188, and 8195; browser `maxlength` attributes are output from `wp_footer` around 8259; Artworks Name is copied to the order on `woocommerce_checkout_create_order` around 8297.

Important execution order. JavaScript visual limits help while Alex types, but submission still passes through the PHP validations in `woocommerce_checkout_process`. Only if WooCommerce accepts the form does WC_Order construction continue and the hooks that copy data into the order fire.

NOD Checkout with streamlined form and order summary.

Field	Rule in the code	Purpose
Artworks Name	Required; maximum 32 characters.	Name that will be associated with the artworks in the order.
First Name	Required; maximum 50 characters.	Buyer name also used in the email.
Email	Required; maximum 100 characters; valid format and verifiable domain.	Destination for the email containing payment instructions.

Artworks Name

This is project-specific data, not a standard WooCommerce field. If Alex enters “Raven”, the order stores Artworks Name = Raven and that value later appears again in administration, the log, and the emails.

DNS domain validation

DNS makes it possible to locate services associated with a domain name on the Internet. The code does more than check whether the email has a valid format: it also attempts to verify that the domain entered after @ has MX or A DNS records.

Example

alex@gmail.com uses a resolvable domain. An address with an invented domain may fail the check. This does not prove that the “alex” mailbox exists; it verifies the domain, not the individual mailbox.

23.1 Reducing the standard fields

Source code: wp-content/themes/blocksy-child/functions.php → woocommerce_checkout_fields filter registered around line 7202 → simplificar_checkout_nombre_y_correo() from line 7212. WooCommerce provides \$fields; the callback returns the reduced billing structure before Checkout is rendered.

woocommerce_checkout_fields replaces the billing section with two fields: First Name and Email. The site sells digital content, so it does not request address, city, or phone number in this flow.

```
$fields['billing'] = [
    'billing_first_name' => [
        'label' => 'First Name', 'required' => true
    ],
    'billing_email' => [
        'label' => 'Email', 'required' => true
    ],
];
```

23.2 Empty fields each time Checkout opens

Source code: functions.php → woocommerce_checkout_get_value filter around line 7233 → limpiar_campos_checkout(\$input, \$key) from line 7244. WooCommerce consults this filter when deciding which initial value to display in each field.

The woocommerce_checkout_get_value filter returns an empty string for First Name and Email. This prevents the form from automatically reusing earlier session or browser data.

```
if (in_array($key, ['billing_first_name', 'billing_email'])) {
    return '';
}
```

23.3 Artworks Name as custom data

Source code: functions.php → woocommerce_before_checkout_billing_form hook around line 7953 → wpo_mover_titulo_facturacion_y_agregar_artwork_field(\$checkout) from line 7961. The callback outputs the artwork_name field before the billing data.

Artworks Name is not part of the standard WooCommerce checkout. The theme inserts it as a custom field and later stores it as general WC_Order metadata. Unlike Frame Color, it does not belong to a specific line item; it describes the entire order.

The field is inserted before the billing form through a WooCommerce hook. woocommerce_form_field() generates the input with the name artwork_name; later, that same name is used to validate \$_POST and store the order-level metadata.

```
add_action(
    'woocommerce_before_checkout_billing_form',
    'wpo_mover_titulo_facturacion_y_agregar_artwork_field'
);

function wpo_mover_titulo_facturacion_y_agregar_artwork_field($checkout) {
    echo '<div id="artwork_name_field">';
    echo '<h3>Name for your artworks</h3>';

    woocommerce_form_field('artwork_name', [
        'type' => 'text',
        'class' => ['artwork-name form-row-wide'],
        'label' => 'Enter the name that will appear on your artworks',
        'required' => true
    ], $checkout->get_value('artwork_name'));

    echo '</div>';
}
```

23.4 Customization summary

Source code: functions.php → woocommerce_get_item_data filter around line 7676 → mostrar_campos_personalizados_en_resumen_pedido(\$item_data, \$cart_item) from approximately line 7686. It receives the existing cart item and returns the additional rows WooCommerce will display in the summary.

woocommerce_get_item_data adds Frame Color to the summary and displays Comic Balloon Text only when the product has tiene_globo and is not detected as a bundle. This lets the buyer review the customization before confirming.

```
Cart item
├─ Frame Color → always displayed
└─ Comic Balloon Text
   └─ tiene_globo and not a bundle → displayed
      └─ otherwise → hidden
```

23.5 Order Notes and standard elements that are removed

Source code: functions.php → woocommerce_checkout_show_terms (approx. line 7606) and woocommerce_enable_order_notes_field (approx. line 7993) filters. They are independent filters: one hides terms and the other disables Order Notes.

The standard Order Notes field is disabled, and some checkout terms and privacy text are also hidden. The intention is to keep the screen brief and focused on the information needed for a digital order and the manual payment method.

```
add_filter('woocommerce_enable_order_notes_field', '__return_false');
add_filter('woocommerce_checkout_show_terms', '__return_false');
```

23.6 Server-side validation

Source code: functions.php → three anonymous callbacks registered on woocommerce_checkout_process around lines 8179, 8188, and 8195. They do not have their own names: WooCommerce executes them during submission before creating the WC_Order; each wc_add_notice(..., 'error') can stop the process.

- Artworks Name: required and maximum 32 characters.

Browser-visible validation is only an aid. These checks run again in PHP during woocommerce_checkout_process; if any rule fails, wc_add_notice() adds the error and WooCommerce stops the normal checkout process.

```
add_action('woocommerce_checkout_process', function() {
    if (empty($_POST['artwork_name'])) {
        wc_add_notice('Please enter a name for your artworks.', 'error');
    } elseif (strlen($_POST['artwork_name']) > 32) {
        wc_add_notice(
            'The name for your artworks cannot exceed 32 characters.',
            'error'
        );
    }
});

add_action('woocommerce_checkout_process', function() {
    if (!empty($_POST['billing_first_name'])
        && strlen($_POST['billing_first_name']) > 50) {
        wc_add_notice(
            'Billing first name cannot exceed 50 characters.',
            'error'
        );
    }
});

add_action('woocommerce_checkout_process', function() {
    $email = sanitize_text_field($_POST['billing_email'] ?? '');

    if (strlen($email) > 100) {
        wc_add_notice('Email cannot exceed 100 characters.', 'error');
        return;
    }

    if (empty($email) || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
```

```

    wc_add_notice('Please enter a valid email address.', 'error');
    return;
}

$domain = substr(strrchr($email, '@'), 1);
if ($domain === false || strpos($domain, '.') === false) {
    wc_add_notice('The email domain is not valid.', 'error');
    return;
}
});

```

Domain checking continues with real DNS records. The code first attempts MX, then MX/A through `checkdnsrr()`, and finally uses `gethostbyname()` as a last resort before considering the domain invalid.

```

$has_dns = false;

if (function_exists('getmxrr')) {
    $mxhosts = [];
    if (@getmxrr($domain, $mxhosts) && !empty($mxhosts)) {
        $has_dns = true;
    }
}

if (!$has_dns && function_exists('checkdnsrr')) {
    $has_dns = @checkdnsrr($domain, 'MX')
        || @checkdnsrr($domain, 'A');
}

if (!$has_dns) {
    $resolved = @gethostbyname($domain);
    $has_dns = ($resolved !== $domain && $resolved !== false);
}

if (!$has_dns) {
    wc_add_notice(
        'The email domain does not appear to exist (check that it is correctly typed).',
        'error'
    );
}

```

- First Name: maximum 50 characters.
- Email: maximum 100 characters and valid format.
- Email domain: must contain a dot and resolve through MX or A DNS records, with a final host-resolution attempt.

These validations run through `woocommerce_checkout_process`. If a condition fails, `wc_add_notice()` adds an error and WooCommerce does not continue with normal order creation.

23.7 Browser-visible validation

Source code: `functions.php` → JavaScript output from `wp_footer` around line 8259 → `DOMContentLoaded` callback that applies `maxlength` to `artwork_name`, `billing_first_name`, and `billing_email`. It is a browser-side aid; final authority remains the PHP validation described in Section 23.6.

JavaScript adds `maxlength` to all three fields and sets `billing_email` to `type="email"`. This improves the user experience, but it does not replace server-side validation: a user can modify HTML in the browser, so PHP checks the limits again.

These limits help the user while typing, but they do not replace the PHP check described in the previous subsection.

```

document.addEventListener('DOMContentLoaded', function() {
    const art = document.querySelector('#artwork_name');
    if (art) art.setAttribute('maxlength', '32');

    const name = document.querySelector('#billing_first_name');
    if (name) name.setAttribute('maxlength', '50');

    const mail = document.querySelector('#billing_email');
    if (mail) mail.setAttribute('maxlength', '100');
});

```

23.8 What happens on confirmation

Alex clicks Pay with PayPal. The browser submits the form; WooCommerce runs the validations registered on `woocommerce_checkout_process`. If there is no error-type `wc_add_notice()`, WooCommerce continues and builds the order. During that construction, Artworks Name and each line-item customization are copied; only afterward does the selected gateway receive the `order_id`.

```
Pay with PayPal
↓
Checkout submit
↓
WooCommerce validates fields
↓
creates WC_Order
↓
manual_pay receives the order_id
```

24. Manual payment method using PayPal

WooCommerce uses the term gateway for the component that determines what happens when the buyer confirms a payment method. The theme registers a custom gateway with the identifier `manual_pay` and the title “Pay with PayPal (manual)”.

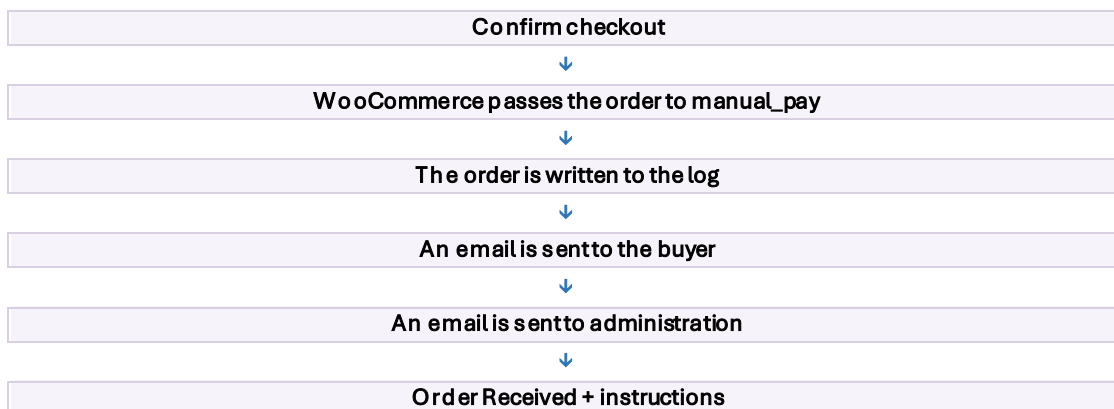
Project location: `wp-content/themes/blocksy-child/functions.php` · `add_action('init', 'registrar_metodo_pago_manual')` appears around line 7262; `registrar_metodo_pago_manual()` begins near 7271 and declares the `WC_Gateway_Manual_Pay` class. Inside that class, `__construct()` configures the gateway, `solo_en_checkout()` controls availability, `get_log_path()/log_manual_pay()` manage logs, and `process_payment($order_id)` begins around line 7322. The `woocommerce_payment_gateways` filter that adds the class appears at the end of the same block, near line 7593.

This distinction avoids a common misunderstanding: `init` does not process a payment. `init` causes WordPress to execute `registrar_metodo_pago_manual()`, and that function declares the gateway class if WooCommerce is available. Payment processing happens much later, when WooCommerce selects an instance of that class during Checkout.

Fundamental clarification

This implementation does not connect to the PayPal API and does not charge automatically. It creates the order, logs the operation, sends instructions by email, and takes the buyer to the Order Received page. Payment is completed manually using the details provided in the email.

```
$this->id = 'manual_pay';
$this->title = 'Pay with PayPal (manual)';
...
return ['result' => 'success', 'redirect' => $this->get_return_url($order)];
```



24.1 Registering the gateway

Source code: `functions.php` → `init` hook → `registrar_metodo_pago_manual()` → declaration of `WC_Gateway_Manual_Pay`.

The gateway is registered during `init`. The code first checks that `WC_Payment_Gateway` exists; if WooCommerce is not available, it does not attempt to declare the custom class.

```
add_action('init', 'registrar_metodo_pago_manual');

if (!class_exists('WC_Payment_Gateway')) return;

class WC_Gateway_Manual_Pay extends WC_Payment_Gateway {
    // ...
}
```

24.2 Configuring WC_Gateway_Manual_Pay

Source code: functions.php → WC_Gateway_Manual_Pay::__construct(). Here \$this represents the specific gateway instance WooCommerce is configuring.

The constructor defines the gateway identity. manual_pay is the internal identifier, and “Pay with PayPal (manual)” is the title representing the method within the purchase flow.

```
$this->id          = 'manual_pay';
$this->title       = 'Pay with PayPal (manual)';
$this->enabled     = 'yes';
$this->supports    = [ 'products' ];
```

24.3 Availability only during Checkout

Source code: functions.php → WC_Gateway_Manual_Pay::solo_en_checkout(). The constructor registers this method on woocommerce_available_payment_gateways.

solo_en_checkout() removes manual_pay from the gateway list when the request is not for checkout or a checkout AJAX update. This prevents the method from being presented in contexts where it cannot be used.

The DOING AJAX exception is necessary because WooCommerce updates parts of checkout through AJAX requests. During those requests, the gateway must remain available even though is_checkout() may not represent a conventional page load.

```
public function solo_en_checkout($gateways) {
    if (is_admin()) return $gateways;

    if (!is_checkout()
        && !(defined('DOING AJAX') && DOING AJAX)) {
        unset($gateways[$this->id]);
    }

    return $gateways;
}
```

24.4 How it is added to WooCommerce

Source code: functions.php → registrar_metodo_pago_manual() → woocommerce_payment_gateways filter, located after the class declaration.

```
add_filter('woocommerce_payment_gateways', function($methods) {
    $methods[] = 'WC_Gateway_Manual_Pay';
    return $methods;
});
```

WooCommerce receives the class name and adds it to its list of available payment methods.

24.5 Pay with PayPal button

Source code: functions.php → woocommerce_review_order_before_submit hook around line 7628 → boton_manual_estilo_paypal() around line 7639. The button submits Checkout, and the hidden input sets payment_method=manual_pay.

Before the standard submit button, the theme inserts a hidden field that forces payment_method = manual_pay and a visible button. That button submits the checkout form; it does not open PayPal or perform an external charge.

```
<input type="hidden" name="payment_method" value="manual_pay">
<button type="submit" class="manual-pay-btn">Pay with PayPal</button>
```

24.6 What process_payment() does

Source code: functions.php → WC_Gateway_Manual_Pay class → process_payment(\$order_id), approximately lines 7322–7590. WooCommerce calls this method when manual_pay is the selected gateway and the WC_Order already exists.

```
process_payment(order_id)
↓
wc_get_order(order_id)
↓
builds and saves the log
↓
builds the buyer email
↓
wp_mail(buyer)
↓
builds the administrator email
↓
wp_mail(admin)
↓
get_return_url(order)
↓
Order Received
```

24.6.1 Guided example: how Alex reaches the gateway

Alex completes Checkout and clicks Pay with PayPal. The hidden input leaves payment_method=manual_pay. WooCommerce validates the form, builds the WC_Order, and retains its ID. With the gateway already selected, WooCommerce invokes WC_Gateway_Manual_Pay::process_payment(\$order_id).

```
Alex clicks Pay with PayPal
↓
Checkout submit
↓
woocommerce_checkout_process validates
↓
WooCommerce builds WC_Order
↓
payment_method = manual_pay
↓
WooCommerce invokes WC_Gateway_Manual_Pay::process_payment(order_id)
↓
wc_get_order(order_id) returns the WC_Order object
```

24.7 What this gateway does NOT do

Clarification

manual_pay does not use the PayPal API, does not check the balance, does not confirm payment, and does not execute payment_complete(). The order remains pending until payment is verified manually. The name “Pay with PayPal” describes the instructions the buyer will receive, not an automatic PayPal integration.

25. Order creation

This section explains the exact moment when temporary cart data and Checkout form data become part of the WC_Order. Timing is the key point: these hooks run while the order is being built.

WC_Gateway_Manual_Pay::process_payment(\$order_id) runs afterward, once WooCommerce already has an order that can be identified by its ID.

Project location: wp-content/themes/blocksy-child/functions.php. The anonymous woocommerce_checkout_create_order_line_item callback is on lines 7651–7659 and copies Comic Balloon Text and Frame Color to each WC_Order_Item. The anonymous woocommerce_checkout_create_order callback is on lines 8297–8301 and stores Artworks Name as WC_Order metadata. Both belong to the order-creation process; they are not methods of WC_Gateway_Manual_Pay.

Do not confuse registration with execution: add_action() associates the callback with a hook. The code inside it does not run when WordPress loads functions.php; it runs later, when WooCommerce reaches that hook during Checkout.

25.1 What exists immediately before the order is created

Before these hooks run, WooCommerce keeps each product and the custom data added by MC Artworks Studio in the cart. For a specific line, \$values represents the cart-item information that WooCommerce is transforming into a permanent order line. In parallel, Artworks Name arrives from the final Checkout form through \$_POST['artwork_name'].

```
CART / CHECKOUT
↓
cart item: product_id + color_frame + comic_balloon_text
form: artwork_name
↓
WooCommerce begins building the WC_Order and its WC_Order_Item objects
```

25.2 Line-item customizations: complete callback

Actual source code: functions.php → anonymous callback registered on woocommerce_checkout_create_order_line_item → lines 7651–7659. WooCommerce calls this callback once for each line it converts from the cart into the order.

```
add_action('woocommerce_checkout_create_order_line_item', function($item, $cart_item_key, $values, $order) {
    if (isset($values['comic_balloon_text']) && !empty($values['comic_balloon_text'])) {
        $item->add_meta_data('Comic Balloon Text', $values['comic_balloon_text']);
    }

    if (isset($values['color_frame']) && !empty($values['color_frame'])) {
        $item->add_meta_data('Frame Color', ucfirst($values['color_frame']));
    }
}, 10, 4);
```

The callback first checks whether each value exists and is not empty. If Comic Balloon Text is available, it adds it to the WC_Order_Item with the visible label “Comic Balloon Text”. If color_frame is available, it adds it with the label “Frame Color” and applies ucfirst() so that the value begins with a capital letter.

The conceptual change is important: color_frame and comic_balloon_text stop being only temporary cart-item data and become metadata for a specific order line. This allows two products in the same WC_Order to preserve different customizations.

25.3 What the four callback parameters mean

Parameter	What it represents in this hook
\$item	The WC_Order_Item that WooCommerce is building for that line. This is where the permanent metadata is added.
\$cart_item_key	The internal key WooCommerce uses to identify that line inside the cart. The callback receives it, although this code does not use it.
\$values	The data from the source cart item. MC Artworks reads comic_balloon_text and color_frame here.
\$order	The WC_Order being built. WooCommerce passes it to the callback, although this fragment does not use it directly.

The “10, 4” at the end of add_action() means priority 10 and four accepted arguments. Therefore, WordPress/WooCommerce passes \$item, \$cart_item_key, \$values, and \$order to the anonymous callback.

25.4 Artworks Name: order-level data

Actual source code: functions.php → anonymous callback registered on woocommerce_checkout_create_order → lines 8297–8301. Unlike Frame Color and Comic Balloon Text, Artworks Name belongs to the complete order rather than to a specific line.

```
add_action('woocommerce_checkout_create_order', function($order, $data) {
    if (isset($_POST['artwork_name']) && $_POST['artwork_name'] !== '') {
        $order->update_meta_data('Artworks Name', sanitize_text_field($_POST['artwork_name'])); // Translated
    }
}, 10, 2);
```

The callback receives the WC_Order being built as \$order. If artwork_name arrived from Checkout and is not empty, sanitize_text_field() cleans the received text and update_meta_data() adds it to the order with the key “Artworks Name”. The callback’s second parameter, \$data, is provided by WooCommerce but is not used in this fragment.

The difference in scope explains why the project uses two different APIs: \$item->add_meta_data() for product-level customizations and \$order->update_meta_data() for data describing the complete order.

25.5 What is inside the WC_Order before the gateway runs

```
WC_Order #1530
├── First Name: Alex
├── Email: alex@example.com
├── Artworks Name: Raven
├── WC_Order_Item: Cyberpunk Portrait
│   ├── Frame Color: Blue
│   └── Comic Balloon Text: Welcome back
```

In the teaching example, Blue and “Welcome back” belong only to Cyberpunk Portrait. Raven, by contrast, belongs to the entire order. When process_payment() receives the order identifier, those data are already available through the WC_Order and WC_Order_Item APIs; the gateway does not need to rebuild them from Cart.

25.6 Guided example: Alex before and after

Before the order is created, Alex’s line still lives in the cart and contains color_frame = blue and comic_balloon_text = Welcome back. During construction, WooCommerce creates the WC_Order_Item and fires woocommerce_checkout_create_order_line_item. The callback copies those values into the visible keys “Frame Color” and “Comic Balloon Text”. During the same Checkout process, woocommerce_checkout_create_order allows Raven to be stored as “Artworks Name” on the WC_Order.

```
BEFORE
cart item → color_frame = blue · comic_balloon_text = Welcome back
Checkout → artwork_name = Raven
↓
DURING CREATION
WC_Order_Item receives Frame Color / Comic Balloon Text
WC_Order receives Artworks Name
↓
AFTER
WC_Order #1530 already contains the data manual_pay will read
```

25.7 Passing the order to the gateway

Actual source code that receives the result of that creation: functions.php → WC_Gateway_Manual_Pay::process_payment(\$order_id) → lines 7322–7323.

```
public function process_payment($order_id) {
    $order = wc_get_order($order_id);
```

WooCommerce passes the order ID to the selected gateway. wc_get_order(\$order_id) converts that identifier into a WC_Order object. From that point onward, the code works with the already-created order: first it generates the log and then builds the two emails.

25.8 Pending status and HPOS

manual_pay does not execute payment_complete() inside process_payment(). Therefore, the theme code does not mark the order as paid. In the documented installation, manual orders remain in wc-pending status while they await manual payment verification. With HPOS active, the theme continues using WC_Order and WC_Order_Item and does not need to write SQL directly against the physical tables.

26. Logs: recording orders and emails

A log is a chronological record used to reconstruct what happened during an operation. In MC Artworks Studio, process_payment() first generates a block with the order data and then appends the result of the buyer email and the administration email to the same file. This section separates three steps that were previously mixed together: building the text, deciding which file to write to, and writing it.

Project location: wp-content/themes/blocksy-child/functions.php → WC_Gateway_Manual_Pay class. process_payment() begins on line 7322; the block that builds \$log spans approximately lines 7329–7377. get_log_path() is on 7298–7308 and log_manual_pay(\$message) on 7310–7320.

26.1 Entering process_payment(): from order_id to WC_Order

```
public function process_payment($order_id) {
    $order = wc_get_order($order_id);
```

order_id is the identifier WooCommerce has just passed to the gateway. wc_get_order() obtains the corresponding WC_Order object and stores it in \$order. From that object, the code reads the visible order number, billing data, Artworks Name, product lines, and total.

```
$order_id = 1530
↓ wc_get_order(1530)
$order = WC_Order #1530
↓
get_order_number() · get_billing_first_name() · get_items() · get_total()
```

26.2 Building the log header and general data

Actual source code: functions.php → WC_Gateway_Manual_Pay::process_payment() → lines 7329–7344.

```
$log = "=====\\n";
$log .= "🕒 " . date('d-m-Y H:i:s', current_time('timestamp')) . "\\n";
$log .= "=====\\n";
$log .= "🛒 NEW MANUAL PAYPAL ORDER\\n";
$log .= "-----\\n";
$log .= "Order #: " . $order->get_order_number() . "\\n";
$log .= "Customer: " . $order->get_billing_first_name() . " " . $order->get_billing_last_name() . "\\n";
$log .= "Email: " . $order->get_billing_email() . "\\n";

// Add Artworks Name if it exists
$artwork_name = $order->get_meta('Artworks Name');
if ($artwork_name) {
    $log .= "Name for the artworks: " . $artwork_name . "\\n";
}

$log .= "-----\\n";
```

The \$log variable begins as a text string. Each .= operator appends content without deleting what was already there. The date is formatted using the time WordPress provides through current_time('timestamp'). The order number, billing name, email, and, if present, Artworks Name are then added.

In Alex's case, \$order->get_meta('Artworks Name') returns Raven. Because the value is not empty, the if (\$artwork_name) condition adds "Name for the artworks: Raven" to the record.

26.3 Iterating over products and customizations

Actual source code: functions.php → process_payment() → first foreach (\$order->get_items()) → lines 7346–7372.

```
foreach ($order->get_items() as $item) {
    $product_name = $item->get_name();
    $total = number_format($item->get_total(), 2);
    $log .= "• {$product_name} - \\${$total}\\n";

    $product_id = $item->get_product_id();
    $tiene_globo = get_field('tiene_globo', $product_id); // show Comic Balloon Text only
    when tiene_globo is enabled

    $meta_lines = [];
    foreach ($item->get_meta_data() as $meta) {
        $key = strtolower($meta->key);
        $value = $meta->value;

        if ($key === 'frame color') {
            $meta_lines[] = "• Frame Color: {$value}";
        }
        if ($tiene_globo && $key === 'comic balloon text') {
```

```

        $meta_lines[] = "• Comic Balloon Text: {$value}";
    }
}

if (!empty($meta_lines)) {
    foreach ($meta_lines as $line) {
        $log .= "    {$line}\n";
    }
}
}

```

`$order->get_items()` returns the `WC_Order` lines. On each iteration, `$item` represents a `WC_Order_Item`. `get_name()` obtains the product name and `get_total()` the amount for that line. `get_product_id()` then allows ACF to be queried with `get_field('tiene_globo', $product_id)`.

The inner loop iterates over the metadata already persisted in Section 25. Frame Color can always be added to the log. Comic Balloon Text is added only when `tiene_globo` is enabled for that product. `$meta_lines` acts as a temporary list: it first gathers the lines that should be shown and then appends them to `$log` with indentation.

```

WC Order Item: Cyberpunk Portrait
↓ get_name() / get_total()
Cyberpunk Portrait - $18.00
↓ get_meta_data()
Frame Color: Blue
Comic Balloon Text: Welcome back
↓
those lines are appended to $log

```

26.4 Total and first write to the log

Actual source code: `functions.php` → end of the `process_payment()` log block → lines 7374–7377.

```

$log .= "-----\n";
$log .= "Total: $" . number_format($order->get_total(), 2) . "\n";

$this->log_manual_pay($log);

```

After all products have been iterated, `get_total()` obtains the `WC_Order` total. At that point, `$log` already contains the complete order record and `$this->log_manual_pay($log)` passes that string to the method responsible for writing it.

`$this` represents the specific `WC_Gateway_Manual_Pay` instance processing the order. `log_manual_pay()` is not a global function; it is a private method of that same class.

26.5 How the daily file is determined

Actual source code: `functions.php` → `WC_Gateway_Manual_Pay::get_log_path()` → lines 7298–7308.

```

private function get_log_path() {
    // Existing directory where logs are stored
    $uploads_dir = WP_CONTENT_DIR . '/uploads/logs';

    // File name using the Argentine date format: day-month-year_orders.log
    $date_str = date('d-m-Y');
    $log_filename = "{$date_str}_orders.log";

    // Return the full path to the file for the current day
    return $uploads_dir . '/' . $log_filename;
}

```

`get_log_path()` does not write any content. It only builds and returns the path to the file for that day. For September 2, 2026, the resulting filename is `02-09-2026_orders.log` inside `wp-content/uploads/logs`.

The method assumes that `wp-content/uploads/logs` already exists and is writable. It contains neither `mkdir()` nor `wp_mkdir_p()`; therefore, the existence and permissions of that folder are part of the project installation.

26.6 How `log_manual_pay()` writes

Actual source code: `functions.php` → `WC_Gateway_Manual_Pay::log_manual_pay($message)` → lines 7310–7320.

```

private function log_manual_pay($message) {
    $log_file = $this->get_log_path();

```

```

        // Visual separator for each record
        $entry = trim($message) . "\n";

        $written = @file_put_contents($log_file, $entry, FILE_APPEND | LOCK_EX);
        if ($written === false) {
            error_log("manual-pay log write FAILED: $entry");
        }
    }
}

```

The method receives the already-built `text`. `get_log_path()` determines the file; `trim($message)` removes extra spaces or line breaks at the ends and adds a final newline. `file_put_contents()` writes with `FILE_APPEND` to add the new event to the end of the existing file and with `LOCK_EX` to request an exclusive lock during writing.

If `file_put_contents()` returns `false`, the code uses `error_log()` as an alternative output. That fallback prevents the failure from disappearing completely, although it does not create a second copy of the order file.

26.7 Complete Alex example: from WC_Order to the file

```

WC_Order #1530
↓
WordPress date/time
↓
Order #: 1530
Customer: Alex
Email: alex@example.com
Name for the artworks: Raven
↓
Cyberpunk Portrait - $18.00
Frame Color: Blue
Comic Balloon Text: Welcome back
↓
Total: $18.00
↓
$this->log_manual_pay($log)
↓
02-09-2026_orders.log

```

The file is not generated from a Checkout snapshot. It is rebuilt from the `WC_Order` that already contains the persisted information. This decision allows the record to reflect the actual order WooCommerce has just created.

26.8 What else is appended to the same log

The order record is only the first event. Later, the same `process_payment()` calls `log_manual_pay()` again after the buyer `wp_mail()` and after the administration `wp_mail()`. Finally, it adds a separator line. As a result, the same daily file can trace the complete sequence of each manual purchase.

```

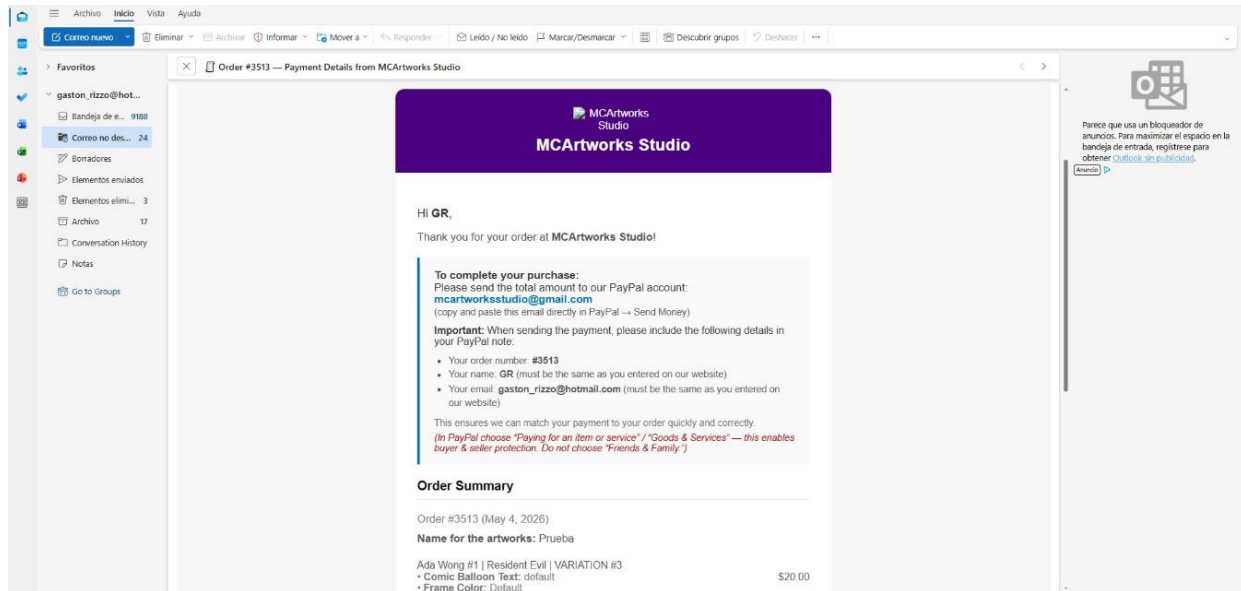
1. WC_Order details
↓
2. buyer email result
↓
3. administration email result
↓
4. final order separator

```

27. Buyer email

After writing the order details to the log, the same execution of `process_payment()` builds an HTML email for the buyer. There is no separate function that “sends the customer email”: the content is assembled inside the method itself and passed to WordPress through `wp_mail()`.

Project location: `wp-content/themes/blocksy-child/functions.php` → `WC_Gateway_Manual_Pay::process_payment($order_id)` → approximately lines 7383–7514. `$order` is already a `WC_Order` and `$artwork_name` has already been retrieved while building the log.



Real example of the email received by the buyer. The screenshot illustrates the visible output; the following subsections explain the code that builds it.

27.1 State available when the email block begins

At this point, Cart is not read again. \$order contains the already-created order; its WC_Order_Item objects retain Frame Color and Comic Balloon Text; \$artwork_name contains the order-level Artworks Name value. The block only prepares HTML representations of those data.

Actual source code: functions.php → beginning of the buyer-email block → lines 7383–7385.

```
// === Build HTML summary ===
$order_date = $order->get_date_created() ? $order->get_date_created()->date('F j, Y') : '';
$items_html = '';
```

\$order_date is obtained from the order creation date and formatted as “F j, Y”. \$items_html starts empty because it will be filled with one HTML row for each product.

27.2 Building \$items_html product by product

Actual source code: functions.php → process_payment() → lines 7387–7416.

```
// Build the product list
foreach ($order->get_items() as $item) {
    $product_name = esc_html($item->get_name());
    $total = number_format($item->get_total(), 2);
    $meta_html = '';

    $product_id = $item->get_product_id();
    $tiene_globo = get_field('tiene_globo', $product_id); // true/false

    foreach ($item->get_meta_data() as $meta) {
        $key = esc_html($meta->key);
        $value = esc_html($meta->value);

        // Show "Comic Balloon Text" only when tiene_globo is true
        if ($tiene_globo && strtolower($key) === 'comic balloon text') {
            $meta_html .= "<br><span style='color:#777;'>•";
        }

        <strong>{$key}</strong> <span>{$value}</span>";
    }

    // Frame Color is always displayed
    if (strtolower($key) === 'frame color') {
        $meta_html .= "<br><span style='color:#777;'>•";
    }

    <strong>{$key}</strong> <span>{$value}</span>";
}
```

```

    }

    $items_html .= "
    <tr>
        <td style='padding:6px 0;border-bottom:1px solid
#eee;'>{$product_name}{$meta_html}</td>
        <td style='padding:6px 0;border-bottom:1px solid #eee;text-
align:right;'>\${$total}</td>
    </tr>";
}

```

This foreach iterates over the WC_Order again, but the goal now is not the log file; it is an HTML table for the emails. `esc_html()` is applied to names, keys, and values before they are embedded in HTML. `$meta_html` gathers the customizations that will be displayed below the product name.

The ACF condition is preserved: Frame Color is included whenever it appears in the metadata; Comic Balloon Text is included only if `tiene_globo` is enabled. Finally, `$items_html` appends a `<tr>` row with the product and price. If the order contains several products, the string ends up containing several consecutive rows.

```

WC Order #1530
↓ get_items()
Cyberpunk Portrait
↓
$meta_html = Frame Color: Blue + Comic Balloon Text: Welcome back
↓
$items_html += product HTML row

```

27.3 Artworks Name and logo URL

Actual source code: `functions.php` → `process_payment()` → lines 7418–7426.

```

    $artwork_html = '';
    if ($artwork_name) {
        $artwork_html = '<p style="color:#555;margin:5px 0 15px;"><strong>Name for the
artworks:</strong> ' . esc_html($artwork_name) . '</p>';
    }

    // Path to the logo
    $uploads_dir = wp_upload_dir();
    $logo_url = trailingslashit($uploads_dir['baseurl']) . 'images/logo.png';

    // $logo_url = home_url('/images/logo.png');

```

`$artwork_html` starts empty and is completed only if the order has Artworks Name. Raven is escaped with `esc_html()` before being inserted into the HTML paragraph. For the logo, `wp_upload_dir()` returns information about the uploads directory and `trailingslashit()` guarantees the trailing slash before `images/logo.png` is appended.

Here PHP only builds a URL and an HTML string. The image is not physically embedded inside `functions.php`; the email uses the resulting URL in the `<img>` `src` attribute.

27.4 Buyer template: header and greeting

Actual source code: `functions.php` → `$message` variable → lines 7428–7443. This is the beginning of a single HTML string that continues in the following subsections.

```

    // === Elegant HTML template ===
    $message = '
    <html>
    <body style="background:#f6f6f6;padding:40px 0;margin:0;font-family:Helvetica,Arial,sans-serif;">
    <table align="center" width="600" cellpadding="0" cellspacing="0"
style="background:#ffffff;border-radius:12px;overflow:hidden;">
        <td style="background:#4B0082;color:#ffffff;padding:25px 30px;text-align:center;">
            
            <div style="font-size:22px;font-weight:bold;letter-spacing:0.5px;">MC Artworks
Studio</div>
        </td>
    </tr>

```

```

        <td style="padding:30px;">
        <p style="font-size:16px;color:#333;">Hi <strong>' . esc_html($order-
>get_billing_first_name()) . ' </strong>, </p>
        <p style="font-size:15px;color:#555;">Thank you for your order at <strong>MCArtworks
Studio</strong>!</p>

```

The string opens <html> and <body>, creates a centered 600 px table, and uses purple #4B0082 for the header. esc_url(\$logo_url) inserts the logo URL. The greeting obtains First Name directly from \$order and protects it with esc_html().

27.5 Buyer template: PayPal payment instructions

Actual source code: continuation of the same \$message → lines 7445–7466. This is not a second template: these lines remain concatenated within the string that began at 7429.

```

        <div style="background:#fafafa;border-left:4px solid #0070ba;padding:15px
20px;margin:20px 0;">
        <p style="margin:0;color:#333;font-size:15px;">
        <strong>To complete your purchase:</strong><br>
        Please send the total amount to our PayPal account:<br>
        <span style="color:#0070ba;font-
weight:bold;">mcartworksstudio@gmail.com</span>
        <br><span style="font-
size:13px;color:#555;">(copy and paste this email directly in PayPal → Send Money)</span>
        </p>
        <p style="margin:10px 0 0;color:#333;font-size:14px;">
        <strong>Important:</strong> When sending the
payment, please include the following details in your PayPal note:
        </p>
        <ul style="margin:8px 0 0
20px;padding:0;color:#555;font-size:13px;line-height:1.5;">
        <li>Your order number: <strong># ' . $order-
>get_order_number() . ' </strong></li>
        <li>Your name: <strong>' . esc_html($order-
>get_billing_first_name()) . ' . esc_html($order->get_billing_last_name()) . ' </strong> (must be the same
as you entered on our website)</li>
        <li>Your email: <strong>' . esc_html($order-
>get_billing_email()) . ' </strong> (must be the same as you entered on our website)</li>
        </ul>
        <p style="margin:10px 0 0;color:#777;font-size:13px;">
        This ensures we can match your payment to your
order quickly and correctly.
        </p>
        <p style="margin:6px 0 0;color:#a00;font-size:13px;font-
style:italic;">
        (In PayPal choose “Paying for an item or service” /
“Goods & Services” – this enables buyer & seller protection. Do not choose “Friends & Family.”)
        </p>
        </div>

```

This block contains the instructions that make the gateway a manual payment method. The email specifies the PayPal account, asks the buyer to include the order number, name, and email in the note, and warns that Goods & Services must be used. The number, name, and email are obtained from the WC_Order, so the buyer receives data linked to the order just created.

27.6 Buyer template: summary, total, delivery, and closing

Actual source code: continuation and end of \$message → lines 7468–7499.

```

        <h3 style="border-bottom:2px solid #eee;padding-bottom:8px;color:#111;">Order
Summary</h3>
        <p style="color:#777;margin:0 0 10px;">Order # ' . $order-
>get_order_number() . ' ( ' . $order_date . ' )</p>
        ' . $artwork_html . '
        <table width="100%" cellpadding="0" cellspacing="0" style="font-
size:14px;color:#555;border-collapse:collapse;">
        ' . $items_html . '
        <tr>
        <td style="padding-top:10px;font-weight:bold;">Total to Pay:</td>

```

```

        <td align="right" style="padding-top:10px;font-weight:bold;color:#111;">$' .
number_format($order->get_total(), 2) . '</td>
    </tr>
</table>

    <p style="margin:25px 0 5px;color:#777;font-size:13px;">
        Once we receive your payment, we'll confirm it via email and deliver your
artworks within 24 hours.
    </p>
    <p style="margin:0;color:#999;font-size:12px;">
        Order will remain valid for 48 hours.
    </p>

    <p style="margin-top:30px;font-size:14px;color:#555;">
        Thank you for your trust!<br>
        <strong>- MCArtworks Studio</strong>
    </p>
</td>
</tr>
<tr>
    <td style="background:#4B0082;color:#ffffff;font-size:12px;text-
align:center;padding:15px;font-weight:600;">
        © ' . date('Y') . ' MCArtworks Studio - All rights reserved
    </td>
</tr>
</table>
</body>
</html>';

```

The summary inserts three pieces that have already been prepared: `$order_date`, `$artwork_html`, and `$items_html`. `get_total()` then adds the order total. The final text states that the artworks will be delivered within 24 hours after payment is confirmed and that the order remains valid for 48 hours. `date('Y')` completes the year in the footer.

The presence of `" . $items_html . "` inside the template does not execute the `foreach` again. At that point PHP simply inserts the HTML string already built in Section 27.2.

27.7 Headers, subject, wp_mail(), and result logging

Actual source code: `functions.php` → end of the buyer email block → lines 7501–7514.

```

// === Send email ===
$headers = [
    'Content-Type: text/html; charset=UTF-8',
    'From: MCArtworks Studio <shop@mcartworksstudio.com>'
];

$subject = '📦 Order #' . $order->get_order_number() . ' - Payment Details from
MCArtworks Studio';
$result = wp_mail($order->get_billing_email(), $subject, $message, $headers);

if (!$result) {
    $this->log_manual_pay("⚠️ ERROR sending email to {$order->get_billing_email()} for order
#{$order->get_order_number()}");
} else {
    $this->log_manual_pay("✅ Email sent successfully to {$order->get_billing_email()} for order
#{$order->get_order_number()}");
}

```

`$headers` specifies text/html content with UTF-8 and defines the visible sender. `$subject` includes the actual order number. `wp_mail()` receives the billing email from the `WC_Order` as the recipient, followed by subject, HTML body, and headers. The returned boolean is stored in `$result` and immediately logged through `log_manual_pay()`.

Evaluation occurs before the administration email begins. Therefore, the buyer result is written to the log even though the `$result` variable is later reused for the second `wp_mail()`.

27.8 Guided example: what values Alex's email receives

```

$order->get_billing_email() → alex@example.com
$subject → Order #1530 - Payment Details from MCArtworks Studio

```

```

$artwork_html → Raven
$item_html → Cyberpunk Portrait + Blue + Welcome back
$order->get_total() → 18.00
↓
wp_mail(alex@example.com, $subject, $message, $headers)
↓
$result
↓
success or error is logged

```

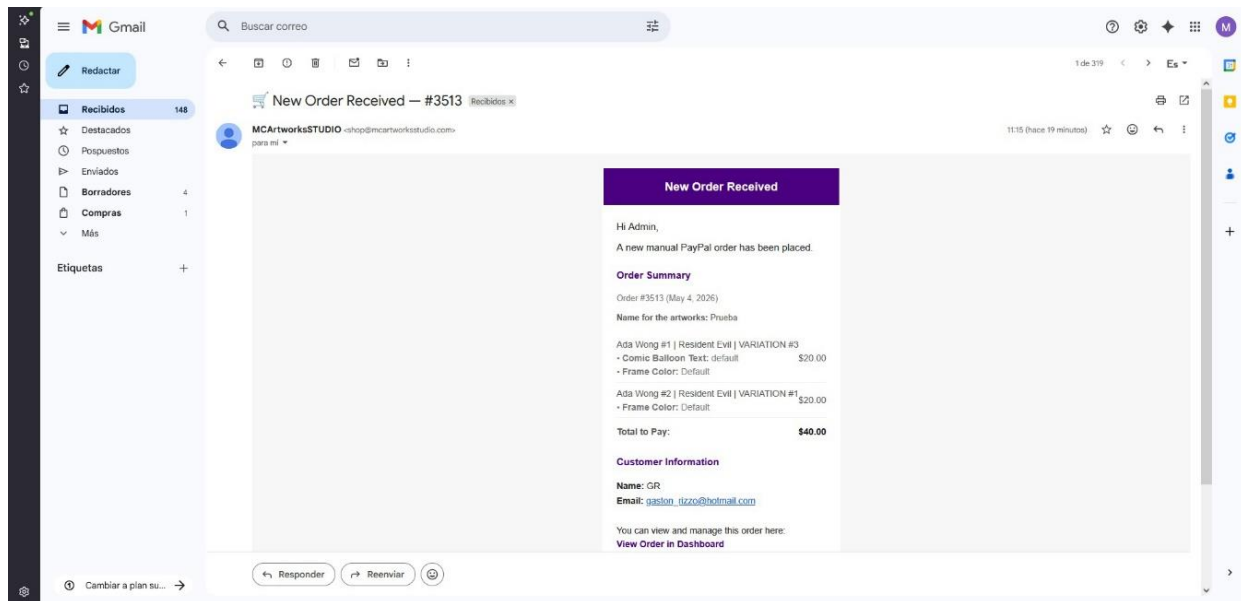
Thus, the template does not contain hand-written fictional values for each order. PHP inserts the data it reads from the specific WC_Order being processed.

28. Administrator email

After evaluating and logging the buyer `wp_mail()`, `process_payment()` continues without leaving the method. The second email uses a different HTML template intended for administration; it reuses the already-built product summary and adds internal customer information together with a link for managing the order.

Project location: `wp-content/themes/blocksy-child/functions.php` → `WC_Gateway_Manual_Pay::process_payment()` → lines 7520–7584. It does not start a second WooCommerce process or recreate the order.

Execution continuity: `$order`, `$order_id`, `$artwork_html`, `$item_html`, and `$headers` remain available in memory. `$order_date` is calculated again before `$admin_message` is built.



Real example of the administration email. The source code that builds this output is documented below.

28.1 Recipient, subject, and date

Actual source code: `functions.php` → beginning of the administration block → lines 7520–7523.

```

$admin_email = 'mcartworksstudio@gmail.com';
$admin_subject = '🛒 New Order Received — #' . $order->get_order_number();

$order_date = $order->get_date_created() ? $order->get_date_created()->date('F j, Y')
: '';

```

`$admin_email` contains the fixed internal recipient configured in the theme. `$admin_subject` includes the number of the order being processed. `$order_date` again obtains the creation date of the same WC_Order for display within the summary.

```

WC_Order #1530
↓
$admin_email = mcartworksstudio@gmail.com
$admin_subject = New Order Received — #1530
$order_date = order creation date

```

28.2 Administration template: header and order summary

Actual source code: functions.php → beginning of \$admin_message → lines 7525–7549. These lines form the first part of a single HTML string.

```
$admin_message = '
<html>
<body style="font-family: Arial, sans-serif; background-color: #f7f7f7; padding: 20px;
color: #222;">
    <table style="max-width:600px; margin:auto; background:#fff; border-radius:8px;
overflow:hidden;">
        <tr>
            <td style="background:#4B0082; color:#fff; text-align:center; padding:15px;
font-size:18px;">
                <strong>New Order Received</strong>
            </td>
        </tr>
        <tr>
            <td style="padding:20px;">
                <p style="font-size:15px; margin:0 0 10px 0;">Hi Admin,</p>
                <p style="font-size:15px; margin:0 0 15px 0;">A new manual PayPal
order has been placed.</p>
                <h3 style="color:#4B0082; margin-top:20px;">Order Summary</h3>
                <p style="color:#777;margin:0 0 10px;">Order #'. $order-
>get_order_number() . ' (' . $order_date . ')</p>
                ' . $artwork_html . '
                <table width="100%" cellpadding="0" cellspacing="0" style="font-
size:14px;color:#555;border-collapse:collapse;">
                    ' . $items_html . '
                    <tr>
                        <td style="padding-top:10px;font-weight:bold;">Total to
Pay:</td>
                        <td align="right" style="padding-top:10px;font-
weight:bold;color:#111;">$' . number_format($order->get_total(), 2) . '</td>
                    </tr>
                </table>
```

The template opens an independent HTML document and creates a centered card. The purple header displays “New Order Received”. Order Summary inserts the number, date, \$artwork_html, and \$items_html. The total is obtained again from \$order->get_total().

Reusing \$items_html is deliberate: the administration email does not read Cart again or rebuild the metadata. It uses exactly the HTML rows process_payment() generated earlier for the buyer, keeping the same product, Frame Color, and Comic Balloon Text in both messages.

28.3 Administration template: customer and Dashboard link

Actual source code: continuation of \$admin_message → lines 7551–7574. The link is generated inside the href attribute through admin_url().

```
<h3 style="color:#4B0082; margin-top:25px;">Customer Information</h3>
<p style="font-size:14px; line-height:1.6em;">
    <strong>Name:</strong> ' . esc_html($order-
>get_billing_first_name()) . ' ' . esc_html($order->get_billing_last_name()) . '<br>
    <strong>Email:</strong> ' . esc_html($order->get_billing_email()) .
'<br>
</p>
<p style="margin-top:25px; font-size:14px;">
    You can view and manage this order here:<br>
    <a href="' . admin_url('post.php?post=' . $order_id .
'&action=edit') . '"
        style="color:#4B0082; font-weight:bold; text-
decoration:none;">
        View Order in Dashboard
    </a>
</p>
```

```

        </td>
      </tr>
      <tr>
        <td style="background:#4B0082; color:#fff; text-align:center; padding:10px;
font-size:12px;">
          © ' . date('Y') . ' MCArtworks Studio – Internal Order Notification
        </td>
      </tr>
    </table>
  </body>
</html>
';

```

Customer Information reads First Name, Last Name, and Email from the WC_Order and escapes them with `esc_html()`. `admin_url()` then builds a WordPress admin-panel URL from `post.php?post=ORDER_ID&action=edit`. `$order_id` is the same identifier WooCommerce passed at the beginning of `process_payment()`.

```

Administration email
↓
View Order in Dashboard
↓
admin_url( post.php?post=1530&action=edit )
↓
order #1530 in the WordPress / WooCommerce admin panel

```

The documentation describes what the current source code builds. The link does not search for the order by name or email; it uses `$order_id` directly.

28.4 Sending, result, and block closing

Actual source code: `functions.php` → lines 7576–7584.

```

        $result = wp_mail($admin_email, $admin_subject, $admin_message, $headers);

        if (!$result) {
            $this->log_manual_pay("⚠ ERROR sending email to the admin for order #{ $order-
>get_order_number() }");
        } else {
            $this->log_manual_pay("✅ Email sent successfully to the admin for order
#{ $order->get_order_number() }");
        }

        $this-
>log_manual_pay("=====
\n\n");

```

`wp_mail()` uses the administration recipient, subject, and body, but reuses `$headers` from the buyer email. The assignment overwrites `$result`; immediately afterward, the code logs success or failure with an administration-specific message. Finally, a separator line is appended to the log file to visually close that order record.

28.5 Guided example: Alex through the entire administration email

```

WC_Order #1530 already exists
↓
$admin_subject = New Order Received – #1530
↓
$admin_message inserts:
Raven
Cyberpunk Portrait
Frame Color: Blue
Comic Balloon Text: Welcome back
Total: $18.00
Alex · alex@example.com
View Order in Dashboard → order_id 1530
↓
wp_mail(mcartworksstudio@gmail.com, ...)
↓
$result
↓
log_manual_pay()

```

This is the difference from the buyer email: both share the order summary, but the administration message removes payment instructions and adds information for reviewing and managing the order internally.

29. Email delivery and error tracking

Sections 27 and 28 explain how the theme builds the two HTML bodies. This section follows the next step: what `wp_mail()` receives, what its result means, and how to distinguish a theme-code problem, an SMTP transport problem, and a log-file problem.

Project location: `functions.php` → `WC_Gateway_Manual_Pay::process_payment()`. The buyer `wp_mail()` is on line 7508 and the administration call on 7576. Both results are logged through `WC_Gateway_Manual_Pay::log_manual_pay()`. WP Mail SMTP is not implemented inside the child theme; it acts afterward within the WordPress mail infrastructure.

29.1 The two actual `wp_mail()` calls

Actual buyer source code: `functions.php` → lines 7502–7508.

```
$headers = [
    'Content-Type: text/html; charset=UTF-8',
    'From: MC Artworks Studio <shop@mcartworksstudio.com>'
];

$subject = '📄 Order #' . $order->get_order_number() . ' – Payment Details from
MC Artworks Studio';
$result = wp_mail($order->get_billing_email(), $subject, $message, $headers);
```

Actual administration source code: `functions.php` → line 7576. `$headers` already exists because it was created earlier for the buyer.

```
$result = wp_mail($admin_email, $admin_subject, $admin_message, $headers);
```

Both calls use the same WordPress function and the same four-argument structure. The recipient, subject, and HTML body differ.

<code>wp_mail()</code> argument	Buyer	Administration
<code>\$to</code>	<code>\$order->get_billing_email()</code>	<code>\$admin_email</code>
<code>\$subject</code>	<code>\$subject</code>	<code>\$admin_subject</code>
<code>\$message</code>	<code>\$message</code>	<code>\$admin_message</code>
<code>\$headers</code>	<code>\$headers</code>	<code>\$headers</code>

29.2 What each argument means

The first argument specifies the recipient. The second is the subject. The third is the complete message body; in this project, it contains HTML. The fourth contains headers, including `Content-Type: text/html; charset=UTF-8` and the `From` sender. No fifth attachments argument is passed, so these emails do not attach files through `wp_mail()`.

`wp_mail()` does not read the `WC_Order` again. When it is called, PHP has already converted the order data into `$message` or `$admin_message`. `wp_mail()` receives prepared strings and passes them to the WordPress mail system.

29.3 What happens after `wp_mail()`

```
functions.php / process_payment()
↓
wp_mail()
↓
WordPress prepares delivery
↓
WP Mail SMTP acts as transport infrastructure
↓
configured SMTP server or service
↓
recipient server
↓
Inbox / Spam / rejection
```

The child theme controls construction of the recipient, subject, HTML, and headers. After `wp_mail()`, delivery also depends on WordPress, the SMTP plugin configuration, the outgoing service, and the receiving server. Therefore, order status and email-delivery status are related but distinct concerns.

29.4 How the buyer result is evaluated

Actual source code: `functions.php` → lines 7510–7514.

```
if (!$result) {
    $this->log_manual_pay("⚠ ERROR sending email to {"$order->get_billing_email()} for order
#{"$order->get_order_number()}");
} else {
    $this->log_manual_pay("✅ Email sent successfully to {"$order->get_billing_email()} for order
#{"$order->get_order_number()}");
}
```

If `$result` is false, the gateway writes an ERROR associated with the buyer email and the order number. If it is true, it records that WordPress successfully accepted the delivery attempt. The log is written before the administration block begins.

29.5 How the administration result is evaluated

Actual source code: `functions.php` → lines 7578–7582.

```
if (!$result) {
    $this->log_manual_pay("⚠ ERROR sending email to the admin for order #{"$order->get_order_number()}");
} else {
    $this->log_manual_pay("✅ Email sent successfully to the admin for order
#{"$order->get_order_number()}");
}
```

The logic is equivalent, but the log message explicitly identifies the administrator. This allows the daily file to show that one email succeeded and the other failed without confusing the two events.

29.6 Why reusing `$result` does not erase the first result

The `$result` variable is overwritten when the second `wp_mail()` runs. However, the previous value has already been evaluated in the if statement on lines 7510–7514 and its message has already been passed to `log_manual_pay()`. What is lost in memory is the earlier boolean; the line already written to the file is not removed.

```
wp_mail(buyer) → $result #1
↓
if/else evaluates $result #1
↓
log_manual_pay() writes the buyer result
↓
wp_mail(admin) → $result #2 replaces the variable
↓
if/else evaluates $result #2
↓
log_manual_pay() writes the administration result
```

29.7 Three failure levels that should not be confused

Level	What can happen	Where to look
1. <code>wp_mail()</code>	<code>wp_mail()</code> returns false and <code>process_payment()</code> records ERROR.	Daily <code>manual_pay</code> file and gateway code.
2. SMTP transport	<code>wp_mail()</code> returns true, but the message may end up in Spam, be rejected, or fail afterward.	WP Mail SMTP logs/configuration and the SMTP service.
3. Log file	Sending may be attempted correctly, but <code>file_put_contents()</code> can fail when saving the record.	uploads/logs permissions and <code>error_log()</code> .

Do not confuse “wp_mail() returned true” with “the recipient read the email” or even with “the message reached Inbox.” true only confirms that WordPress was able to process the send request at that point in the flow.

29.8 What happens if the log file itself fails

Actual source code: functions.php → end of log_manual_pay() → lines 7316–7319.

```
$written = @file_put_contents($log_file, $entry, FILE_APPEND | LOCK_EX);
if ($written === false) {
    error_log("manual-pay log write FAILED: $entry");
}
```

If writing the file returns false, error_log() receives the text as an alternative output. The project does not implement a retry queue, a second backup file, or a custom database for these events here.

29.9 Why the order continues even if an email fails

Actual source code: functions.php → end of process_payment() → lines 7586–7589. The return executes after both email attempts.

```
return [
    'result' => 'success',
    'redirect' => $this->get_return_url($order),
];
```

The email if statements only record success or failure; they do not return an error or cancel the WC_Order. Therefore, process_payment() still reaches result = success and returns the order return URL. On the next request, the browser opens Order Received, which is covered in Section 30.

```
wp_mail() may fail
↓
the ERROR is logged
↓
the WC_Order is preserved
↓
process_payment() continues
↓
return result = success + get_return_url($order)
↓
Order Received
```

29.10 Step-by-step operational troubleshooting

To diagnose a manual order, it is useful to follow the same order as the code. First verify that the WC_Order exists and contains the products and customizations. Then review the daily manual_pay file to see what each wp_mail() returned. If the log reports success but the message does not appear in the mailbox, review WP Mail SMTP and the SMTP service. If the event does not even appear in the file, check uploads/logs and the server error log.

1. WooCommerce → does the WC_Order exist and contain the correct data?
- ↓
2. uploads/logs/DATE_orders.log → what did process_payment() record?
- ↓
3. WP Mail SMTP / SMTP service → did transport accept or reject the message?
- ↓
4. receiving server → Inbox, Spam, or rejection?

30. Order Received and complete example

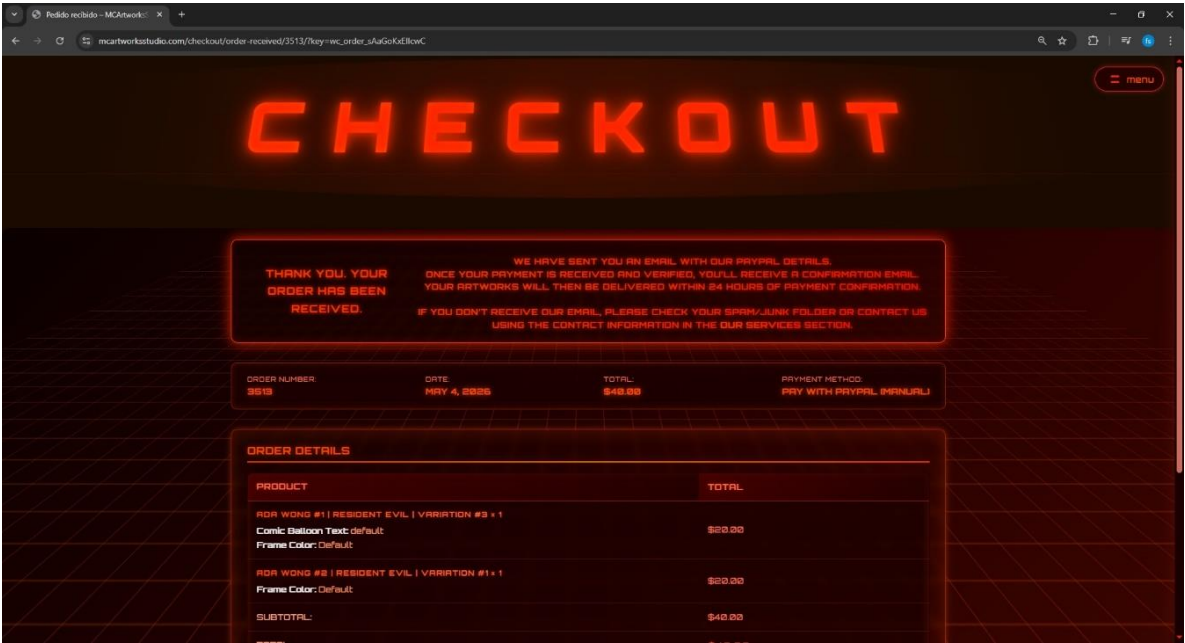
After the manual method is processed, WooCommerce opens the order return page. The Order Received screen is also customized to preserve the faction appearance and show the buyer what to do next.

HTTP request boundary. When process_payment() returns result=success and get_return_url(\$order), that method ends. The browser is redirected and opens the Order Received URL in a new request. The functions in this section run during that new WordPress load; they do not continue inside process_payment().

Project location: functions.php · the gateway return is approximately on lines 7586–7589; wpo_filtrar_meta_en_pedido() is registered on woocommerce_order_item_get_formatted_meta_data around line 7842 and begins near 7861; traducir_textos_pedido_recibido() is registered on wp_footer around line 8127 and begins near 8133; mostrar_mensaje_y_traducir_meses() is registered on wp_footer around line 8318 and begins near 8332.

REQUEST 1: Checkout
↓

```
WC_Gateway_Manual_Pay::process_payment()
↓
return success + get_return_url($order)
↓
browser redirect
=====
REQUEST 2: Order Received
↓
WordPress + WooCommerce rebuild the screen
↓
wp_footer runs translations/additional message
↓
the browser executes the output JavaScript
```



Post-checkout screen with an already-created order.

The same data throughout the purchase

Stage	Visible /stored data
Shop	Cyberpunk Portrait — USD 18
Cart	Frame Color = Blue · Comic Balloon Text = Welcome back
Checkout	Artworks Name = Raven · First Name = Alex · Email = alex@example.com
Order	Product + customization metadata + Artworks Name
Log	Order number, customer, email, product, options, and total
Buyer email	Summary + manual payment instructions
Administration email	Summary + customer data + link to the order
Order Received	Order confirmation and next payment step

30.1 How Order Received is reached

Source code: functions.php → end of WC_Gateway_Manual_Pay::process_payment(), approximately lines 7586–7589. The returned URL belongs to WooCommerce and triggers the subsequent navigation.

At the end of process_payment(), the gateway does not output a final page itself. It returns a success result and the URL calculated by get_return_url(\$order) to WooCommerce. WooCommerce then redirects to the Order Received endpoint.

```
return [
    'result' => 'success',
    'redirect' => $this->get_return_url($order),
];
```

30.2 What WooCommerce provides and what the theme provides

WooCommerce generates the base structure with the order number, date, email, payment method, products, and total. The child theme keeps that information, but changes text, adds instructions, and adapts the presentation so that the final screen is consistent with the rest of the site.

30.3 Dynamic text translation

Source code: functions.php → wp_footer hook → traducir_textos_pedido_recibido(), approximately lines 8127–8172. PHP checks is_order_received_page() and, when appropriate, outputs JavaScript; the text replacement then occurs in the browser.

traducir_textos_pedido_recibido() runs only when is_order_received_page() is true. JavaScript searches for Spanish text generated by WooCommerce and replaces it with its English equivalents without editing the plugin templates.

The function does not modify WooCommerce templates. It iterates over text nodes already generated on the page and replaces only the known strings with their English equivalents.

```
const traducciones = [
  { find: 'Gracias. Tu pedido ha sido recibido.',
    replace: 'THANK YOU. YOUR ORDER HAS BEEN RECEIVED.' },
  { find: 'Número de pedido:', replace: 'ORDER NUMBER:' },
  { find: 'Fecha:', replace: 'DATE:' },
  { find: 'Correo electrónico:', replace: 'EMAIL:' },
  { find: 'Método de pago:', replace: 'PAYMENT METHOD:' }
];

traducciones.forEach(t => {
  const regex = new RegExp(t.find, 'gi');

  jQuery('body').find('*').contents()
    .filter(function() {
      return this.nodeType === 3 && regex.test(this.nodeValue);
    })
    .each(function() {
      this.nodeValue = this.nodeValue.replace(regex, t.replace);
    });
});
```

“Gracias. Tu pedido ha sido recibido.”

↓

THANK YOU. YOUR ORDER HAS BEEN RECEIVED.

“Número de pedido:” → ORDER NUMBER:

“Fecha:” → DATE:

“Método de pago:” → PAYMENT METHOD:

30.4 Additional message after checkout

Source code: functions.php → wp_footer hook with priority 100 → mostrar_mensaje_y_traducir_meses(), approximately lines 8318–8399. PHP prepares \$extra_text and passes the text to the script through wp_json_encode().

mostrar_mensaje_y_traducir_meses() adds a practical explanation to the thank-you message: the PayPal details were sent by email, the payment must be verified, and the artworks are delivered within 24 hours after payment is confirmed. It also recommends checking Spam/Junk if the email does not appear.

The message is created as an additional inside the existing confirmation paragraph. The content comes from \$extra_text and is passed to JavaScript through wp_json_encode() so that a PHP string does not have to be inserted manually into the script.

```
var p = document.querySelector(
  '.woocommerce-thankyou-order-received'
);

if (p) {
  var span = document.createElement('span');
  span.className = 'woocommerce-thankyou-extra';
  span.style.display = 'block';
  span.style.marginTop = '8px';
```

```

    span.innerHTML = <?php echo wp_json_encode($extra_text); ?>;
    p.appendChild(span);
}

```

30.5 Dates and month names

The same function scans the order HTML and changes Spanish month names to English. This transformation is applied only on order-received; it does not modify the date stored in WooCommerce.

The replacement is performed only on the HTML of the order container displayed on Order Received. It does not change the date stored in WC_Order or the WordPress locale.

```

var monthsMap = {
    'enero': 'January',
    'febrero': 'February',
    'marzo': 'March',
    'abril': 'April',
    'mayo': 'May',
    'junio': 'June',
    'julio': 'July',
    'agosto': 'August',
    'septiembre': 'September',
    'octubre': 'October',
    'noviembre': 'November',
    'diciembre': 'December'
};

Object.keys(monthsMap).forEach(function(spanish) {
    var re = new RegExp('\\b' + spanish + '\\b', 'ig');
    html = html.replace(re, monthsMap[spanish]);
});

```

septiembre → September
 octubre → October
 noviembre → November
 diciembre → December

30.6 Visible metadata by product

Source code: `functions.php` → `woocommerce_order_item_get_formatted_meta_data` filter → `wpo_filtrar_meta_en_pedido()`, approximately lines 7842–7877. This filter acts on metadata already formatted by WooCommerce before it is displayed.

`wpo_filtrar_meta_en_pedido()` controls the metadata WooCommerce displays beneath each product. Its purpose is to prevent Comic Balloon Text from appearing on lines where it does not apply.

This fragment shows exactly where the observation about `es_bundle` comes from: the function checks it together with `tiene_globo` before deciding whether to remove Comic Balloon Text from the visible metadata.

```

function wpo_filtrar_meta_en_pedido($formatted_meta, $item) {
    $product = $item->get_product();
    if (!$product) return $formatted_meta;

    $product_id = $product->get_id();
    $is_bundle = get_field('es_bundle', $product_id);
    $tiene_globo = get_field('tiene_globo', $product_id);

    if ($is_bundle || !$tiene_globo) {
        $formatted_meta = array_filter(
            $formatted_meta,
            function($meta) {
                return strpos(
                    $meta->display_key,
                    'Comic Balloon Text'
                ) === false;
            }
        );
    }
}

```

```
return $formatted_meta;  
}
```

Difference between the code and the ACF configuration

wpo_filtrar_meta_en_pedido() checks es_bundle in addition to tiene_globo. The code expects that field, but the current ACF review does not include it among the four confirmed fields. When es_bundle is not configured, that check returns an empty value and the filter can rely only on tiene_globo when deciding whether to hide Comic Balloon Text.

30.7 Complete Alex example

Alex is no longer in the same Checkout response. His browser loaded a new WooCommerce URL. In that new request, WordPress retrieves the corresponding order, WooCommerce generates its summary, and only then do the theme callbacks modify the visible text and add the post-payment instructions.

```
SHOP  
Cyberpunk Portrait – USD 18  
↓  
CART  
Frame Color = Blue  
Comic Balloon Text = Welcome back  
↓  
CHECKOUT  
Artworks Name = Raven  
First Name = Alex  
Email = alex@example.com  
↓  
WC_ORDER #1530  
product + metadata + Artworks Name  
↓  
DAILY LOG  
order details  
↓  
BUYER EMAIL + ADMIN EMAIL  
↓  
get_return_url()  
↓  
ORDER RECEIVED  
confirmation + instructions for completing payment
```

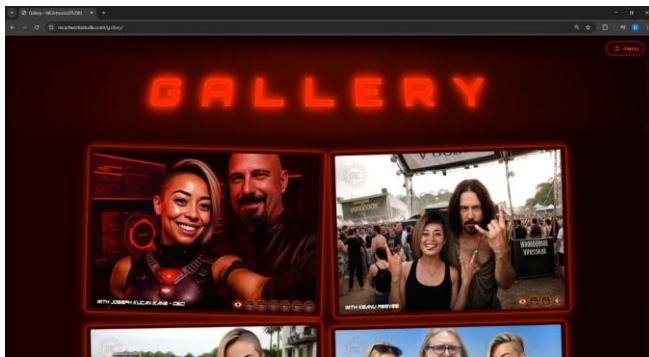
PART V — GALLERY, SERVICES, AND INTERFACE

31. Gallery

Gallery is a custom screen generated by the child theme. It does not obtain its videos from WooCommerce, Advanced Custom Fields (ACF), or a database query. The content list is defined directly in functions.php, inside `cnc_gallery_parallax()`. That function decides which files make up each frame, which alternative variants it has, and whether a sound control should be displayed.

Project location: `wp-content/themes/blocksy-child/functions.php` - `add_action('wp_head', 'cnc_gallery_parallax')` around line 8410 and `cnc_gallery_parallax()` from approximately line 8418 until before the Our Services block. The function brings together Gallery's PHP configuration, HTML, and JavaScript. Integration with the menu overlay belongs to `wp-content/themes/blocksy-child/header.php`; the visual styles depend on `gdi-faction-min.css` / `nod-faction-min.css`. The existing technical content in 31.1–31.16 is retained, and the Alex example in 31.17 merely walks through it from the user's perspective.

The MP4 files physically exist on the server, but the folder is not scanned automatically. Storing a file inside uploads does not make it appear in Gallery: it participates only if its URL was explicitly included in the `$videos` array for the corresponding faction.



NOD faction Gallery



GDI faction Gallery

31.1 Where each part of Gallery comes from

The screen is built by combining four different pieces. Separating them makes it possible to understand where each element the visitor ultimately sees comes from.

- `functions.php`: contains `cnc_gallery_parallax()`, the GDI/NOD video arrays, the HTML for the 12 frames, and the JavaScript that controls loaders, variants, sound, and parallax.
- `gdi-faction-min.css` / `nod-faction-min.css`: provide the visual appearance of the active faction: colors, frames, title, grid, controls, loaders, and responsive behavior.
- `wp-content/uploads/videos/`: contains the physical MP4 files. In the supplied NOD case, the Gallery files are in `wp-content/uploads/videos/NOD-GALLERY/`.
- `header.php`: integrates Gallery with the main menu; when the overlay is opened over `/gallery`, it pauses external `<audio>` elements, mutes external `<video>` elements, and saves their state so it can be restored when the menu closes.

```
/gallery/
↓
WordPress loads the child theme
↓
the faction cookie determines GDI or NOD
↓
the CSS for that faction is loaded
↓
wp_head executes cnc_gallery_parallax()
↓
functions.php selects the $videos array
```

```

↓
PHP generates 12 frame-container elements
↓
the browser loads the 12 main videos
↓
JavaScript connects loaders and controls

```

31.2 Where it is activated and where it is implemented

There is no `page-gallery.php` file in the child theme. The current implementation is grouped in `functions.php`. WordPress executes `cnc_gallery_parallax()` through `wp_head`, and the function stops immediately if the requested page is not Gallery.

```

add_action('wp_head', 'cnc_gallery_parallax');

function cnc_gallery_parallax() {
    if (!is_page('gallery')) return;

    // Gallery configuration, HTML, and JavaScript
}

```

Therefore, the same function brings together three layers: it first prepares the data in PHP, then outputs the Gallery HTML, and finally adds the JavaScript that controls the interaction.

31.3 How the faction is determined and where the visual appearance comes from

Gallery reads the same faction cookie used by the rest of the site. If it does not exist, it uses GDI as the default value. That choice affects both the video files and the background and loader.

```

$faction = $_COOKIE['faction'] ?? 'gdi';

if ($faction === 'gdi') {
    $body_bg = 'linear-gradient(270deg, #001f3f, #003f7f, #005bbb, #001f3f)';
    $loader_class = 'video-loader gdi';
    $videos = [ /* GDI configuration */ ];
} elseif ($faction === 'nod') {
    $body_bg = 'linear-gradient(270deg, #2a0000, #550000, #2a0000)';
    $loader_class = 'video-loader nod';
    $videos = [ /* NOD configuration */ ];
}

```

Loading the faction CSS is also decided from the cookie, but the current implementation contains two active `wp_enqueue_scripts` blocks with priority 999. The block marked PRODUCTION removes previous styles and uses `filemtime()` as the version; the block marked DEVELOPMENT VERSION attempts to enqueue the same stylesheet with a `?v=` parameter generated through `time()` to invalidate the cache during development.

```

/* PRODUCTION */
add_action('wp_enqueue_scripts', function() {
    $faction = $_COOKIE['faction'] ?? 'gdi';
    if (empty($faction)) return;

    wp_dequeue_style('gdi-faction');
    wp_dequeue_style('nod-faction');

    if ($faction === 'gdi') {
        $css_file = get_stylesheet_directory() . '/gdi-faction-min.css';
        $version = file_exists($css_file) ? filemtime($css_file) : '1.0';
        wp_enqueue_style('gdi-faction', get_stylesheet_directory_uri() . '/gdi-faction-min.css', [],
        $version, 'all');
    }
    // NOD uses the same pattern with nod-faction-min.css.
}, 999);

/* DEVELOPMENT VERSION */
add_action('wp_enqueue_scripts', function() {
    $faction = $_COOKIE['faction'] ?? '';

```

```

if (empty($faction)) return;

$theme_uri = get_stylesheet_directory_uri();
$timestamp = time();

if ($faction === 'gdi') {
    $url = $theme_uri . '/gdi-faction-min.css?v=' . $timestamp;
    wp_enqueue_style('gdi-faction', $url, [], null, 'all');
}
// NOD uses the same pattern with nod-faction-min.css.
}, 999);

```

Both blocks are active in functions.php. For that reason, they are documented separately: one is intended for production and the other for development. The GDI/NOD selection still depends on the faction cookie, but the versioning and caching strategy is not the same in both blocks.

One detail that can be confusing when reading the code is that the container keeps the `gdi-gallery-wrapper` class even when the faction is NOD. This does not mean NOD uses the GDI design: both stylesheets define that same selector with different rules, and the cookie decides which one is loaded.

The background is also applied at two levels. `cnc_gallery_parallax()` outputs a background for body through `$body_bg`, while the faction stylesheet defines the background and effects specific to `.gdi-gallery-wrapper`. The loader works similarly: there is a base `.video-loader` and then a `.video-loader.gdi` or `.video-loader.nod` variant.

31.4 The actual source of the videos: the \$videos array

Source code: `wp-content/themes/blocksy-child/functions.php` → `cnc_gallery_parallax()` → faction selection and `$videos` arrays approximately between lines 8421–8454. Each entry is defined manually in PHP; Gallery does not scan the folder to discover files.

The central point of Gallery is `$videos`. `glob()`, `scandir()`, `wp_upload_dir()`, `ACF`, and `WooCommerce` queries are not used to discover files. Every video that can appear is written manually in this array.

Each entry represents one Gallery frame and contains exactly three data fields:

main: the main video for the frame. It is created from PHP and begins loading when Gallery opens.

alt: an array of one or more alternative videos. They are not created initially; JavaScript generates them on demand.

sound: a Boolean flag indicating whether PHP should output the sound button for that frame.

```

[
    'main' => '/videos/nod-gallery/video2_main.mp4',
    'alt' => [
        '/videos/nod-gallery/video2_alt_one.mp4',
        '/videos/nod-gallery/video2_alt_two.mp4'
    ],
    'sound' => true
]

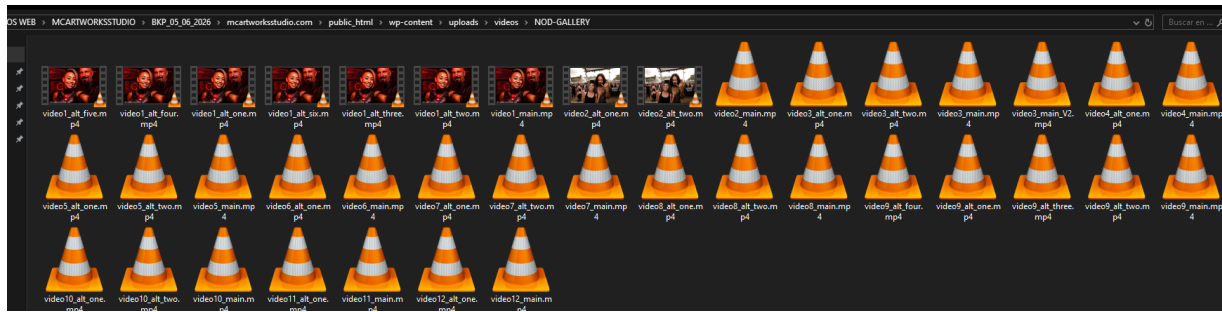
```

Both GDI and NOD have 12 entries, so the screen generates 12 frames for either faction. In the current code, each faction references 12 main files and 26 alt files: 38 video URLs per configuration.

The number of alternatives is not the same for every frame. For example, NOD video1 has six variants, video2 has two, and video4 has one. The number of  buttons is obtained directly from that count.

31.5 Physical files and public URLs are not the same thing

In the supplied screenshot, the NOD files are physically located in `public_html/wp-content/uploads/videos/NOD-GALLERY/`. The files `video1_main.mp4`, `video1_alt_one.mp4`, `video1_alt_two.mp4`, and the rest of the series can be seen there.



Physical NOD Gallery files inside wp-content/uploads/videos/NOD-GALLERY/.

However, `functions.php` does not use that physical path when building the HTML. The URLs written in `$videos` are of the form `/videos/nod-gallery/video1_main.mp4`. The child theme does not contain the rule that maps `/videos/nod-gallery/` to `wp-content/uploads/videos/NOD-GALLERY/`; that mapping belongs to the server configuration and must be preserved when the site is deployed.

Confirmed physical location:

public_html/wp-content/uploads/videos/NOD-GALLERY/video1_main.mp4

URL used by functions.php:

```
/videos/nod-gallery/video1_main.mp4
```

This also makes it possible to distinguish storage from configuration. The NOD screenshot shows 39 items, while the NOD array in functions.php references 38 files. The physical files include video3_main_V2.mp4, but that name is not included in the theme's \$videos array. Therefore, that file exists on the server but Gallery does not use it.

The practical rule is simple: an extra file in the folder is ignored if it is not in \$videos; conversely, if \$videos references a nonexistent or incorrectly mapped URL, the browser will attempt to load it and Gallery has no mechanism of its own to substitute another file.

31.6 How PHP converts \$videos into the 12 frames

Source code: `functions.php` → `cnc_gallery_parallax()` → `foreach($videos as $v)` approximately between lines 8494–8510. This section runs on the server and leaves `main`, `data-alt`, and `data-sound` written into the HTML that the browser will receive.

Once the faction array has been selected, PHP iterates over it with `foreach()`. Each entry produces a frame-container. The main video is written directly into the HTML; the alt paths are stored inside the `data-alt` attribute in JSON format, and sound is converted to 1 or 0 inside `data-sound`.

```
<?php foreach ($videos as $v): ?>

<div class="frame-container"
    data-alt='<?= json_encode($v['alt']) ?>'
    data-sound='<?= $v['sound'] ? 1 : 0 ?>'>

    <div class="video-wrap">
        <video class="vid-main"
            src="<?= $v['main'] ?>"
            autoplay
            muted
            loop
            playsinline
            preload="auto">
        </video>
    </div>

    <div class="frame"></div>

    <div class="video-controls">
        <span class="icon-eye">👁</span>

        <?php foreach ($v['alt'] as $index => $alt): ?>
            <span class="icon-video"
                data-alt="<?= $index ?>">🎬</span>
        <?php endforeach; ?>
    </div>
</div>
```

```

        <?php if ($v['sound']): ?>
            <span class="icon-sound"><img alt="sound icon" data-alt="0" /></span>
        <?php endif; ?>
    </div>
</div>

<?php endforeach; ?>

```

This block explains where each control comes from. Every frame receives a  button. The inner foreach creates one  button for each alt position. The  button appears only when sound is true.

31.7 How the paths pass from PHP to JavaScript

Source code: `functions.php` → `cnc_gallery_parallax()` → Gallery script started around line 8514; each control locates its `.frame-container` and reconstructs `altSources` from `dataset.alt`. This is the bridge between the data output by PHP and the browser logic.

PHP and JavaScript are connected through the frame-container `data-*` attributes. PHP does not create the alternative videos: it only leaves their URLs serialized in `data-alt`. When JavaScript initializes the controls, it retrieves that attribute and converts it back into an array.

```

const container = ctrl.closest('.frame-container');

const altSources = JSON.parse(container.dataset.alt);
const altHasSound = parseInt(container.dataset.sound);

```

Each  button contains only a numeric index in `data-alt`. For example, `data-alt="0"` means use `altSources[0]`, `data-alt="1"` means use `altSources[1]`, and so on. The button does not need to repeat the full URL because the list is already stored in the frame-container.

```

<span class="icon-video" data-alt="0"><img alt="video icon" data-alt="0" /></span>
<span class="icon-video" data-alt="1"><img alt="video icon" data-alt="1" /></span>

```

```

JavaScript:
idx = 0 → altSources[0]
idx = 1 → altSources[1]

```

31.8 What loads when the page opens and what is created only on click

Source code: `functions.php` → `cnc_gallery_parallax()` → `.video-controls` JavaScript around lines 8580–8643. `altCache` lives inside the initialization of each control; the `<video class="vid-alt">` elements are created only when the user requests a variant.

When Gallery opens, the 12 `vid-main` elements already exist because PHP output them in the HTML. All of them use `autoplay`, `muted`, `loop`, `playsinline`, and `preload="auto"`. Therefore, the browser can begin preparing the main videos during the initial load.

The alt variants work differently. There are not 26 alternative players created from the start. Only their URLs exist inside `data-alt`. The `<video class="vid-alt">` element is created when the visitor clicks the corresponding  button.

```

const altCache = {};

let v = altCache[idx];

if (!v) {
    v = document.createElement('video');
    v.src = altSources[idx];
    v.loop = true;
    v.playsInline = true;
    v.classList.add('vid-alt');

    wrap.appendChild(v);
    altCache[idx] = v;
}

```

`altCache` retains the player that has already been created. If the user later returns to the same variant, JavaScript reuses that element instead of creating another one.

31.9 Initial loader and waiting for loadeddata

Source code: functions.php → cnc_gallery_parallax() → “Initial loader” block approximately between lines 8546–8568; variants create an equivalent loader around lines 8628–8640. Both paths wait for readyState/loadeddata before revealing the video.

Each main video starts hidden through CSS. JavaScript adds a loader over video-wrap and waits until the browser has enough data to display the MP4. The loader class comes from PHP: video-loader gdi or video-loader nod.

```
const loader = document.createElement('div');
loader.className = '<?php echo $loader_class; ?>';
loader.innerHTML =
  '<div>Loading...</div><div class="spinner"></div>';

wrap.appendChild(loader);

function showMain() {
  loader.style.opacity = 0;
  setTimeout(() => loader.remove(), 500);
  main.classList.add('visible');
}

if (main.readyState >= 2) {
  showMain();
} else {
  main.addEventListener('loadeddata', showMain, { once: true });
}
```

readyState >= 2 means the browser already has enough resource data to display the current frame. The alt variants use the same criterion: if they have not yet reached that state, a temporary loader is created and finalize() runs after loadeddata.

31.10 How each button works

Source code: functions.php → cnc_gallery_parallax() → showVideo() from approximately line 8570 and .icon-eye/.icon-video/.icon-sound listeners between 8595–8652. They all act on the frame already created; they do not generate a new WordPress request.

 **button — return to the main video.** If a variant was active, it is paused and hidden. Then showVideo() displays vid-main, resets currentTime to 0, and starts playback again.

```
function showVideo(wrap, video) {
  wrap.querySelectorAll('video').forEach(v => {
    v.classList.remove('visible');
    v.pause();
  });

  video.classList.add('visible');
  video.currentTime = 0;
  video.play();
}

ctrl.querySelector('.icon-eye').addEventListener('click', () => {
  if (currentAlt) {
    currentAlt.pause();
    currentAlt.classList.remove('visible');
  }

  showVideo(wrap, main);
  setCurrent(main);
});
```

 **button — open a variant.** The button reads its data-alt index. If altCache already contains that index, it reuses the player. If it does not exist, it creates a new <video>, assigns the corresponding URL from altSources, and stores it in the cache.

```
ctrl.querySelectorAll('.icon-video').forEach(icon => {
  const idx = parseInt(icon.dataset.alt);

  icon.addEventListener('click', () => {
```

```

    let v = altCache[idx];

    if (!v) {
        v = document.createElement('video');
        v.src = altSources[idx];
        v.loop = true;
        v.playsInline = true;
        v.muted = !altHasSound;
        v.classList.add('vid-alt');

        wrap.appendChild(v);
        altCache[idx] = v;
    }

    // if already loaded → finalize()
    // otherwise → loader + loadeddata → finalize()
  });
});

```

 /  **button — control the visible video.** The button exists only when sound is true. If an active variant exists, it acts on currentAlt; otherwise it acts on main. When sound is enabled, `muteAllGlobal()` mutes all the other Gallery videos.

```

soundBtn.addEventListener('click', () => {
    const video = currentAlt || main;

    video.muted = !video.muted;

    if (!video.muted) {
        muteAllGlobal(video);
    }

    updateAllSoundIcons();
});

```

31.11 Only one video with sound across the entire Gallery

Audio control is global, not merely local to the frame. `muteAllGlobal()` iterates over all videos that exist inside `.frame-container` and applies `muted = true` to all except the one that was just enabled. `updateAllSoundIcons()` then checks each frame to display  or  according to the visible video.

```

function muteAllGlobal(exceptVideo) {
    document.querySelectorAll('.frame-container video').forEach(v => {
        if (v !== exceptVideo) {
            v.muted = true;
        }
    });

    updateAllSoundIcons();
}

```

31.11.1 Initial state of the sound icon

There is another difference between the actual state of the player and the icon shown when Gallery loads. When sound is true, PHP outputs the button with the initial text ; however, the main video is created with the muted attribute. `showMain()` only removes the loader and adds the visible class: it does not call `updateAllSoundIcons()`.

```

<!-- PHP outputs the button this way when sound = true -->
<span class="icon-sound" title="Activar/Desactivar sonido">

```

```
// updateAllSoundIcons() is not called here
}
```

Consequently, when a frame with `sound = true` is opened for the first time, the button displays  even though the main video is actually muted. The icon text is synchronized only when a later interaction calls `updateAllSoundIcons()`, for example when switching videos or using the sound control itself.

31.12 Visual movement: frame floating and video parallax

Source code: `functions.php` → `cnc_gallery_parallax()` → `--rot-angle` around lines 8541–8544 and the Parallax block from approximately 8656 to 8696. `requestAnimationFrame()` animates only the `.vid-main` visible in the browser.

Gallery combines two different effects. The first affects the entire frame-container: JavaScript generates a random angle between approximately -6° and $+6^\circ$ and stores it in the `--rot-angle` CSS variable. The stylesheets use that value inside the `gdiFloat` animation to produce the frame's floating motion.

```
const angle = (Math.random() * 12 - 6).toFixed(2) + 'deg';
container.style.setProperty('--rot-angle', angle);
```

The second effect is parallax. It is applied only to the main `.vid-main` video. The cursor position inside `video-wrap` is converted into a target offset, and `requestAnimationFrame()` gradually moves the current position toward that target.

```
const maxX = 18, maxY = 12, ease = 0.12;

const onMove = e => {
  const r = videoWrap.getBoundingClientRect();

  tX = -(((e.clientX - r.left) / r.width) - 0.5) * maxX;
  tY = -(((e.clientY - r.top) / r.height) - 0.5) * maxY;
};

function step() {
  cX += (tX - cX) * ease;
  cY += (tY - cY) * ease;

  main.style.transform =
    `translate3d(calc(-50% + ${cX}px),
      calc(-50% + ${cY}px), 0)`;

  raf = requestAnimationFrame(step);
}
```

The alt variants do not receive this parallax because the code explicitly selects `.vid-main`. The grid is also responsive: the stylesheets show two columns on desktop and a single column below 900 px.

31.13 Integration with the main menu

Source code: `wp-content/themes/blocksy-child/header.php` → `stopAllOtherAudio()` around line 514 and `restorePreviousAudio()` near line 533. This logic belongs to the menu, not to `cnc_gallery_parallax()`: it coordinates Gallery media when the HUD opens or closes.

Gallery also connects to `header.php`. This integration does not work only with videos. When the main menu is opened over `/gallery`, `stopAllOtherAudio()` checks all `<audio>` and `<video>` elements outside `#zajOverlay`. Audio elements that were playing are marked with `data-was-playing` and paused; videos that had sound enabled are marked with `data-was-unmuted` and muted. When the menu closes, `restorePreviousAudio()` restores only the states that had been recorded.

```
function stopAllOtherAudio() {
  document.querySelectorAll('audio').forEach(a => {
    if (!a.closest('#zajOverlay')) {
      a.dataset.wasPlaying = !a.paused ? 'true' : 'false';
      a.pause();
    }
  });

  document.querySelectorAll('video').forEach(v => {
    if (!v.closest('#zajOverlay')) {
      v.dataset.wasUnmuted = !v.muted ? 'true' : 'false';
      v.muted = true;
    }
  });
}
```



```
updateAllSoundIcons();
}
```

The consequence is important: if a variant belonging to a frame with `sound = false` contains an audio track, the current code can play it with sound and mute the other videos, but that frame has no  button because PHP did not generate one. The user therefore has no Gallery control of its own to mute that variant again. If the MP4 has no audio track, the problem is not noticeable, but the JavaScript logic remains the same. This documentation describes the existing behavior; it does not present it as the intended behavior.

31.16 Complete example: NOD, frame 2

The second NOD frame makes it possible to follow the complete flow because it has one main video, two variants, and `sound = true`.

```
[
  'main' => '/videos/nod-gallery/video2_main.mp4',
  'alt' => [
    '/videos/nod-gallery/video2_alt_one.mp4',
    '/videos/nod-gallery/video2_alt_two.mp4'
  ],
  'sound' => true
]
```

PHP generates a frame-container with one , two  buttons, and one  button. When Gallery opens, `video2_main.mp4` already exists as `vid-main` and starts muted. If the user clicks the first , JavaScript reads `data-alt="0"`, looks for `altCache[0]`, and, because it does not yet exist, creates a `vid-alt` with `video2_alt_one.mp4`. When `loadeddata` confirms that the file is ready, it displays it and stores it in the cache. If its sound is enabled, `muteAllGlobal()` mutes the videos in the other frames. Clicking  pauses the variant and returns the frame to `video2_main.mp4` from second 0.

```
NOD video2
↓
main = video2_main.mp4
alt[0] = video2_alt_one.mp4
alt[1] = video2_alt_two.mp4
sound = true
↓
PHP generates:    
↓
click the first 
↓
idx = 0
↓
altCache[0] does not exist
↓
create vid-alt + load video2_alt_one.mp4
↓
loadeddata
↓
display variant + store it in altCache
↓
click 
↓
return to video2_main.mp4
```

31.17 Guided example: Alex uses Gallery

The previous section already followed NOD frame 2 technically. This example walks through the same mechanism from a person's experience, without replacing any of the rules, inconsistencies, or implementation details documented in 31.1–31.16.

Alex enters Gallery with `faction=nod`. Before he sees the screen, PHP reads the `faction`, `cnc_gallery_parallax()` selects the NOD array, and generates the twelve frame-container elements. The main videos are already part of the HTML; the alternative variants still exist only as URLs inside `data-alt`.

Alex moves the cursor over the second frame. The frame retains its floating motion, and the main .vid-main video receives the parallax calculated from the pointer position. The alt variants do not receive that effect because the parallax code explicitly selects the main video.

Alex then clicks the first variant button. JavaScript reads data-alt="0" and checks altCache[0]. Because this is the first time, it does not find an existing player: it creates a new <video class="vid-alt">, assigns video2_alt_one.mp4, inserts it inside the frame, and waits for loadeddata. Once there is enough data, it displays the variant and stores that element in altCache.

Alex returns to the main video with the back button and later clicks the same variant again. This time altCache[0] already contains the previously created <video>, so JavaScript reuses it instead of building a second player. If Alex enables sound in a frame that has the control, muteAllGlobal() mutes the other videos to prevent multiple tracks from playing at the same time.

```
Alex opens Gallery with NOD
↓
PHP generates 12 frames from $videos
↓
Alex moves the cursor → parallax on vid-main
↓
Alex clicks a variant
↓
read data-alt
↓
Does altCache[idx] exist?
├ no → create vid-alt + wait for loadeddata + store
└ yes → reuse the player
↓
display variant
↓
back button → main video from the beginning
```

If Alex opens MENU while Gallery has active audio or video, header.php records which resources were playing or unmuted, pauses/mutes them outside the overlay, and, when the menu closes, attempts to restore only the states it had saved. In this way, the HUD does not mix its audio with the content Alex was viewing in Gallery.

Relationship to the existing implementation

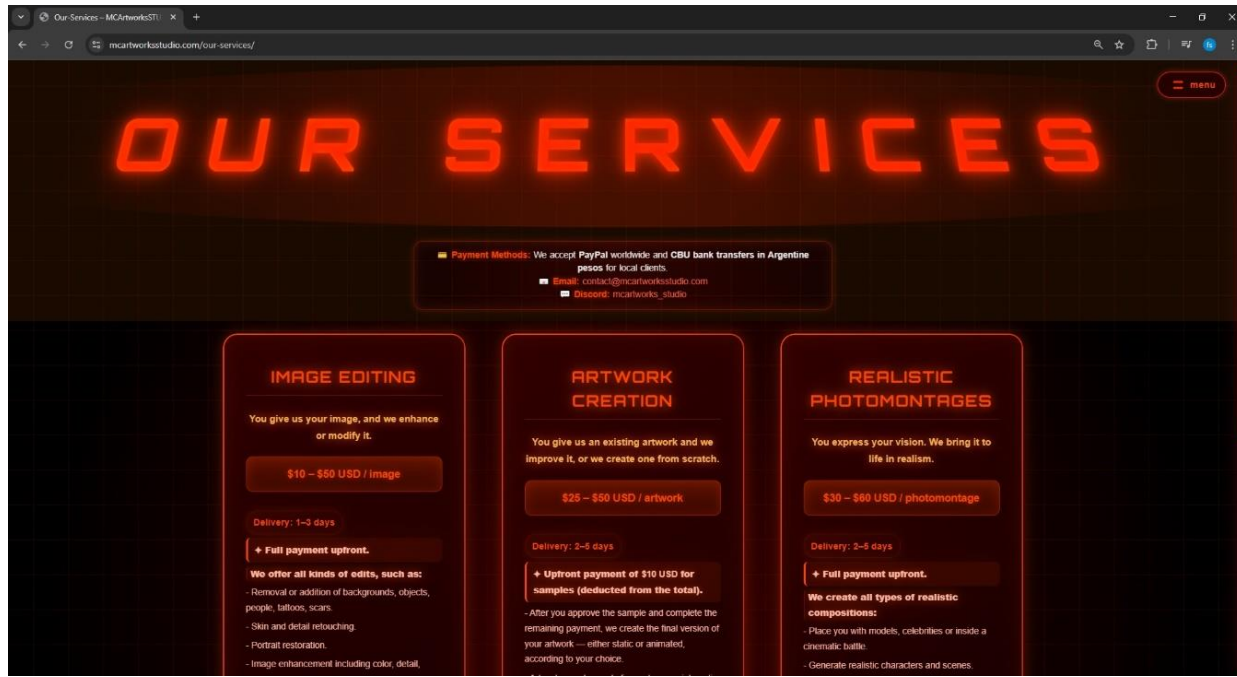
This Alex walkthrough is a teaching-oriented reading of the logic already documented. It does not replace the sections on altCache, loaders, sound, parallax, load failures, or integration with header.php; all of those behaviors remain the ones that define how Gallery actually works.

32. Services section (Our Services)

Our Services is an informational screen generated directly by the child theme. It does not query WC_Product or initiate a WooCommerce purchase flow: PHP outputs the contact panel, three service cards, and a final terms block, while the faction stylesheets determine their appearance.

Project location: wp-content/themes/blocksy-child/functions.php → add_action('wp_head', 'mostrar_servicios_personalizados') at line 8713 → mostrar_servicios_personalizados() from line 8718 to 8809. The styles for .your-services, .your-services-card, .payment-info-panel, and .info-panel are defined in gdi-faction.css / nod-faction.css and their minified versions.

Current implementation detail: the callback is registered in wp_head, but it outputs visible body HTML. The documentation describes that architecture as it exists; a future reorganization could move that content to a more semantically appropriate point in the body.



NOD Our Services: contact panel, three cards, and appearance applied by the faction.

32.1 Activation: from the URL to the callback

```
add_action('wp_head', 'mostrar_servicios_personalizados');

// =====
// Function that adds custom HTML content in <head> for the "Our Services" page.
// =====
function mostrar_servicios_personalizados() {

    if (is_page('our-services')) : ?>

        <style>
```

WordPress loads functions.php and registers the callback. In a normal request, wp_head triggers mostrar_servicios_personalizados(); the function checks is_page('our-services'). If the current page does not match, the block does not output the section. If it matches, it begins generating inline CSS and HTML.

```
/our-services/
↓
WordPress resolves the page
↓
wp_head
↓
mostrar_servicios_personalizados()
↓
is_page('our-services') = true
↓
PHP outputs visible content
```

32.2 Contact panel and payment methods

Actual source code: functions.php → mostrar_servicios_personalizados() → lines 8736–8749.

```
<div class="custom-shop-title">OUR SERVICES</div>

<div class="payment-info-panel">
  <p>
    <span class="highlight">Payment Methods:</span> We accept <b>PayPal</b> worldwide and
    <b>CBU bank transfers in Argentine pesos</b> for local clients.
    <br>
    <span class="highlight">Email:</span>
    <a href="mailto:contact@mcartworksstudio.com">contact@mcartworksstudio.com</a>
```

```

    <br>
    <span class="highlight">Discord:</span>
    <a href="https://discordapp.com/users/1432397605471649905" target="_blank">mcartworks_studio</a>
  </p>
</div>

```

The panel is static theme content: PayPal and CBU are shown as informational payment methods; the email link uses mailto:, and Discord opens an external URL. Nothing in this block calls the manual_pay gateway or generates a WC_Order.

32.3 The three cards come from the theme HTML

Actual source code: functions.php → .your-services → lines 8751–8795. There is no WooCommerce or ACF query to obtain these cards.

```

<section class="your-services">
  <div class="your-services-card">
    <h3>IMAGE EDITING</h3>
    <p>You give us your image, and we enhance or modify it.</p>
    <div class="price-box">$10 – $50 USD / image</div>
    <p><span class="highlight-delivery">Delivery: 1–3 days</span></p>
    <p class="highlight-upfront">♦ Full payment upfront.</p>
    <p class="highlight-makes">We offer all kinds of edits, such as:</p>
    <p>- Removal or addition of backgrounds, objects, people, tattoos, scars.</p>
    <p>- Skin and detail retouching.</p>
    <p>- Portrait restoration.</p>
    <p>- Image enhancement including color, detail, sharpness.</p>
    <p>- Gender, face, hair, or clothing changes.</p>
    <p><img alt="video icon" data-bbox="231 418 245 432"/> Want your edited image turned into a video?</p>
    <p>We can create up to three unique short videos from it.</p>
  </div>

  <div class="your-services-card">
    <h3>ARTWORK CREATION</h3>
    <p>You give us an existing artwork and we improve it, or we create one from scratch.</p>
    <div class="price-box">$25 – $50 USD / artwork</div>
    <p><span class="highlight-delivery">Delivery: 2–5 days</span></p>
    <p class="highlight-upfront">♦ Upfront payment of <b>$10 USD</b> for samples (deducted from
the total).</p>
    <p>- After you approve the sample and complete the remaining payment, we create the final
version of your artwork – either static or animated, according to your choice.</p>
    <p>- Artworks can be made for posters, social media, wallpapers, Steam, and more.</p>
    <p>- If you need us to make any adjustments or corrections, we will do so.</p>
    <p>- If it's for Steam, we deliver in your preferred format: Full, Frame, or Workshop.</p>
  </div>

```

The second and third cards continue with the same structure: title, description, price-box, turnaround time, and terms. The visible values are written literally in functions.php, so changing a price or text requires modifying that file or moving those data into a manageable configuration.

Service	Price written in the code	Delivery time written in the code
Image Editing	USD 10–50 / image	1–3 days
Artwork Creation	USD 25–50 / artwork	2–5 days
Realistic Photomontages	USD 30–60 / photomontage	2–5 days

32.4 Important Information and the Google Fonts dependency

```

<div class="section-divider"></div>

<div class="info-panel">
  <h4>Important Information</h4>
  <p><img alt="warning icon" data-bbox="181 908 195 922"/> We do not work with explicit content (nudity or extreme gore).</p>

```

```

    <p>Ⓢ Prices may vary depending on project complexity.</p>
    <p>Ⓢ Delivery times may vary according to complexity and workload.</p>
  </div>

  <link href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400;700&display=swap"
rel="stylesheet">

<?php endif;

```

The final block explains content restrictions and clarifies that prices and turnaround times may vary. Immediately afterward, Orbitron is loaded from Google Fonts. That dependency is visual: if the external font is unavailable, the content still exists, but the browser will use a fallback typeface.

32.5 NOD/GDI appearance: same structure, different CSS

The HTML structure does not change between factions. The visual difference comes from the NOD and GDI CSS, which define the same selectors with different palettes, shadows, backgrounds, and borders. The main selectors include .your-services, .your-services-card, .payment-info-panel, .info-panel, .price-box, and .section-divider.

```

same Our Services HTML
├ faction = gdi → gdi-faction-min.css
├ faction = nod → nod-faction-min.css
↓
same information, different visual identity

```

32.6 Guided example: Alex checks a service

Alex opens /our-services/ with faction=nod. WordPress reaches wp_head, the function confirms the slug, and PHP outputs the contact panel and the three cards. The NOD CSS turns that structure into the red interface shown in the screenshot. If Alex clicks the email or Discord link, he leaves for a contact channel; no item is added to the cart and no WooCommerce order is created.

Do not confuse these concepts: Our Services documents services offered by the studio, but its cards are not WooCommerce products. Shop/Bundles and Our Services are two different mechanisms.

33. JavaScript and dynamic page changes

This section no longer works as an API glossary. It follows four actual project mechanisms and shows when they act: DOMContentLoaded to work with an existing initial DOM, IntersectionObserver to activate previews as they approach the viewport, MutationObserver to react to HTML that a plugin creates again, and WooCommerce/jQuery events to continue after an AJAX operation. requestAnimationFrame() is used when the change is tied to the browser painting cycle.

Important boundary: PHP can output a <script>, but it does not execute document.addEventListener(), MutationObserver, or requestAnimationFrame(). That JavaScript travels in the HTTP response and runs later in the browser.

33.1 DOMContentLoaded: Checkout waits for the fields to exist

Actual source code: functions.php → anonymous wp_footer callback conditioned by is_checkout() → lines 8259–8271.

```

add_action('wp_footer', function() {
    if (is_checkout()) {
        ?>
        <script>
        document.addEventListener('DOMContentLoaded', function(){
            const art = document.querySelector('#artwork_name');
            if(art) art.setAttribute('maxlength', '32');

            const name = document.querySelector('#billing_first_name');
            if(name) name.setAttribute('maxlength', '50');

            const mail = document.querySelector('#billing_email');
            if(mail) mail.setAttribute('maxlength', '100');
        });
    }
});

```

The listener is registered when the browser interprets the script. The inner function runs when the initial DOM has already been built. Only then does `querySelector()` find the inputs and `setAttribute()` add visible character limits. These restrictions improve the interface, but server-side PHP validation remains the final authority.

```
PHP outputs <script>
↓ HTTP response
browser creates DOM
↓ DOMContentLoaded
querySelector() finds inputs
↓
maxLength changes the HTML in memory
```

33.2 IntersectionObserver: Shop activates videos as they approach

Actual source code: `functions.php` → `video_lazy_loop_shop()` → lines 2979–3033. The main observer is at 3015–3022 and uses `rootMargin: 200px`.

```
const observer = new IntersectionObserver((entries, obs) => {
  entries.forEach(entry => {
    if(entry.isIntersecting){
      reproducirVideo(entry.target, false); // scroll: no delay
      obs.unobserve(entry.target);
    }
  });
}, { rootMargin: "200px" });

videos.forEach(video => {
  const rect = video.getBoundingClientRect();
  const inViewport = rect.top < window.innerHeight && rect.bottom > 0;
  if(inViewport){
    reproducirVideo(video, true); // initially visible: 850 ms
  } else {
    observer.observe(video); // scroll: immediate
  }
})
```

`entries` contains the elements whose relationship with the viewport changed. When `entry.isIntersecting` is true, `entry.target` is the corresponding video. `reproducirVideo()` copies `data-src` to `src` if it does not already exist and starts playback. `obs.unobserve()` prevents that element from continuing to be observed after activation.

The 200 px `rootMargin` allows activation to occur before the video is fully visible. This lets the browser begin preparing the MP4 while Alex approaches it by scrolling.

33.3 MutationObserver: HUSKY replaces nodes after AJAX

Actual source code: `functions.php` → `agregar_boton_lupa_buscador()` → `MutationObserver` around lines 1591–1615.

```
const observer = new MutationObserver((mutations) => {
  for (const m of mutations) {
    // Check added nodes
    for (const node of m.addedNodes) {
      if (!(node instanceof HTMLElement)) continue;
      if (node.matches && node.matches('.woof_husky_txt-input')) {
        agregarLupaSiInputListoEl(node);
        return;
      }
      const found = node.querySelector && node.querySelector('.woof_husky_txt-input');
      if (found) {
        agregarLupaSiInputListoEl(found);
        return;
      }
    }
  }
  // Optional: if #btn_search was removed, try adding it again
  for (const node of m.removedNodes) {
    if (!(node instanceof HTMLElement)) continue;
    if (node.id === 'btn_search') {
      // short timeout to allow HUSKY to re-render
      setTimeout(() => {
        const input = findInput();
        if (input) agregarLupaSiInputListoEl(input);
      }, 100);
    }
  }
})
```

```
    }, 20);
  }
}
```

HUSKY can rebuild its input after filters are applied. The observer does not query the server: it observes DOM mutations that have already occurred. If `.woof_husky_txt-input` appears among the `addedNodes`, the theme runs `agregarLupaSilInputListoEl()` again. If it detects that `#btn_search` was removed, it waits briefly and looks for the new input generated by the plugin.

```
HUSKY performs AJAX
↓
replaces part of the DOM
↓
MutationObserver receives mutations
↓
finds the new input
↓
MC Artworks adds the magnifying glass again
```

33.4 requestAnimationFrame(): Gallery synchronizes parallax with painting

Actual source code: `functions.php` → `cnc_gallery_parallax()` → Parallax block, approximately lines 8662–8695.

```
const maxX = 18, maxY = 12, ease = 0.12;
let tX=0,tY=0,cX=0,cY=0,raf=null,inSide=false;

const onMove = e => {
  const r = videoWrap.getBoundingClientRect();
  tX = -(((e.clientX-r.left)/r.width)-0.5)*maxX;
  tY = -(((e.clientY-r.top)/r.height)-0.5)*maxY;
};

function step(){
  cX += (tX-cX)*ease;
  cY += (tY-cY)*ease;
  main.style.transform = `translate3d(calc(-50% + ${cX}px), calc(-50% + ${cY}px),0)`;
  if(!inSide && Math.abs(cX)<0.25 && Math.abs(cY)<0.25){
    main.style.transform='translate3d(-50%,-50%,0)';
    cancelAnimationFrame(raf);
    raf=null;
    cX=0;cY=0;
    return;
  }
  raf=requestAnimationFrame(step);
}

videoWrap.addEventListener('mouseenter',()=>{
  inSide=true;
  videoWrap.addEventListener('mousemove',onMove);
  if(!raf) step();
});

videoWrap.addEventListener('mouseleave',()=>{
  inSide=false;
  videoWrap.removeEventListener('mousemove',onMove);
  tX=0;tY=0;
});
```

`onMove()` does not move the video directly: it calculates a target position `tX/tY`. `step()` moves `cX/cY` toward that target using `ease` and applies `translate3d()`. At the end, it requests the next cycle with `requestAnimationFrame(step)`. When the cursor leaves and the position returns almost to zero, `cancelAnimationFrame()` stops the cycle.

33.5 added_to_cart event: continuing after WooCommerce AJAX

Actual source code: `functions.php` → Shop synchronization script → `added_to_cart` listener around lines 3664–3685.

```
$(document.body).on('added_to_cart', function(event, fragments, cart_hash, $button) {
  if (!$button || !$button.length) return;

  const pid = $button.data('product_id');
  if (!pid) return;
```

```

// Hide the 'Add to Cart' button that was just used
$button.hide();

// Remove duplicate 'View Cart' instances
$button.siblings('a.added_to_cart.wc-forward, a.button.wc-forward').remove();

// Insert a single clean 'View Cart' button
const verCarrito = $('<a>', {
  href: wc_cart_url,
  class: 'added_to_cart wc-forward',
  'data-product_id': pid,
  text: 'Ver carrito'
});

$button.after(verCarrito);
});

```

added_to_cart is not a native DOM event: WooCommerce triggers it after adding the product through AJAX. The callback receives the button that originated the operation, obtains product_id, hides Add to Cart, removes duplicate buttons, and creates a View Cart link. In this way, the theme reacts after WooCommerce has already confirmed the cart operation.

Related finding: in the active code, wc_cart_url is localized as a JavaScript object with a url property. There is also an earlier block that would have localized it as a string, but it is commented out and does not run. The actual problem is that some later routines use wc_cart_url directly as href instead of wc_cart_url.url. The inconsistency is consolidated in section 39.

33.6 When each mechanism is used in this project

Mechanism	What it waits for	Actual MC Artworks case
DOMContentLoaded	Initial DOM created	Checkout applies maxlength to fields
IntersectionObserver	visibility/proximity change	Shop activates previews from data-src
MutationObserver	change in DOM structure	HUSKY recreates the search field after AJAX
added_to_cart	end of a WooCommerce AJAX operation	Shop changes Add to Cart to View Cart
requestAnimationFrame	next painting cycle	Gallery interpolates parallax

33.7 Guided example: Alex passes through several mechanisms

```

Alex opens Shop
↓ DOMContentLoaded
logic is initialized
↓ scroll
IntersectionObserver activates a preview
↓ applies a HUSKY filter
AJAX replaces the DOM
↓ MutationObserver restores the magnifying glass
↓ Add to Cart
WooCommerce triggers added_to_cart
↓
View Cart is displayed
↓ opens Gallery
requestAnimationFrame animates the parallax

```

34. Browser adaptation

Multimedia compatibility is not resolved through a single branch. Intro detects Firefox and Chromium browsers, chooses different HLS paths, and configures HLS.js differently. Shop, on the other hand, contains a Chrome/Opera-specific fix that takes video snapshots before navigation to avoid black frames.

34.1 Detection used by Intro

Actual source code: wp-content/themes/blocks-child/hud-intro.php → HLS synchronization script around line 1032. The source is minified on a single line; it is shown here reindented without changing its logic.

```
const isFirefox = typeof InstallTrigger !== 'undefined';
const isChromium = !!window.chrome;

const videos = isFirefox
  ? {
    GDIvideo: '/videos/gdi-intro-firefox/GDI-INTRO.m3u8',
    NODvideo: '/videos/nod-intro-firefox/NOD-INTRO.m3u8'
  }
  : {
    GDIvideo: '/wp-content/uploads/videos/GDI-INTRO/GDI-INTRO.m3u8',
    NODvideo: '/wp-content/uploads/videos/NOD-INTRO/NOD-INTRO.m3u8'
  };
```

Browser detection first decides which paths to use. Firefox points to /videos/...-firefox/, while the rest of that branch uses paths inside wp-content/uploads/videos. This is browser detection; later, the code also performs capability detection through `Hls.isSupported()` and `canPlayType()`.

34.2 HLS.js versus native HLS support

```
if (window.Hls && Hls.isSupported()) {
  const hls = new Hls(/* browser-specific configuration */);
  hls.loadSource(src);
  hls.attachMedia(vid);
  hls.on(Hls.Events.MANIFEST_PARSED, () => resolve());
} else if (vid.canPlayType('application/vnd.apple.mpegurl')) {
  vid.src = src;
  vid.addEventListener('canplaythrough', ready);
} else {
  resolve();
}
```

`Hls.isSupported()` asks whether HLS.js can work in that browser. If it can, `loadSource()` loads the .m3u8 manifest and `attachMedia()` connects it to the `<video>` element. If HLS.js is not the available path but the video element reports native support for `application/vnd.apple.mpegurl`, the manifest is assigned directly to `src`.

34.3 Different configurations for Firefox and Chromium

Branch	Verified HLS.js configuration	Observable objective
Firefox	<code>enableWorker=true · lowLatencyMode=true · backBufferLength=60</code>	Specific path with a larger back buffer
Chromium	<code>enableWorker=false · lowLatencyMode=false · backBufferLength=1 · maxBufferLength=10 · maxMaxBufferLength=20 · timeouts and retries</code>	More constrained path with explicit timeouts

After loading both feeds, the script waits for `Promise.all()`, moves both videos to time 0, pauses them, waits for a `requestAnimationFrame()`, and starts playing them again. A `setInterval` compares their `currentTime` values and corrects differences greater than 0.05 seconds. The interval is 1500 ms for Chromium and 2000 ms for the other branch.

34.4 autoplay, muted, and playsinline: one explanation

Interface videos often combine `autoplay`, `muted`, and `playsinline`. `muted` reduces the restrictions many browsers apply to autoplay with sound; `playsinline` requests that the resource remain embedded in the page instead of forcing a fullscreen player. These flags alone do not guarantee that `play()` will succeed: the code uses `.play().catch(() => {})` in several places because the browser may reject playback.

34.5 Chrome/Opera-specific fix in Shop

Actual source code: functions.php → fix_videos_chrome_shop_universal() → wp_footer hook at 3039; detection and general snapshot logic from 3054.

```
function fix_videos_chrome_shop_universal() {
    if (!is_shop()) return;
    ?>
    <script>
    (function(){
        const ua = navigator.userAgent;
        const isChrome = /Chrome/.test(ua) && /Google Inc/.test(navigator.vendor);
        const isOpera = /OPR\//.test(ua);
        if (!(isChrome || isOpera)) return;

        // === General snapshot (for the main gallery) ===
        function snapshotVideo(video){
            try {
                // Ignore main-menu videos
                if (video.classList.contains('zaj-card-video')) return;

                const w = video.offsetWidth || video.clientWidth;
                const h = video.offsetHeight || video.clientHeight;
                const canvas = document.createElement('canvas');
                canvas.width = video.videoWidth || w;
                canvas.height = video.videoHeight || h;
                canvas.style.width = w + 'px';
                canvas.style.height = h + 'px';
                canvas.style.position = 'absolute';
                canvas.style.top = video.offsetTop + 'px';
                canvas.style.left = video.offsetLeft + 'px';
                canvas.style.zIndex = '9999';
                canvas.style.pointerEvents = 'none';
                const ctx = canvas.getContext('2d');
                ctx.drawImage(video, 0, 0, canvas.width, canvas.height);
                const parent = video.parentElement;
                if (getComputedStyle(parent).position === 'static') {
                    parent.style.position = 'relative';
                }
                parent.insertBefore(canvas, video);
                return true;
            }
        }
    })();
}
```

The function continues only in Shop and only for Chrome or Opera according to userAgent/vendor. snapshotVideo() creates a Canvas matching the video size, draws the current frame through drawImage(), and places that Canvas in front of the player. If the capture fails, it attempts to use the poster as a fallback image.

34.6 When the snapshot is taken

```
// [1] Internal links (pagination, View Cart, etc.)
document.addEventListener('click', function(e){
    const link = e.target.closest && e.target.closest('a[href]');
    if (!link) return;

    // Ignore links that open videos with Fancybox or clicks inside <video>
    if (link.hasAttribute('data-fancybox') || e.target.closest('.video-loop-link') ||
e.target.tagName === 'VIDEO') return;

    // Ignore "Add to Cart" or "View Cart" links
    if (link.classList.contains('added_to_cart') || link.classList.contains('add_to_cart_button'))
return;

    if (link.target === '_blank') return;

    const href = link.getAttribute('href');
    if (!href || href.startsWith('#')) return;

    e.preventDefault();
});
```

```
// Take a snapshot of everything
snapshotAllVideos();
snapshotCartVideos();

    setTimeout(() => window.location.href = href, 80);
}, true);

// [2] Forms (HUSKY / WOOF / AJAX)
document.addEventListener('submit', function(){
    snapshotAllVideos();
```

The code intercepts internal links and form submissions. Before navigation, `snapshotAllVideos()` and `snapshotCartVideos()` visually freeze the players and navigation is delayed by 80 ms. The goal is to avoid the black frame that can appear when the browser tears down the video before the next page is displayed.

34.7 Limits of this strategy

- Detection through `userAgent/vendor` may require maintenance if browsers change.
- Canvas depends on the frame being drawable; the code provides the poster as a fallback in `snapshotVideo()`.
- HLS.js is loaded from a CDN with `@latest` in different project blocks, so the effective version can change without modifying the child theme.
- The intended result is an equivalent experience, not execution of exactly the same code in every browser.

34.8 Guided example: Alex opens Intro in two browsers

```
Alex opens Intro
├─ Firefox
│   └─ /videos/*-firefox/ paths
│       └─ Hls config: worker + low latency + backBuffer 60
└─ Chromium
    └─ /wp-content/uploads/videos/... paths
        └─ Hls config: more constrained buffers/timeouts
↓
Hls.isSupported() or native HLS
↓
GDI + NOD are loaded
↓
Promise.all()
↓
playback + synchronization
```

35. Multimedia resources and external services

This section follows actual resources from their source to the browser. The rule is always the same: identify where the data or file lives, who builds the URL, when the browser requests it, and which external dependency is involved.

35.1 Case A: video_destacado for a Shop product

Actual source code: `functions.php` → Shop audiovisual card → lines 2902–2924.

```
$faction = $_COOKIE['faction'] ?? 'gdi';
$uploads_dir = wp_upload_dir();

if ($faction == 'gdi')
    $poster_path = trailingslashit($uploads_dir['baseurl']) . 'images/irina_poster.jpg';
else
    $poster_path = trailingslashit($uploads_dir['baseurl']) . 'images/poster.jpg';

$video_url = get_field('video_destacado', $product->get_id());
if (!$video_url) return;

?>

<a href="<?php echo esc_url($video_url); ?>"
    data-fancybox="videos-tienda"
    class="video-loop-link video-loop-wrapper">
    <div class="video-container" style="position:relative;">
```

```
<video
  width="630"
  height="870"
  poster="<?php echo esc_url($poster_path); ?>"
  data-src="<?php echo esc_url($video_url); ?>"
  preload="metadata"
```

`get_field('video_destacado', $product->get_id())` obtains the URL associated with the product from ACF. PHP does not immediately trigger the download as `src`: it writes it to `data-src`. Later, the `IntersectionObserver` from section 33 copies that value to `src`. The poster is built from `wp_upload_dir()['baseurl']` and escaped with `esc_url()`.

```
WC_Product
↓ product_id
ACF video_destacado
↓ URL
HTML data-src
↓ IntersectionObserver
src
↓ HTTP request
MP4
```

35.2 Case B: Gallery public URL versus physical file

Gallery does not use Media Library to discover its files: `$videos` contains URLs such as `/videos/nod-gallery/video2_main.mp4`. The physical review confirms files inside `wp-content/uploads/videos/NOD-GALLERY/`. The child theme does not contain the rule that connects both paths, so that mapping belongs to the server and must be preserved during migration.

Concept	Example
URL received by the browser	<code>/videos/nod-gallery/video2_main.mp4</code>
Confirmed physical location	<code>wp-content/uploads/videos/NOD-GALLERY/video2_main.mp4</code>
Responsible for the mapping	Server configuration outside the child theme

Do not confuse these concepts: a public URL does not by itself prove the physical path of the file. They are two different layers.

35.3 Case C: email logo built with `wp_upload_dir()`

Actual source code: `functions.php` → `WC_Gateway_Manual_Pay::process_payment()` → lines 7423–7425.

```
// Path to the logo
$uploads_dir = wp_upload_dir();
$logo_url = trailingslashit($uploads_dir['baseurl']) . 'images/logo.png';
```

`wp_upload_dir()` returns information about the uploads directory. The code uses `baseurl`, not the physical disk path, because the email HTML needs a URL that the mail client can request.

35.4 Case D: Fancybox and HLS.js arrive from jsDelivr

Actual source code: `functions.php` → `cargar_fancybox_tienda()` → lines 30–43. HLS.js is also loaded from jsDelivr with `@latest` in `functions.php` and `hud-intro.php`.

```
add_action('wp_enqueue_scripts', 'cargar_fancybox_tienda');

// =====
// Function that loads the libraries required to initialize Fancybox.
// =====
function cargar_fancybox_tienda() {

    // Load jQuery (already included with WordPress)
    wp_enqueue_script('jquery');

    if (is_shop() || is_product_category() || is_product_tag()) {
        wp_enqueue_style('fancybox-css', 'https://cdn.jsdelivr.net/npm/@fancyapps/ui/dist/fancybox.css', [],
            '4.0', 'all');
        wp_enqueue_script('fancybox-js', 'https://cdn.jsdelivr.net/npm/@fancyapps/ui/dist/fancybox.umd.js',
```

```
[], '4.0', true);
}
```

Fancybox is loaded from jsDelivr through URLs that do not explicitly pin the package version in the path itself. The value 4.0 passed to `wp_enqueue_style()/wp_enqueue_script()` acts as the resource version for WordPress and may appear as a query parameter, but it is not equivalent to requesting `@fancyapps/ui@4.0` from jsDelivr. HLS.js, by contrast, explicitly appears as `hls.js@latest`. In both cases, pinning the package version in the CDN URL would improve deployment reproducibility.

35.5 Case E: HLS is not a single MP4 file

A .m3u8 manifest describes a stream composed of segments. HLS.js or native support reads that manifest and requests segments according to playback and buffering. Therefore, the correct flow is manifest → segments → buffer → video, not “download a complete MP4.”

35.6 Music and sounds

The `wp-content/uploads/songs` and `wp-content/uploads/sounds` folders contain resources used by Intro, the menu, and the faction selector. As with videos, PHP/HTML outputs URLs and the browser creates the audio objects. Their existence in uploads is independent of cookies or the WooCommerce session.

35.7 Final resource matrix

Resource	Source of truth / origin	Who builds or exposes the URL	Consumer
video_destacado	ACF + attachment/uploads	<code>get_field()</code> / WordPress	Shop/Bundles
Gallery MP4	<code>\$videos</code> array + physical files	<code>functions.php</code> + server mapping	Gallery
Email logo	<code>uploads/images/logo.png</code>	<code>wp_upload_dir()</code>	mail client
Intro HLS	.m3u8 manifests	<code>hud-intro.php</code>	HLS.js / native video
Fancybox	jsDelivr	<code>wp_enqueue_script/style</code>	browser
SMTP	WordPress/plugin configuration	<code>wp_mail()</code> → WP Mail SMTP	mail server

36. Cache and performance

Different mechanisms coexist in the project: HTTP caching, CSS versioning, lazy loading, HLS buffering, WooCommerce sessions, and JavaScript logical caches. This section preserves that distinction and also analyzes the performance costs arising from the current implementation.

36.1 CSS versioning: `filemtime()` and `time()`

Actual source code: `functions.php` → two `wp_enqueue_scripts` callbacks with priority 999 → lines 92–185.

```
if ($faction === 'gdi') {
    $css_file = get_stylesheet_directory() . '/gdi-faction-min.css';
    $version = file_exists($css_file) ? filemtime($css_file) : '1.0';

    wp_enqueue_style(
        'gdi-faction',
        get_stylesheet_directory_uri() . '/gdi-faction-min.css',
        [],
        $version,
        'all'
    );
} elseif ($faction === 'nod') {
    $css_file = get_stylesheet_directory() . '/nod-faction-min.css';
    $version = file_exists($css_file) ? filemtime($css_file) : '1.0';

    wp_enqueue_style(
```

```

        'nod-faction',
        get_stylesheet_directory_uri() . '/nod-faction-min.css',
        [],
        $version,
        'all'
    );
}
}, 999);
add_action('wp_enqueue_scripts', function() {
    // Detect faction from the cookie
    $faction = $_COOKIE['faction'] ?? '';

    // If the cookie does not exist or is empty, do not load any CSS
    if (empty($faction)) return;

    // Theme directories
    $theme_dir = get_stylesheet_directory();
    $theme_uri = get_stylesheet_directory_uri();

    // Generate a dynamic parameter based on the current time
    // This forces the file to reload on every refresh
    $timestamp = time();

    // Load the CSS corresponding to the faction
    if ($faction === 'gdi') {
        $url = $theme_uri . '/gdi-faction-min.css?v=' . $timestamp;
        wp_enqueue_style('gdi-faction', $url, [], null, 'all');
    } elseif ($faction === 'nod') {
        $url = $theme_uri . '/nod-faction-min.css?v=' . $timestamp;
        wp_enqueue_style('nod-faction', $url, [], null, 'all');
    }
}, 999);

```

The first block uses `filemtime()`: as long as the file does not change, the version can remain stable. The second generates `time()` on every request and builds a different URL. Both callbacks are active and use the same `gdi-faction/nod-faction` handles. It should not be assumed that the second one cleanly “replaces” the first; their coexistence is a redundant strategy that should be unified.

36.2 Shop lazy loading as an optimization

The cards are output with `data-src` and the video is activated near the viewport. This avoids starting all Shop MP4 files during the initial load. The observer uses `rootMargin=200px`, so it can preload a resource shortly before it enters the screen.

36.3 Gallery: 12 main videos with `preload=auto`

Actual source code: `functions.php` → `cnc_gallery_parallax()` → generation of each `vid-main` at lines 8494–8498.

```

        <?php foreach($videos as $v): ?>
        <div class="frame-container" data-alt='<?= json_encode($v['alt']) ?>' data-sound='<?=
        $v['sound'] ? 1 : 0 ?>'>
                <div class="video-wrap">
                        <video class="vid-main" src="<?= $v['main'] ?>" autoplay muted loop
        playsinline preload="auto"></video>
                </div>
        </div>
        </?>
    </foreach>

```

Gallery generates 12 `vid-main` elements when the page opens, and all include `preload="auto"`. The alt variants are created on demand, which reduces their initial cost, but the main videos can begin requesting data from the start. This contrasts with Shop, where `data-src` defers activation.

36.4 Bundles: complete query and filtering in PHP

Actual source code: `functions.php` → Bundles block → `wc_get_products()` at 4504–4510 and manual filtering from 4517.

```

    $all_bundles = wc_get_products([
        'status'      => 'publish',
        'limit'       => -1,
    ]);

```

```

        'orderby'    => 'title',
        'order'      => 'ASC',
        'tax_query'  => $tax_query,
    ]));

    // === PAGINATION ===
    $paged = max(1, absint(get_query_var('paged') ?: get_query_var('page') ?: (isset($_GET['paged']) ?
intval($_GET['paged']) : 1)));
    $per_page = 12;
    $offset = ($paged - 1) * $per_page;

    // === MANUAL FILTERING ===
    $filtered_products = [];
    foreach ($all_bundles as $bundle) {
        $price = floatval($bundle->get_price());
        $name = $bundle->get_name();
        if ($price >= $min_price && $price <= $max_price && ($q === '' || stripos($name, $q) !== false)) {
            $filtered_products[] = $bundle;
        }
    }

```

limit=-1 requests all bundles compatible with tax_query, and then the code iterates over the result to filter by price/text and paginate. With the current catalog it may work, but the cost grows with the number of bundles because pagination occurs after the complete set has been retrieved.

36.5 HLS: buffering versus full download

Intro/Home work with manifests and segments. Consumption depends on the buffer and playback. The Chromium configuration limits maxBufferLength/maxMaxBufferLength, while Firefox uses a different backBufferLength. This is a dynamic cost, not a monolithic download.

36.6 What is already optimized

- Shop defers previews through data-src + IntersectionObserver.
- Gallery creates alternative variants only when the user requests them and then reuses them through altCache.
- Minified CSS stylesheets are used for factions during normal loading.
- HLS makes it possible to work by segments instead of requiring a complete MP4 for Intro/Home.

36.7 Verified costs and review points

Point	Current behavior	Possible improvement
Faction CSS	filemtime() and time() active with the same handle	Keep one strategy according to the environment
Gallery	12 main videos with preload=auto	Review preload/initial activation
Bundles	wc_get_products(limit=-1) + manual filtering	Move more filtering/pagination into the query
HLS.js	CDN with @latest	Pin a known version
Shop	lazy loading through observer	Keep; it is an effective optimization
Fancybox	jsDelivr URL without a pinned package version; ver=4.0 is WordPress versioning, not a package pin	Pin a known version in the CDN URL

Do not confuse performance with caching: reducing initial downloads, avoiding complete queries, and controlling buffers are different problems from deciding whether an HTTP response can be reused.

37. Data handling and security

Security is analyzed by following actual project inputs. Validation, sanitization, and escaping are different stages: validation decides whether a value meets a rule; sanitization normalizes or cleans it before use; escaping prepares it for an output context such as HTML or a URL.

37.1 Case A: Artworks Name from Checkout to WC_Order

Actual source code: functions.php → woocommerce_checkout_create_order callback → lines 8297–8301.

```
add_action('woocommerce_checkout_create_order', function($order, $data) {
    if ( isset($_POST['artwork_name']) && $_POST['artwork_name'] !== '' ) {
        $order->update_meta_data('Artworks Name', sanitize_text_field($_POST['artwork_name'])); // Translated
    }
}, 10, 2);
```

```
input artwork_name
↓ POST
sanitize_text_field()
↓
$order->update_meta_data()
↓ WC_Order
$order->get_meta()
↓
esc_html() when inserted into HTML
```

Sanitization does not replace Checkout length validation. Nor does it replace later escaping when the same value is output inside the email or on a page.

37.2 Case B: additional email validation

Actual source code: functions.php → woocommerce_checkout_process callbacks around 8195–8256.

```
add_action('woocommerce_checkout_process', function() {
    $email = '';

    // Capture the Checkout email
    if (!empty($_POST['billing_email'])) {
        $email = sanitize_text_field($_POST['billing_email']);
    } elseif (!empty($_POST['billing']['email'])) {
        $email = sanitize_text_field($_POST['billing']['email']);
    }

    // 1) Maximum length
    if (!empty($email) && strlen($email) > 100) {
        wc_add_notice('Email cannot exceed 100 characters.', 'error'); // Translated
        return;
    }

    // 2) Basic format validation with filter_var
    if (empty($email) || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
```

The code cleans the value, validates its format, and then separates the domain. Later, it attempts to verify DNS records through MX/A resolution and a host fallback. That test can detect some domain errors, but it does not prove that the individual mailbox exists or will accept mail.

37.3 Case C: public AJAX endpoint obtener_id_desde_slug

Actual source code: functions.php → wp_ajax_obtener_id_desde_slug / wp_ajax_nopriv_obtener_id_desde_slug → lines 3693–3725.

```
add_action('wp_ajax_obtener_id_desde_slug', 'obtener_id_desde_slug_callback');
add_action('wp_ajax_nopriv_obtener_id_desde_slug', 'obtener_id_desde_slug_callback');

/* =====
 * Function that responds to the AJAX request to obtain a product ID
 * from its slug (the final part of the URL).
 * This function is registered for both logged-in and logged-out users.
 * Process:
 * - Verifies that the 'slug' parameter was sent via POST.
 * - Sanitizes the received slug to prevent injection or invalid characters.
 * - Looks up the product in the database using the slug.
 * - If the product is found, returns JSON containing the ID.
 * - If the product is not found or the slug is missing, execution ends without a response.
 * ===== */
function obtener_id_desde_slug_callback() {
    // Verify that the 'slug' parameter was received via POST
    if (!isset($_POST['slug'])) {
```

```

    wp_die(); // End without a response if slug is missing
}

// Clean and sanitize the received value to prevent injections or invalid characters
$slug = sanitize_title($_POST['slug']);

// Look up the product in the database using its slug (human-readable URL name)
$producto = get_page_by_path($slug, OBJECT, 'product');

if ($producto) {
    // If the product exists, return success and an array containing the product ID
    wp_send_json_success(['id' => $producto->ID]);
} else {
    wp_die(); // End without a response if the product is not found
}
}

```

The endpoint is available to authenticated and unauthenticated users. It checks that slug exists, applies `sanitize_title()`, searches for a product through `get_page_by_path()`, and, if found, returns only its ID with `wp_send_json_success()`.

Control not present in the current callback: this code does not contain `check_ajax_referer()` or a capability check. Its scope must be interpreted precisely: it is a read endpoint that returns a product ID; the absence of a nonce does not by itself imply a critical vulnerability or modification capability.

37.4 Case D: faction cookie

Actual source code: `faction-selection.php` → when the faction is confirmed, `document.cookie` is written; `functions.php` and `header.php` read `$_COOKIE['faction']`.

```
document.cookie = "faction=" + $faction.dataset.faction + "; path=/; max-age=" + (60 * 60 * 24 * 365);
```

The cookie is used to decide appearance and resources. In many PHP locations, `gdi` is applied as the default value, but not every reader explicitly validates that the content is only `gdi` or `nod`. A cookie is browser-controllable input; conceptually, it should not be treated as a credential or as proof of identity.

37.5 Output escaping

Function	Verified use	Purpose
<code>esc_html()</code>	name, email, metadata in HTML/email	avoid interpreting text as markup
<code>esc_url()</code>	videos, posters, logo, links	prepare a URL for HTML output
<code>sanitize_text_field()</code>	Artworks Name / POST values	clean input text
<code>sanitize_title()</code>	AJAX slug	normalize a slug
<code>wc_clean()</code>	WooCommerce data	normalize values from the WooCommerce domain

37.6 Logs: write permission is not the same as privacy

The logs contain name, email, products, customizations, and total. PHP needs write permission on `wp-content/uploads/logs`, but that permission does not prove the directory is blocked from HTTP requests. Protection depends on server configuration or a different logging strategy. `get_log_path()` also does not create the folder if it is missing.

Do not confuse these concepts: “PHP can write” and “a visitor cannot download the file” are different properties.

37.7 Visual restrictions versus actual security

The child theme blocks selection, dragging, or `contextmenu` on different screens. Those choices may make accidental actions more difficult, but they are not DRM and do not prevent inspection of resources the browser has already downloaded. Actual security depends on validation, permissions, server configuration, access controls, and correct input/output handling.

37.8 AIOS and the scope of available information

All-In-One Security is part of the reviewed environment, but the specific configuration of its modules is not defined in the child theme code. Therefore, this documentation does not attribute specific rules to it unless they are demonstrated by the available information.

37.9 Main input matrix

Input	Origin	Verified handling	Persistence/output
Artworks Name	Checkout POST	validation + sanitize_text_field	WC_Order meta → email/log
Email	Checkout POST	format + DNS + limits	billing data → wp_mail/log
Frame Color / Balloon	Cart	flow cleanup/validation	WC_Order_Item meta
slug AJAX	public POST	sanitize_title	JSON product ID
faction	browser cookie	default and comparisons	CSS/resource selection

38. Project installation, migration, and verification

38.1 What the project includes and what must be reconstructed

Element	Is it inside blocksy-child?	What to do during migration
Child-theme PHP/CSS code	Yes	Copy to wp-content/themes/blocksy-child and activate together with Blocksy.
WordPress Core	No	Install/configure WordPress.
Blocksy	No	Install the compatible parent theme.
WooCommerce/ACF/HUSKY/Xootix/WP Mail SMTP	No	Install and configure each plugin.
Database	Not part of the child theme	Restore/import the corresponding installation.
wp-content/uploads	Not part of the child theme	Restore images, MP4, audio, HLS, and required logs/directories.
/videos/ configuration	Not confirmed inside the child theme	Recreate an equivalent rule or mapping on the server.

The installation must verify each layer before continuing. Copying only the child theme does not reproduce the site, because products, ACF, pages, plugins, uploads, and server rules all participate in the actual behavior.

38.2 What must be reconstructed outside the child theme

The technical documentation is reproducible only if code, database, uploads, and plugin/server configuration are distinguished.

38.3 Layers that must be available

Component	Location/ origin	Responsibility
WordPress + PHP + web server	environment	runs the platform
Blocksy	parent theme	child-theme requirement
blocksy-child	custom code	must be copied and activated
Database	WordPress/WooCommerce site	products, orders, options
wp-content/uploads	physical files	media, sounds, videos, logs

Component	Location/ origin	Responsibility
Plugins	WordPress Admin	WooCommerce, ACF, HUSKY, Xootix, WP Mail SMTP, etc.
Server rules	web server	public paths such as /videos/...

Do not confuse these concepts

Copying only the child theme does not reproduce the complete site.

The objective of each step is to leave a verifiable condition before continuing.

38.4 Assembly and verification order

```
install WordPress and verify admin access
↓
install/activate Blocksy
↓
copy blocksy-child to wp-content/themes
↓
activate child theme
↓
install WooCommerce, ACF, HUSKY/WOOF, Xootix, and WP Mail SMTP
↓
restore database and uploads
↓
create/verify pages with required slugs
↓
verify ACF fields
↓
verify catalog/cart/checkout
```

Do not confuse these concepts

The exact HUSKY/Xootix configuration is not fully expressed in the PHP code; it must also be verified in WordPress Admin.

The code checks specific pages with `is_page()` and expects resources at certain paths. A migration must preserve those expectations.

38.5 Dependencies external to the child theme

Component	Location/ origin	Responsibility
/intro/	WordPress Page	activates hud-intro
/faction-selection/	WordPress Page	activates NOD/GDI selector
/bundles/	WordPress Page	activates Bundles content
/gallery/	WordPress Page	activates Gallery
/our-services/	WordPress Page	activates Services
ACF	field group	video_destacado, productos_del_bundle, tiene_globo, video_workshop
uploads	file system	media and sounds

Do not confuse these concepts

The code also checks `es_bundle`, but the earlier ACF review did not confirm that field within the reviewed configuration; it must be verified in the actual installation.

MC Artworks Studio runs on a WordPress installation with the Blocksy parent theme, the blocksy-child child theme, WooCommerce, and the plugins used by the project. To reproduce the site correctly, the database, the wp-content/uploads structure, and the associated multimedia resources must also be available.

38.6 Recommended installation order

1. Install WordPress in an environment compatible with WooCommerce.

2. Install and activate the Blocksy parent theme.
3. Copy blocksy-child to wp-content/themes/ and activate the child theme.
4. Install WooCommerce.
5. Install Advanced Custom Fields and recreate or import the video_destacado, video_workshop, productos_del_bundle, and tiene_globo fields with their location rules.
6. Install HUSKY/WOOF and Xootix Side Cart and verify their configuration according to the integration documented in the child theme.
7. Configure the intro, faction-selection, bundles, gallery, our-services, cart, and checkout pages according to the routes used by the code.
8. Restore wp-content/uploads, including categories, images, videos, songs, sounds, and the files managed by Media Library, together with any HLS resources used by the theme.
9. Create or verify wp-content/uploads/logs with write permissions and appropriate protection.
10. Configure email delivery through WP Mail SMTP or the selected SMTP service; the theme requests sends through wp_mail().
11. Create or import WooCommerce products and categories. Bundles are simple products in the Bundles category and use productos_del_bundle to relate the artworks.
12. Configure the domain and paths for the environment where the project is deployed.

Layer-by-layer verification. After WordPress + Blocksy + blocksy-child, the home page should load without PHP errors. After WooCommerce, Shop/Cart/Checkout should exist. After ACF, a test product should return its fields. After restoring uploads, posters and videos should resolve over HTTP. After SMTP, a test message should be sent before validating manual_pay.

If a layer fails, it is best to stop the installation there: fixing a path, permission, or plugin problem after loading the entire system makes it more difficult to identify the cause. Sections 38.14 and 38.15 turn these checks into a functional test and symptom-based diagnosis.

Auxiliary security plugin

AIOS is part of the reviewed environment as an auxiliary security plugin. It is not a functional requirement of the Shop → Cart → Checkout → Order Received flow, and its configuration is managed outside the child theme.

Plugin configuration

The documentation describes the integration expected by blocksy-child, but it does not have a complete export of the HUSKY/WOOF and Xootix Side Cart administrative configuration. When migrating the project, those settings must be verified in the original installation before its behavior can be considered reproduced.

Content and resources

The database stores products, configuration, sessions, and orders, while wp-content/uploads contains multimedia files and logs. Both components must be available to reproduce the complete site.

Environment configuration

When deploying the project on another domain, URLs and paths that depend on the environment must be adjusted. WordPress provides functions such as home_url(), get_permalink(), and wp_upload_dir() to obtain these addresses from the site configuration, avoiding hard-coding them in different places in the code.

38.7 Pages and slugs expected by the code

```
/intro/
/faction-selection/
/bundles/
/gallery/
/our-services/
/cart/
/checkout/
```

Several functions use is_page() or specific URLs. If a slug is changed without modifying the corresponding code, that logic may stop running.

38.8 Minimum ACF configuration

- `video_destacado`: primary product resource used in Shop and Bundles.
- `video_workshop`: optional Workshop resource.
- `productos_del_bundle`: relationship between the bundle and the artworks it represents.
- `tiene_globo`: determines whether the product supports Comic Balloon Text.

`functions.php` also checks `es_bundle` inside `wpo_filtrar_meta_en_pedido()`. The code expects that field, but the current ACF review does not include it among the four confirmed fields. If `es_bundle` is not configured, `get_field()` returns an empty value and that function does not recognize bundles through that route; filtering then relies on `tiene_globo`.

38.9 WooCommerce

To reproduce the current environment, Shop, Cart, and Checkout must exist, the currency must be USD, and HPOS must be available if the same order-storage model is to be preserved. The category with the bundles slug is especially important because Shop excludes it through `pre_get_posts`.

38.10 Multimedia resources

Copying blocksy-child is not enough. `wp-content/uploads` must also be restored with images, videos, songs, sounds, attachments, and HLS streams. In Gallery, for example, the NOD files are physical MP4 files inside `uploads/videos/NOD-GALLERY/`.

38.11 Logs and email

`wp-content/uploads/logs` must exist with permissions that allow PHP to write. WP Mail SMTP or the selected SMTP service must be configured and tested before deployment is considered complete, because `manual_pay` depends on email to deliver payment instructions.

38.12 How to verify the site layer by layer

Two critical dependencies cannot be validated merely by seeing that WordPress loads: email and multimedia paths.

38.13 From a simple test to the complete flow

```
configure WP Mail SMTP and send a test
↓
verify that the gateway's wp_mail() travels through the configured transport
↓
verify that /videos/... URLs respond on the server
↓
open a first visit without a faction cookie
↓
choose a faction
↓
open Shop and play previews
↓
add to Cart and customize
↓
complete Checkout
↓
confirm WC_Order + log + emails + Order Received
```

Do not confuse these concepts

A site can “open” and still be incomplete if SMTP, HLS, `/videos/` paths, or plugin configurations fail.

Installing files does not prove that the environment is complete. The final verification must walk through the system layer by layer to quickly locate which component fails.

38.14 Functional test after installation

1. Select GDI or NOD and verify that the choice is preserved.
2. Open Home and verify that it loads the content corresponding to the selected faction.
3. Enter Shop and verify that the video previews play correctly.

4. Use ADD TO CART and verify that WooCommerce adds the product and Xootix updates the side cart.
5. Open Cart, choose Frame Color and, when applicable, complete Comic Balloon Text; then update the cart.
6. Continue to Checkout and verify validation of Artworks Name, First Name, and Email.
7. Confirm Pay with PayPal and verify that WooCommerce creates the WC_Order and retains its data and customizations.
8. Verify that the daily log file receives the order information and the results of the email attempts.
9. Verify the attempt to send to the buyer and to administration through WP Mail SMTP or the configured SMTP service.
10. Open Order Received and confirm that it displays the order information and the instructions for completing the manual payment.

Verification criterion

If any of these steps fails, it is best to review the corresponding layer before changing others: page/slug, plugin, database, physical file, JavaScript, WooCommerce session, log permissions, or SMTP configuration.

38.15 Quick diagnosis by symptom

Test/symptom	What it confirms	Check first if it fails
WordPress opens the site	Core, server, basic DB, and active theme.	WordPress configuration, server, DB connection.
Intro plays GDI/NOD	hud-intro.php, HLS/MP4, and multimedia paths.	uploads, /videos/, HLS.js, browser console.
Faction Selection preserves the choice	faction cookie + PHP reading.	faction-selection.php, browser cookies.
Shop displays products	WooCommerce + query + theme.	products, Bundles category, Shop hooks.
Preview activates when scrolling	data-src + observer + MP4 file.	video_destacado, URL, console/network.
Cart retains Blue after F5	WooCommerce session + restoration callbacks.	cart item, session, cookies.
Checkout creates order	validations + WC_Order/HPOS.	WooCommerce notices, PHP logs, Checkout configuration.
*_orders.log is created	uploads/logs permissions + gateway.	folder, permissions, error_log.
Email arrives	wp_mail + WP Mail SMTP + SMTP service.	SMTP test, plugin/server logs.

38.16 Complete test with Alex

```

first visit
-> Intro
-> choose NOD/GDI
-> Home
-> Shop
-> Cyberpunk Portrait
-> Add to Cart
-> Welcome back + Blue
-> Checkout + Raven
-> create order
-> log
-> emails
-> Order Received

```

38.17 Final environment checklist

Check	Expected state before considering the deployment valid
WordPress + Blocksy + child theme	The home page and base pages load without PHP errors
Plugins	WooCommerce, ACF, HUSKY/WOOF, Xootix, and WP Mail SMTP active/configured

Check	Expected state before considering the deployment valid
Database	Products, categories, pages, options, and orders are available
Uploads	Images, MP4, HLS, songs, and sounds respond over HTTP
/videos/	The external mapping resolves the URLs used by Intro/Gallery
ACF	video_destacado, video_workshop, productos_del_bundle, and tiene_globo available
Logs	uploads/logs exists, PHP can write, and public access is controlled
SMTP	Test message and gateway wp_mail() pass through the configured transport
Functional flow	Intro → faction → Shop → Cart → Checkout → Order Received works end to end

39. Possible development directions

The improvements in this section are derived from behaviors present in the project's current code. They do not describe functions the project already has: they form a maintenance roadmap that relates each finding to its location, technical consequence, and a possible direction for improvement.

39.1 Prioritized findings matrix

Priority	Current behavior	Location/ evidence	Possible development and objective
High	functions.php exceeds 8,800 lines and concentrates many responsibilities	functions.php	Progressively separate logic by areas such as Shop, Cart, Checkout, emails, Gallery, and Services to make reading, maintenance, and error location easier without changing the current behavior.
High	Two active callbacks version the same CSS with filemtime() and time()	functions.php 92-185	Use a single versioning strategy to prevent two different mechanisms from attempting to control the cache of the same CSS. In production, filemtime() allows the version to be renewed when the file changes.
High	Add to Cart blocking callback registered twice	functions.php: 2685 and 2690	Keep a single add_action() so that the same callback cannot run twice during wp_footer.
High	wc_cart_url is actively localized as an object {url: ...}, but some consumers use it directly as href	functions.php: active localization 3567–3574; later consumers use href: wc_cart_url	Unify the representation of wc_cart_url: use wc_cart_url.url if the object is retained, or expose a string directly. This avoids passing an object where the code expects a URL.
High	Logs containing personal data; get_log_path() does not create the logs folder	WC_Gateway_Manual_Pay	Ensure the folder exists before writing, prevent public access, and define how long records will be retained. Alternatively, use the WooCommerce logging system to centralize their management.

Priority	Current behavior	Location/ evidence	Possible development and objective
Medium	HLS.js is loaded through @latest	functions.php / hud-intro.php	Use a specific HLS.js version instead of @latest, so that a new library release cannot change Intro or Home without first being tested.
Medium	Gallery generates 12 main videos with preload=auto	cnc_gallery_parallax()	Reduce or stagger preloading of the main videos to prevent the browser from trying to prepare all twelve resources from the start when the user does not yet need them.
Medium	Gallery: no MP4 error/fallback and inconsistent sound=false behavior	cnc_gallery_parallax()	Add an error state or fallback when a video cannot load, and correct the sound logic so that a resource marked as having no sound does not end up playing audio without an available control to mute it.
Medium	Bundles retrieves limit=-1 and filters/paginates later in PHP	functions.php 4504+	Move as much filtering and pagination as possible into the query to avoid loading and iterating over all bundles when the catalog grows.
Medium	/videos/... depends on mapping external to the child theme	Intro / Gallery	Define multimedia paths in a single place and document the server rule that maps /videos/... to physical files, to make migrations to another domain or server easier.
Medium	Public slug AJAX without nonce/capability	AJAX callback 3693–3725	Determine whether the endpoint needs to verify the origin of the request through a nonce. Because it only returns a product ID, the control should match the actual risk and not add unnecessary permissions.
Medium	DNS check during Checkout	Checkout validation callbacks	Assess whether the DNS check provides enough benefit compared with the additional resolution time and possible false negatives, since a temporary DNS failure could prevent continuing with an email whose format is valid.
Low	Right-click/selection blocking is presented as UX, not security	various scripts	Keep them only if they contribute to the visual experience, making it clear that they do not prevent access to resources already downloaded by the browser or replace server security controls.
Medium	Fancybox is loaded from jsDelivr without pinning the package version in the URL	cargar_fancybox_tienda(): @fancyapps/ui/dist/...; ver=4.0 is a WordPress parameter	Specify a concrete Fancybox version in the jsDelivr URL so that the project continues using the same version until an update is deliberately tested.

39.2 Double registration of the same callback

Actual source code: functions.php → add_action('wp_footer', 'deshabilitar_boton_add_to_cart_mientras_procesa') appears on lines 2685 and 2690 before a single definition of the function.

```
add_action('wp_footer', 'deshabilitar_boton_add_to_cart_mientras_procesa');

/* =====
// Function that disables the "Add to Cart" button while the product is being processed
/* ===== */
add_action('wp_footer', 'deshabilitar_boton_add_to_cart_mientras_procesa');
function deshabilitar_boton_add_to_cart_mientras_procesa() {
```

WordPress retains every action registration. Keeping two identical add_action() calls can cause the callback to be invoked twice on the same hook. The improvement is to keep a single registration and leave the function definition separate.

39.3 wc_cart_url: localized object and consumers that expect a URL

Actual source code: functions.php → wp_localize_script around 3567–3574. The active block exposes wc_cart_url as { url: wc_get_cart_url() }; an earlier block that exposed it as a string is commented out. Later consumers use href: wc_cart_url, so they expect a string where they currently receive an object.

```
// Add the cart URL for JavaScript
/*
add_action('wp_enqueue_scripts', function() {
    wp_localize_script('jquery', 'wc_cart_url', wc_get_cart_url());
});
*/

add_action('wp_enqueue_scripts', function() {
    wp_localize_script('jquery', 'wc_cart_url', array(
        'url' => wc_get_cart_url()
    ));
});

const verCarrito = $('<a>', {
    href: wc_cart_url,
    class: 'added_to_cart wc-forward'
});
```

With the active code, wc_cart_url has the form {url: ...}. Using that object directly as href is not equivalent to using the URL contained in wc_cart_url.url. The correction is to choose a single representation: keep the object and use wc_cart_url.url in all consumers, or localize a simple string and consistently retain href: wc_cart_url.

39.4 Performance and external dependencies

The analysis of the implementations described in sections 34–36 shows three areas: Gallery can start 12 main videos with preload=auto; Bundles retrieves all results before filtering/paginating; HLS.js resolves with @latest. None of these points means the site does not work, but they do affect predictability, consumption, or scalability as the catalog and traffic grow.

39.5 Paths and environment configuration

The /videos/... URLs and some data written directly in functions.php make a migration dependent on external configuration. A reasonable development direction is to group environment paths and values in WordPress functions/configuration, retain URLs built with home_url(), get_permalink(), or wp_upload_dir() where appropriate, and explicitly document the server rule that exposes /videos/.

39.6 Operational security and logs

The priority improvement is not to “add more validation” indiscriminately. It is to align each control with its actual risk: protect logs containing personal data, ensure directory creation/retention, review the AJAX endpoint according to its use, and keep interface measures separate from server access controls.

39.7 Priority criteria

High: may cause duplicate execution, inconsistent behavior, maintenance problems, or meaningful operational exposure. Medium: affects predictability, performance, migration, or hardening. Low: improves clarity/UX without changing the commercial core.

The priority is for technical maintenance, not a claim that the site is broken. The documentation records the existing behavior so that improvements can be made without losing the NOD/GDI flow, multimedia experience, or commercial journey.

40. Technical scope and conclusion

40.1 Final responsibility map

Layer	Primary responsibility in MC Artworks Studio
WordPress Core	Receives requests, loads themes/plugins, hooks, pages, AJAX, and base APIs
Blocksy	Parent theme and structure on which blocksy-child works
blocksy-child	HUD, factions, multimedia, Shop/Cart/Checkout customization, gateway, logs, emails, Gallery, and Services
WooCommerce	WC_Product, cart, session, Checkout, WC_Order, gateways, and HPOS
ACF	Custom product metadata such as video_destacado, tiene_globo, and productos_del_bundle
MariaDB	Products, options, sessions, orders, and persistent metadata
wp-content/uploads + server	Physical files, logs, and public paths/multimedia mappings
WP Mail SMTP / SMTP	Transport after wp_mail() calls

40.2 Two journeys that summarize the project

FIRST VISIT

URL → WordPress → template_redirect → presentation/Intro → NOD/GDI selection → faction cookie → faction CSS/resources → menu/Home

PURCHASE

WC_Product → Cart item → customizations → Checkout → WC_Order/WC_Order_Item → manual_pay → log → buyer email → admin email → return URL → Order Received

If a reader can follow both journeys and, at every transition, identify the file, callback, input data, and result, the documentation fulfills its teaching objective: explaining not only what the user sees, but how WordPress, WooCommerce, the child theme, plugins, database, and physical resources cooperate.

40.3 Information sources used

Source	Use in this document
Source code of the blocksy-child child theme	Implementation of PHP, JavaScript, CSS, hooks, routes, and custom site behavior.
WordPress, WooCommerce, and ACF configuration and data	Catalog structure, orders, custom fields, and functional environment configuration.
Project operation screenshots	Visible walkthrough of the visitor experience and verification of the main screens.
Project base technical documentation	Prior information about dependencies, database, installation, and environment characteristics.

40.4 Conclusion

MC Artworks Studio combines WordPress and WooCommerce with its own audiovisual layer developed on blocksy-child. The project does not replace WordPress systems: it relies on their hooks, pages, plugins, APIs, and storage to build a NOD/GDI experience, a multimedia catalog, and a customized purchase flow.

The main complexity lies in the connections between layers: PHP generates structure and scripts; the browser executes events and observers; ACF relates products to resources; WooCommerce retains the cart and orders; HPOS persists WC_Order; the manual gateway logs and communicates the purchase; uploads and the server deliver multimedia resources; SMTP transports the emails.

The main maintenance points identified are the concentration of functions.php, duplicate registrations, overlapping cache strategies, unpinned external dependencies, environment-bound paths, multimedia loading costs, and operational log controls. Documenting them makes it possible to evolve the project without confusing current behavior with intended behavior.